軟體工程

第四課：系統塑模與統一架構 (System Modeling & Unified Architecture)

授課教師：薛念林 教授

資訊工程學系

逢甲大學

第四章：課程藍圖與核心架構

第一部分：塑模基礎與核心靜態模型

- **4.1 系統塑模基礎：** 塑模本質、四大核心視角、五大必備圖表與盲人摸象寓言。
- **4.2 統一塑模語言 (UML)：** 1990 年代方法論之戰、三巨頭三位一體分工與 OMG 標準化。
- **4.3 功能塑模與使用案例：** 參與者目標、系統邊界、include/extend 關聯與 AI 提示詞指引。
- **4.4 結構塑模與領域類別圖：** 領域實體、可見度符號、關聯重數、整體與部分及 AI 提示詞。
- **4.5 互動塑模與循序圖：** 訊息傳遞時序、生命線、啟動條、BCE 模式與 AI 提示詞指引。

第二部分：工作流程、行為狀態與文字塑模

- **4.6 流程塑模與活動圖：** 並行流程、決策分支、分岔/結合、泳道職責與 AI 提示詞。
- **4.7 行為塑模與有限狀態機：** 反應型系統、訂單生命週期狀態機、事件觸發、守衛條件與 AI 提示詞。
- **4.8 PlantUML 宣告式文字塑模：** 架構即代碼 (Code-as-Architecture)、語法速查與多圖管線。
- **4.9 概念複習與統整：** 核心工程原則回顧、填空小測驗與經典文獻。

4.1 系統塑模基礎

> *"A language that doesn't affect the way you think about programming is not worth knowing."*
> *— Alan Perlis*

什麼是系統塑模？ (What is System Modeling?)

> *"系統塑模是為系統開發抽象模型的過程，每個模型代表該系統不同的視角或觀點。"*
> *— Ian Sommerville, Software Engineering (10th ed.)*

- **系統塑模的核心工程原則：**
  - **抽象化 (Abstraction)：** 隱藏不必要的實作細節，突顯關鍵的架構結構、資料流動與相依關係。
  - **多重視角 (Multiple Perspectives)：** 沒有任何單一圖表能解釋整個複雜軟體；不同利害關係人需要不同的視角。
  - **溝通與驗證 (Communication & Verification)：** 作為產品經理、系統架構師、開發者與測試工程師之間的通用視覺語言。
  - **實作藍圖 (Blueprint for Implementation)：** 提供正式的結構與行為規格，作為後續編寫程式碼與資料庫結構的精確藍圖。

盲人摸象與多重視角模型 (The Parable of the Elephant & Multiple Perspectives)

🦏 寓言啟示：盲人摸象與系統塑模的「多重視角」必要性

① 摸到象鼻：「像一條巨蟒 / 水管！」

🎯 **互動視角 (Interaction Perspective)**

- 關注：訊息傳遞順序、呼叫流向與動態協同
- 盲點：忽視靜態資料結構與內部持久化

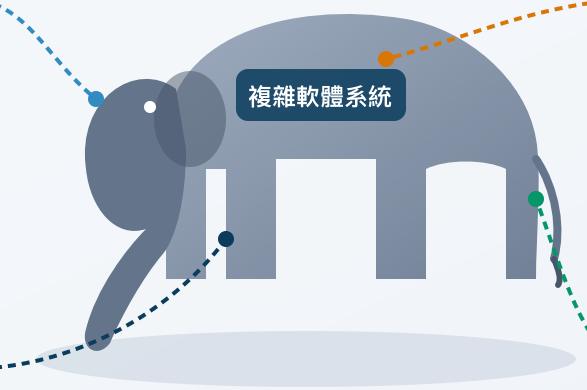
對應 UML：循序圖 (Sequence Diagram)

② 摸到象腿：「像一根堅硬巨柱！」

🏛️ **結構視角 (Structural Perspective)**

- 關注：實體資料屬性、關聯、組合與靜態骨幹
- 盲點：看不出系統隨時間演進的動態行為

對應 UML：類別圖 (Class Diagram)



③ 摸到象身：「像一面高大磚牆！」

🛡️ **外部視角 (External / Context)**

- 關注：系統邊界、運行環境與外部第三方 API
- 盲點：無法得知內部核心邏輯如何拆解實現

對應 UML：環境圖 (Context Diagram)

④ 摸到象尾：「像一條擺動細繩！」

⚡ **行為視角 (Behavioral Perspective)**

- 關注：業務流程流轉、外在事件刺激與狀態轉換
- 盲點：無法單獨反映靜態物件職責與系統邊界

對應 UML：活動圖 / 狀態圖 (Activity / State)

💡 結論：任何單一模型皆為「局部投影」；唯有綜合不同視角，才能完整還原軟體架構全貌！

系統塑模的 4 大核心視角 (4 Core Perspectives)

- **1. 外部視角 (External Perspective)：**
  - 塑模系統處在的運行環境、外部合作夥伴與邊界範圍。
  - 明確劃分系統邊界：哪些由內部團隊開發，哪些委託給外部第三方服務。
- **2. 互動視角 (Interaction Perspective)：**
  - 塑模外部參與者與系統之間，或是內部協同物件之間的動態溝通與訊息傳遞。
- **3. 結構視角 (Structural Perspective)：**
  - 塑模系統資料組織、類別、屬性、方法與關聯性的靜態架構，與執行時間先後無關。
- **4. 行為視角 (Behavioral Perspective)：**
  - 塑模系統的動態執行行為、業務流程步驟，以及系統針對外在事件所產生的反應式狀態轉換。

現代實務必備的 5 大 UML 核心模型

模型 / 圖表	對應章節	所屬視角	動態 / 靜態本質	主要軟體工程職責
1. 使用案例模型	4.3	互動視角	功能契約	界定系統邊界、參與者目標與系統功能範疇
2. 領域類別圖	4.4	結構視角	靜態骨幹	定義領域實體、屬性、方法與整體/部分關聯
3. 物件循序圖	4.5	互動視角	動態時序	沿時間軸追蹤訊息傳遞與 BCE 強韌性職責劃分
4. 業務活動圖	4.6	行為視角	動態流程	塑模業務流程、並行分岔/結合與跨角色泳道
5. 有限狀態機圖	4.7	行為視角	反應生命週期	捕捉事件驅動的離散狀態流轉、守衛與不變量

> *落實與加速： 透過 **4.8 PlantUML** (架構即代碼) 統一實踐，各章節深度整合專屬 **AI 提示詞指引**。*

觀念檢核測驗 1 (CCQ 1)

軟體架構師若欲定義「**領域實體資料的靜態組織方式，以及類別之間的繼承、關聯與包含關係**」（完全不隨執行時序變動），應採取哪一種塑模視角？

- **A.** 外部視角 (External Perspective)
- **B.** 互動視角 (Interaction Perspective)
- **C.** 結構視角 (Structural Perspective)
- **D.** 行為視角 (Behavioral Perspective)

4.2 統一塑模語言 (UML)

> *「標準化的視覺語法，將軟體工程從封閉的方法論孤島，淬煉為全球共通的工程學門。」*

標準化的迫切需求：1990 年代「方法論之戰」

- **物件導向程式設計的崛起 (1980 年代末至 1990 年代初)：**
  - 軟體產業從程序導向程式碼 (C, Pascal) 大規模轉移至物件導向典範 (C++, Smalltalk)。
  - 工程師迫切需要一套圖形符號來視覺化表達類別、物件與關聯。
- **百家爭鳴的「方法論之戰 (Method Wars)」時代：**
  - 超過 **50 種相互競爭的物件導向塑模記號**充斥商業市場。
  - 各派大師激烈爭論：類別究竟該畫成雲朵、矩形還是橢圓？繼承到底該用空心三角、實心箭頭還是虛線？
  - **嚴重的產業割裂碎片化：** 各公司之間無法交換架構模型，CASE 塑模軟體彼此互不相容，工程師換工作就得被迫重新學習一套新符號。

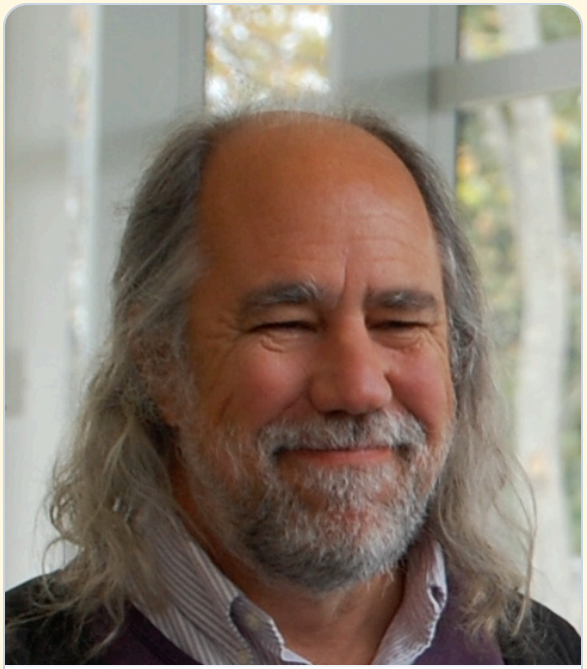
統一與標準化：從 Rational 到 OMG

- **1994 – Rational 展開歷史性大整合：**
  - Jim Rumbaugh 加入 Rational 與 Grady Booch 聯手，融合 Booch 方法與 OMT 為 Unified Method (v0.8)。
- **1995 – 三巨頭 (The Three Amigos) 合體：**
  - Ivar Jacobson 加入 Rational，帶入革命性的**使用案例 (Use Case)** 方法論 (OOSE)。
- **1997 – OMG 國際官方標準化：**
  - 正式提交至**物件管理組織 (OMG)**，於 1997 年 11 月一致通過採納為 **UML 1.1** 國際產業標準。
- **2005 – UML 2.0 重大架構改版：**
  - 擴充至 13 種（後續擴增至 14 種）正式圖表，具備形式化執行元模型 (Metamodels)。

UML 奠基先驅：Grady Booch

- **角色與學術榮譽：**
  - ACM Fellow、IEEE Fellow、IBM 院士、Rational 共同創辦人兼首席科學家。
- **開創經典方法論：**
  - **Booch 方法 (Booch Method)** 與重量級專書：《物件導向分析與設計》。
- **對 UML 的核心貢獻：**
  - 專精於**物件導向實作設計 (Design & Implementation)**，精確定義微觀物件協同與宏觀系統架構。
  - 賦予 UML 精準對應至 C++、Java 等 OOP 程式語言的實作直譯能力。
- **著名軟體工程箴言：**

> *"良好的架構，是讓未來的重大決策顯得容易且自然的藝術。"*

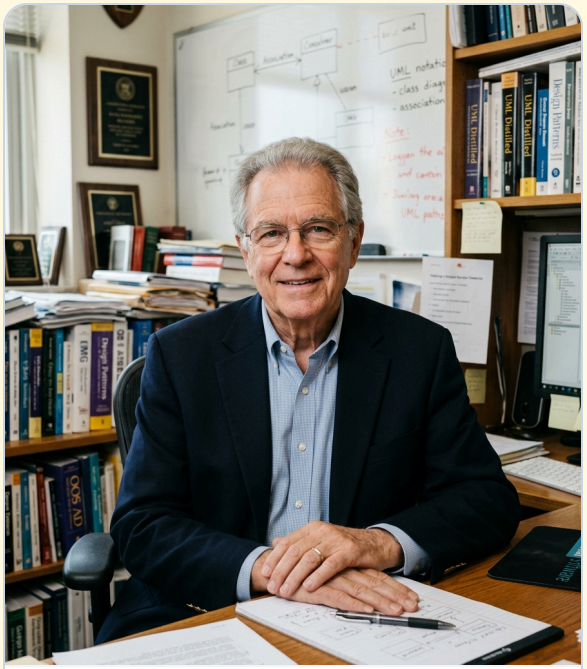


Grady Booch
Rational Software / IBM Fellow

UML 奠基先驅：James Rumbaugh

- **角色與學術榮譽：**
  - 奇異公司 (GE) 全球研發中心軟體主任研究員、Rational Software、IBM。
- **開創經典方法論：**
  - **物件塑模技術 (OMT, Object Modeling Technique)** 與專書：《物件導向塑模與設計》。
- **對 UML 的核心貢獻：**
  - 強調嚴謹的**領域分析 (Domain Analysis)**、語義資料塑模與實體關聯對應。
  - 首創將靜態物件結構與 David Harel 的**狀態圖 (Statecharts)** 深度結合，用以塑模動態反應式系統。
- **著名軟體工程箴言：**

> *"如果不透徹理解領域的真實本質，就絕不可能打造出卓越的軟體。"*



James Rumbaugh
GE Research / Rational Software

UML 奠基先驅：Ivar Jacobson

- **角色與學術榮譽：**
  - 愛立信 (Ericsson) 首席架構師、Objectory AB 創辦人、Rational Software、SEMAT 發起人。
- **開創經典方法論：**
  - **物件導向軟體工程 (OOSE)**。
- **對 UML 的核心貢獻：**
  - **使用案例 (Use Cases) 的發明者 (1986)：** 革命性地將系統架構錨定在可衡量的使用者目標之上。
  - 提出經典的**邊界–控制–實體 (BCE, Boundary–Control–Entity)** 強韌性分析模式與元件化架構。
- **著名軟體工程箴言：**

> *"一個沒有使用者的系統，就沒有任何存在的理由。塑模永遠要先從使用者的目標開始。"*

Ivar Jacobson
Ericsson / Objectory / Rational

UML 三巨頭：軟體塑模的三位一體 (The Trinity of Modeling)

Ivar Jacobson

核心回答「Why」(目標 / 使用者)

- **使用案例驅動：** 外部參與者想達成什麼目標？
- **BCE 強韌性分析：** 徹底分離 UI 邊界與核心資料實體。
- **元件契約規範：** 可重用子系統的架構邊界。

James Rumbaugh

核心回答「What」(領域與資料)

- **領域物件分析：** 現實世界存在哪些核心概念？
- **類別與資料綱要：** 結構化的實體關聯關係。
- **有限狀態機：** 反應式事件驅動的生命週期。

Grady Booch

核心回答「How」(設計與程式碼)

- **物件導向設計：** 類別之間如何精巧協同運作？
- **巨觀架構拆解：** 分層模組與相依性控制。
- **實作直譯對應：** 視覺模型與 OOP 程式碼直接對齊。

> 📌 *架構整合精髓：Jacobson 捕捉系統存在的初衷目的 **(Why)**；Rumbaugh 塑模領域存在的資料本質 **(What)**；Booch 設計程式執行的實作架構 **(How)**。*

觀念檢核測驗 2 (CCQ 2)

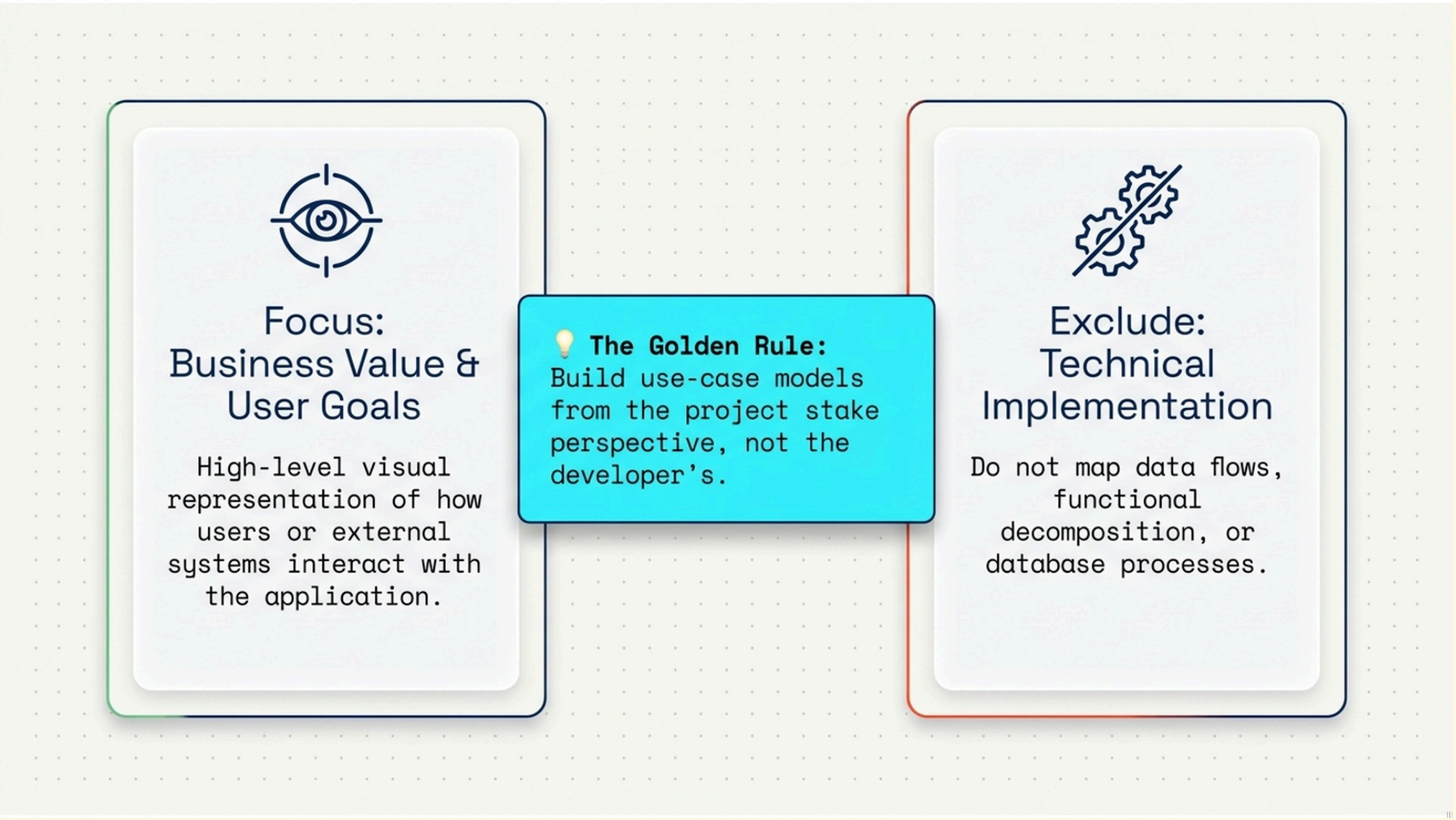
在被尊稱為「UML 三巨頭」的軟體工程大師中，哪一位以於 1986 年發明**使用案例 (Use Cases)**、將軟體架構直接錨定於使用者具體目標而聞名？

- **A.** Grady Booch
- **B.** James Rumbaugh
- **C.** Ivar Jacobson
- **D.** Martin Fowler

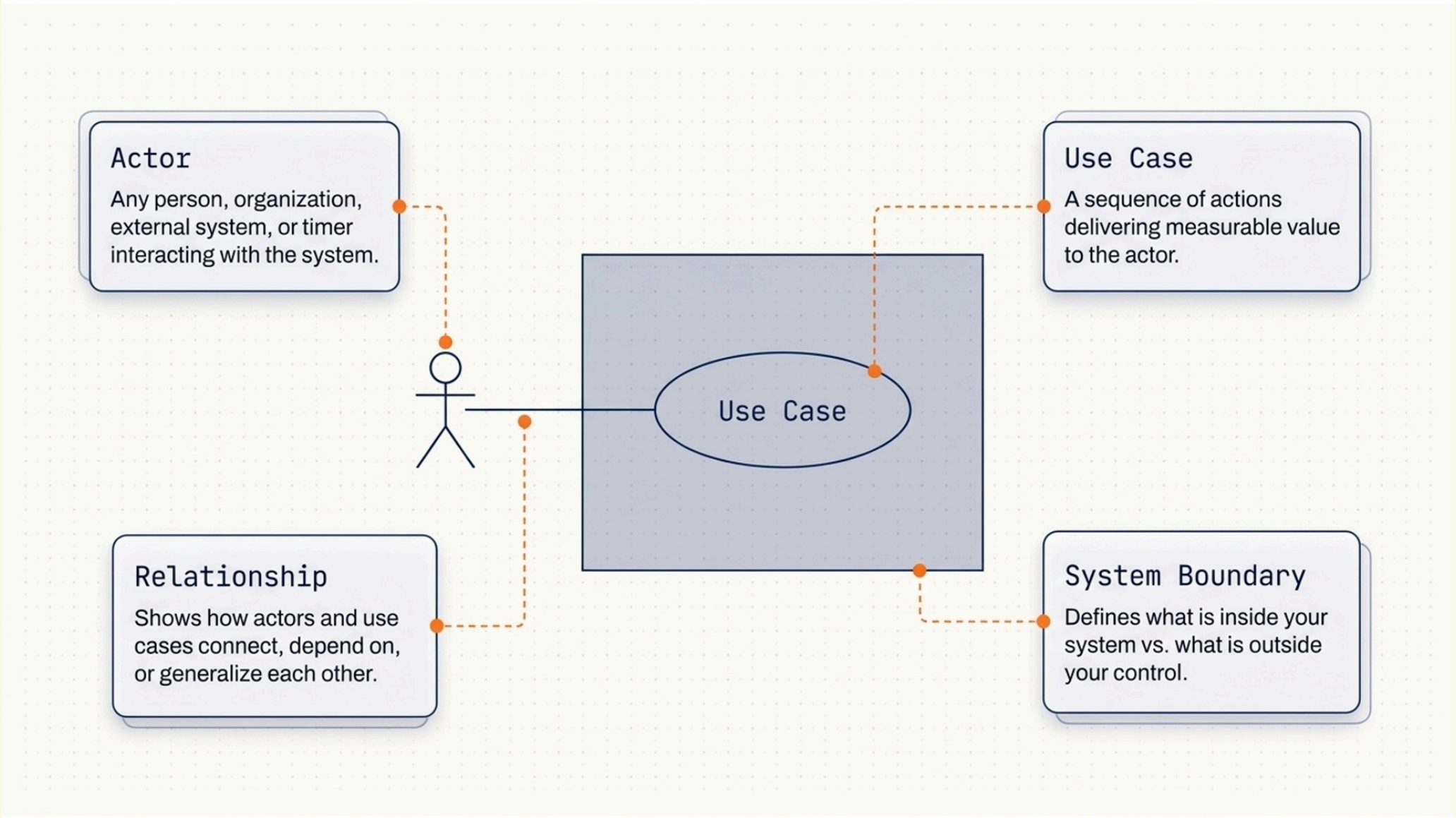
4.3 功能與使用案例模型

> *"使用案例是系統與其參與者之間為了達成可衡量業務目標所簽署的行為契約。"*
> *— Alistair Cockburn*

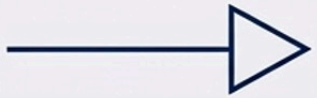
業務價值與使用者目標導向 (Business Value & User Goals)



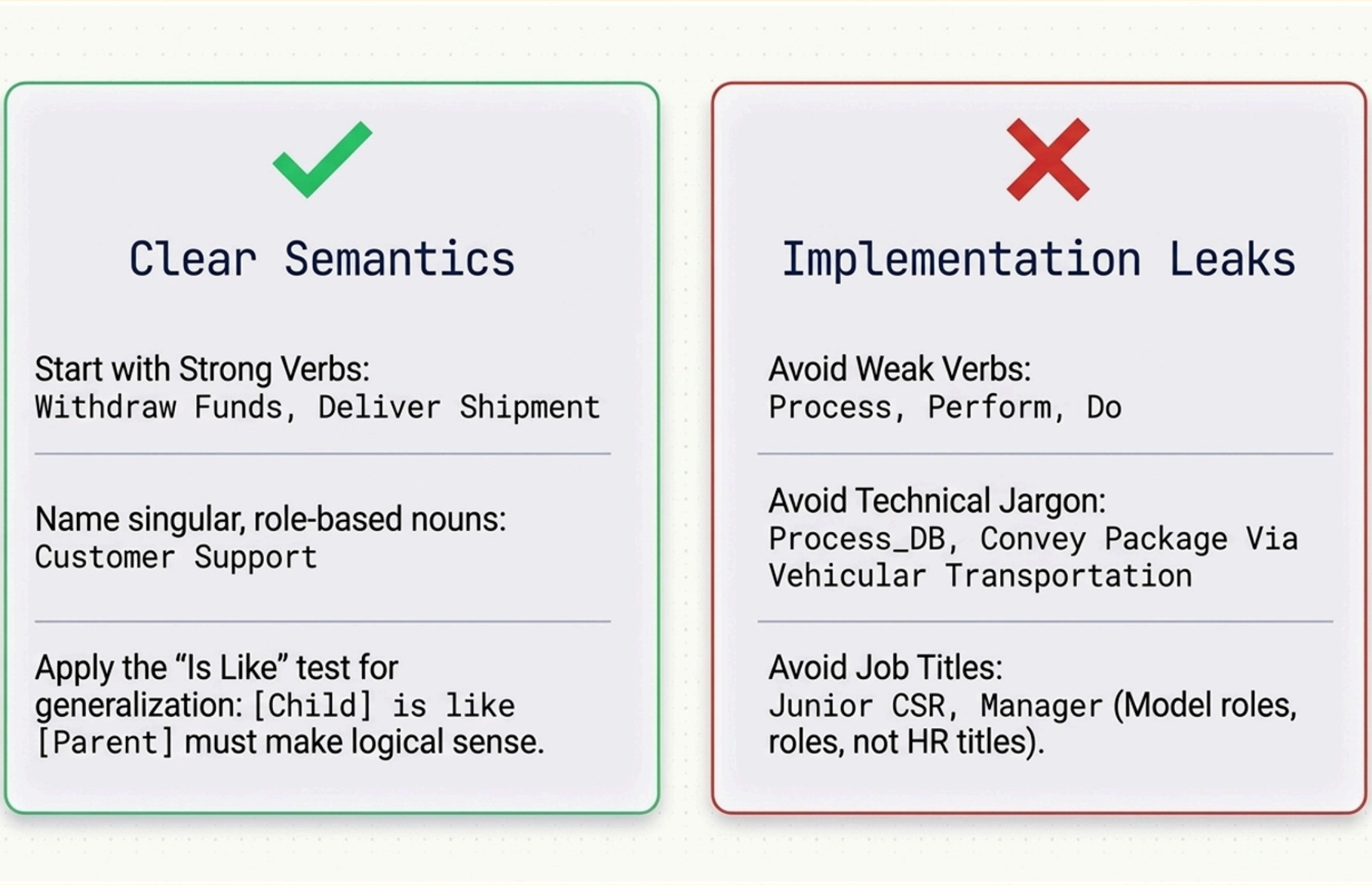
參與者分類學與系統邊界 (Actor Taxonomy & Boundaries)



使用案例關聯矩陣 (Use Case Relationship Matrix)

Relationship	UML Notation	Behavior	Real-World Analogy
Association		Actor participates in a use case.	A customer walks into a store.
<<include>>		Mandatory, reusable logic. Runs at a specific point.	Calling a function: `checkout()` → `calculateTax()`
<<extend>>		Optional/conditional logic. Might run during certain steps.	A pop-up coupon or conditional error check.
Generalization		Is a relationship. Child inherits logic from parent.	International Student is like a Student, but with extra steps.

清楚的使用案例命名語意 (Clear Use Case Naming)



使用案例塑模核心法則 (Rules of Building Use Case Models)

NESTING LIMIT

Max 2 Levels of Nesting

Avoid functional decomposition (e.g., A includes B includes C).
Keep it requirements-focused, not design-focused.

ACTOR INTERACTIONS

No Actor-to-Actor Lines

Actors do not interact directly on the diagram.
Detail these interactions in your text specifications.

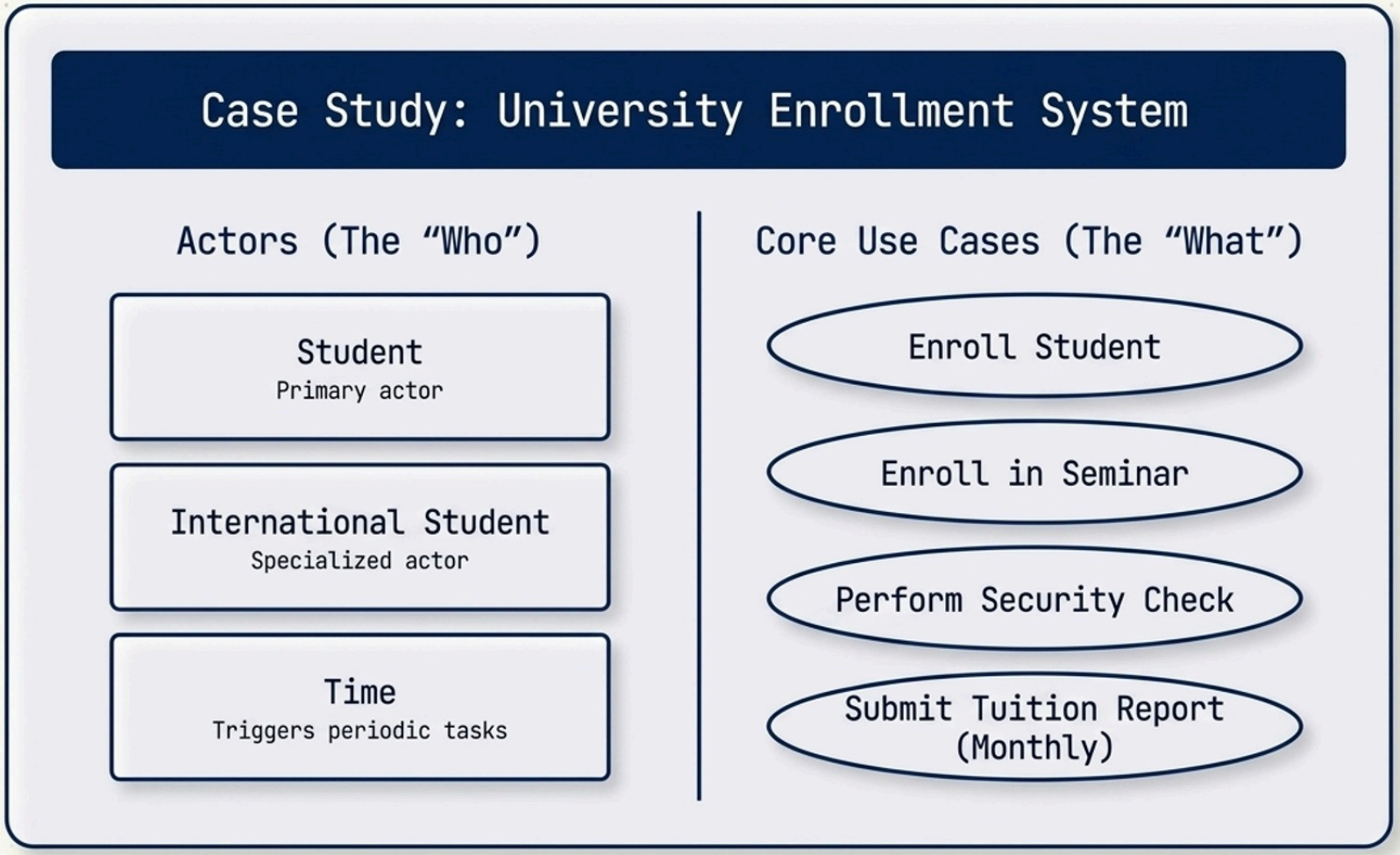
SYSTEM BOUNDARIES

Use Boundaries with Purpose

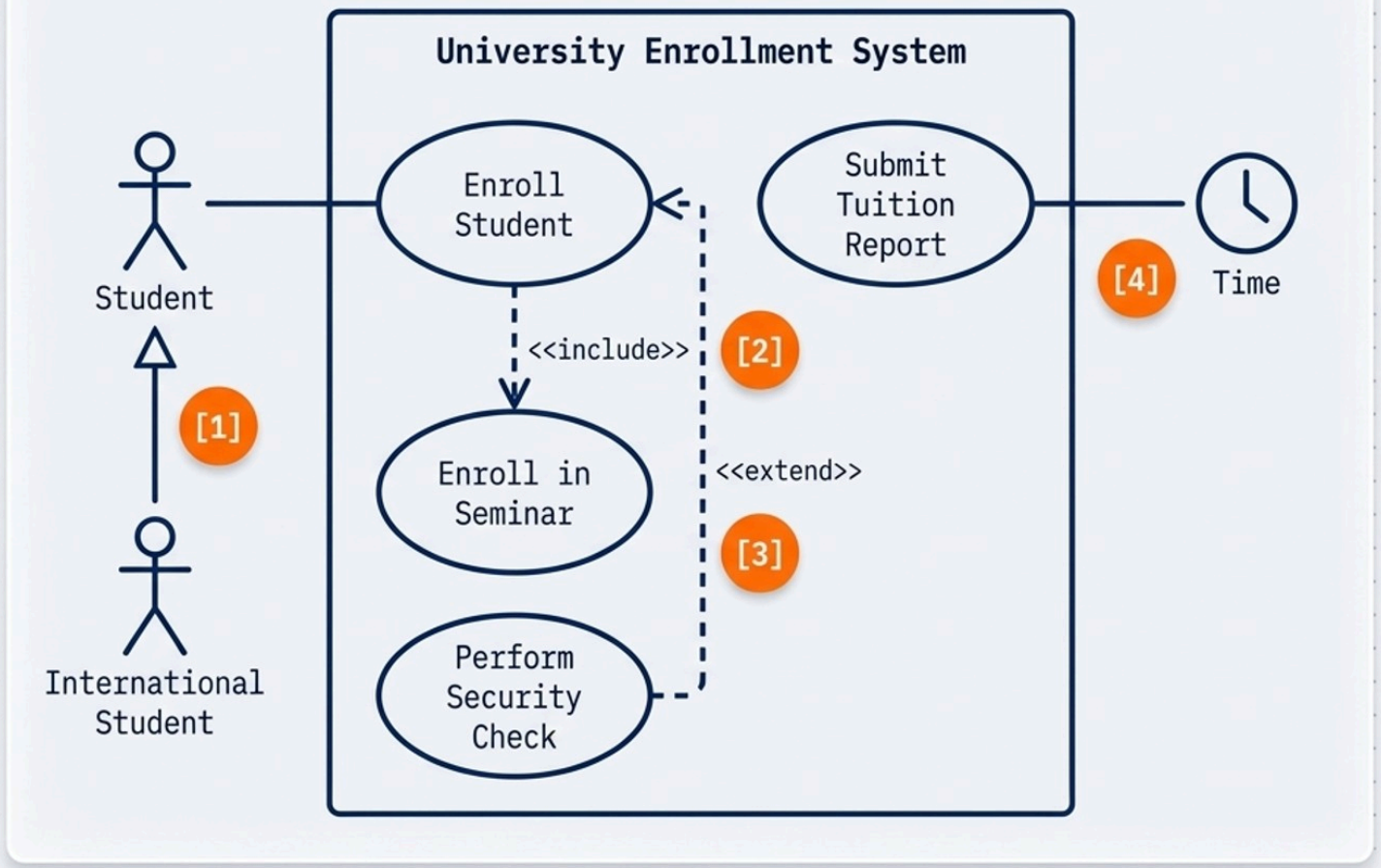
Only use the boundary box to define system scope or distinct release phases (Phase 1, Phase 2). Omit if redundant.

Note: Call out external systems with <<system>> and use a Time actor (clock icon) for scheduled triggers.

實戰案例：大學選課系統 (University Enrollment System)



實戰案例：參與者泛化與關聯整合 (Actor Generalization & Synthesis)



Blueprint Legend

[1] Generalization:
Int'l Student inherits base student logic.

[2] <<include>>:
Enrolling always requires a seminar (Mandatory).

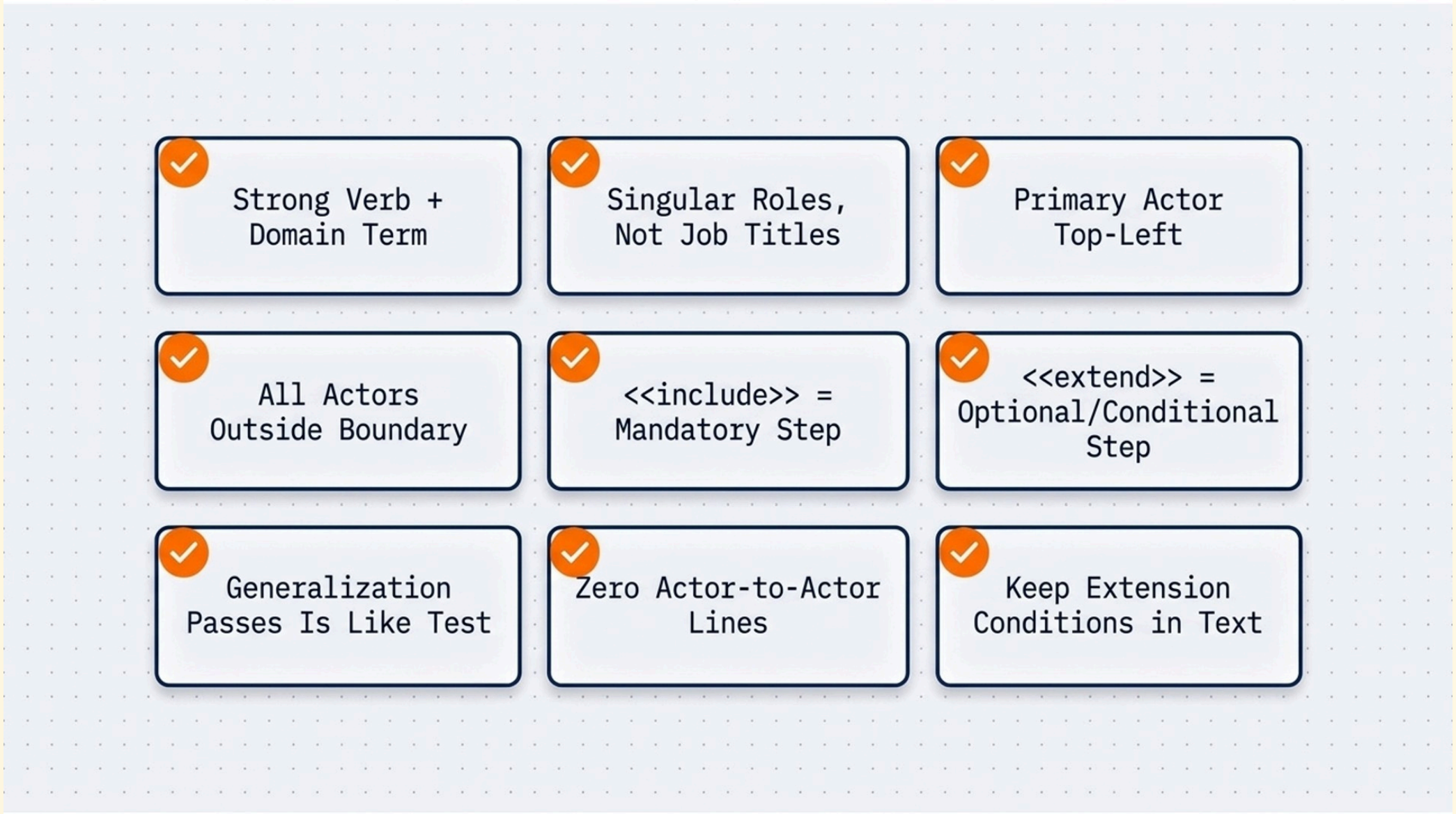
[3] <<extend>>:
Security check only runs for certain students (Conditional).

[4] Time Actor:
Triggers the monthly tuition report.

使用案例常見反模式 (Common Anti-Patterns)

The Mistake (Symptom)	Why It Fails (Diagnosis)	The Fix (Cure)
✗ Technical names (`Process_DB_Transaction`)	Stakeholders don't understand implementation details.	✓ Use domain-friendly names (`Withdraw Funds`, `Generate Report`).
✗ Drawing arrows showing data flow	Associations show initiation, not data movement.	✓ Remove arrowheads unless showing passive actors.
✗ Overusing `<<extend>>` lines	Creates messy, unreadable diagrams.	✓ Move optional/conditional logic into textual use-case specs.
✗ Connecting Customer to Admin directly	Actors don't interact on use-case diagrams.	✓ Describe human interaction in the text scenario.

使用案例最佳實踐檢核清單 (Best Practices Checklist)



使用案例規格書：UC-01 學生選課 (Enroll Student)

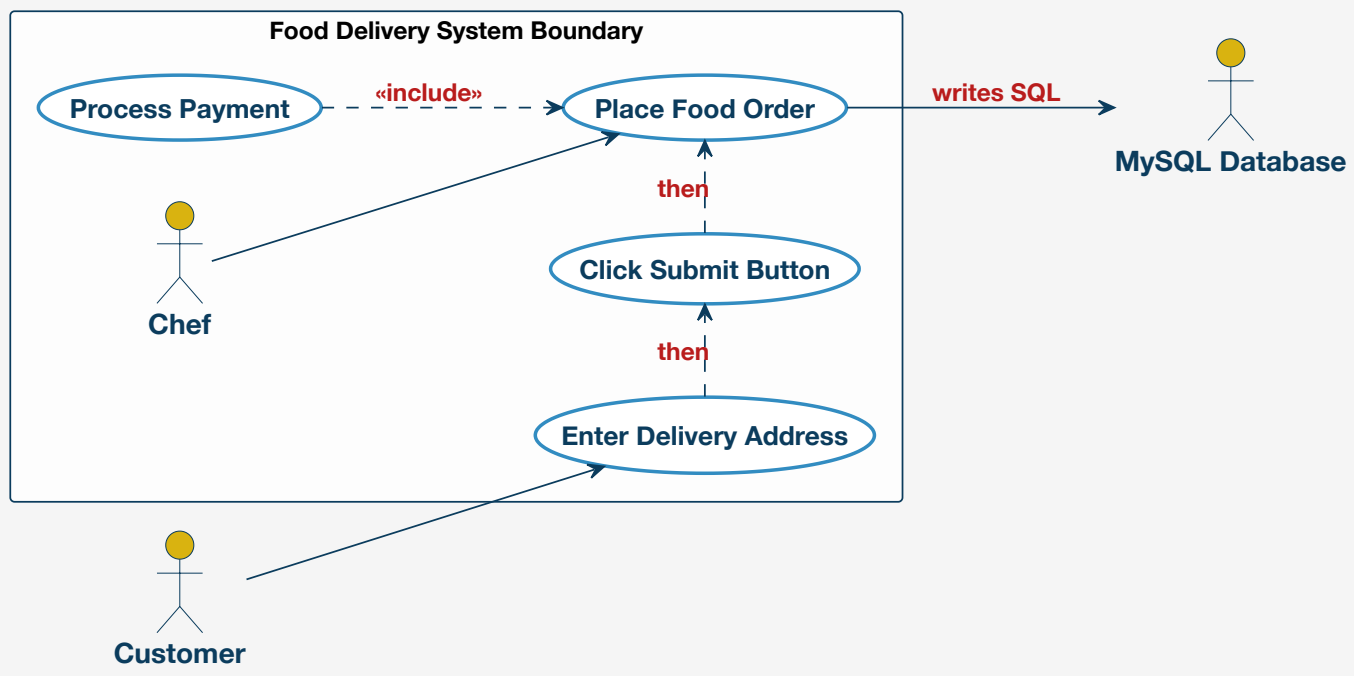
欄位名稱	規格細節說明
使用案例代號 / 名稱	**UC-01：學生選課 (Enroll Student in Course)**
主要參與者 (Actor)	學生 (透過學校單一登入 SSO 驗證身分)
前置條件 (Preconditions)	學生具備有效學籍在學狀態；當前處於選課開放時段內
後置條件 (Postconditions)	學生正式納入該課程名冊；該課程剩餘修課名額自動扣減 1
主要成功路徑 (Main Flow)	**1.** 學生依學系代碼或開課課號搜尋開放修習的課程清單。 **2.** 系統回傳並展示符合條件的班級資訊、上課時間表與剩餘名額。 **3.** 學生選定目標課程，向系統提出正式加選請求。 **4.** 系統執行 `<<include>> 檢查先修科目資格`，比對學生歷史歷年成績單。 **5.** 系統鎖定保留名額，寫入選課交易紀錄，並產生成員確認通知單。 **6.** 學生即時檢視個人更新後的課表，並收到選課成功通知信。
例外與擴充路徑 (Extensions)	**4a. 先修科目未通過：** 系統提示未修畢之必要擋修科目代碼，中斷選課流程。 **4b. 上課時間衝堂：** 系統發出警告，提示與已選課程時間重疊，放棄本次加選。 **5a. 課程名額已滿：** 系統觸發 `<<extend>> 加入候補等候名單` 流程。

觀念檢核測驗 3 (CCQ 3)

在 UML 使用案例模型中，若使用案例 A 與使用案例 B 之間存在 `<<include>>` 關聯（A 指向 B），這在軟體架構上代表什麼意義？

- **A.** 使用案例 B 是選擇性的，僅在特定異常狀況下才會由 A 觸發執行。
- **B.** 每當使用案例 A 執行時，使用案例 B 所定義的行為「必定且強制」會被納入執行。
- **C.** 使用案例 A 繼承了使用案例 B 的所有屬性與操作方法。
- **D.** 使用案例 B 是外部第三方系統，不屬於軟體邊界範疇。

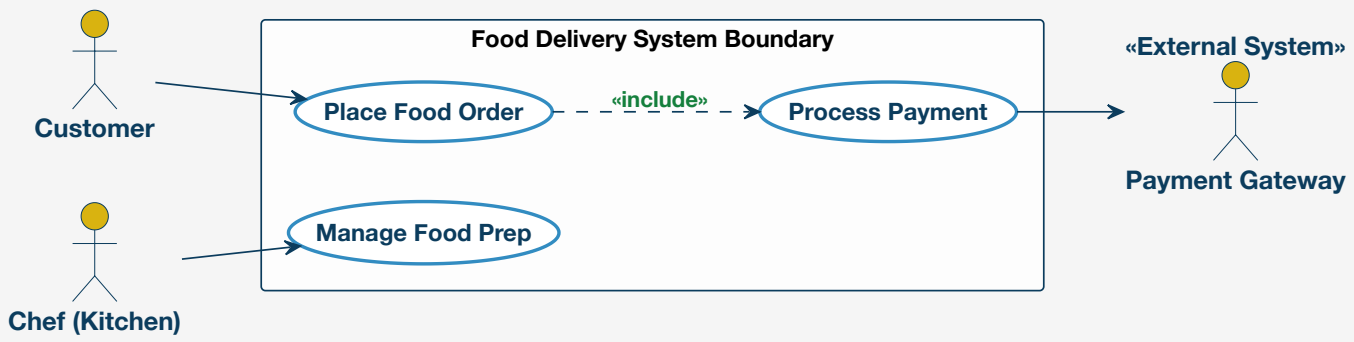
觀念診斷練習：找出這張使用案例圖的 3 大致命錯誤！



審查這份學生繪製的使用案例圖草稿：
你能找出圖中潛藏的 **3 大關鍵塑模謬誤（反模式）** 嗎？

- **1. 顆粒度與操作流程分解 (Granularity & Workflow)：**
  - 將 `輸入外送地址` ➔ `點擊送出按鈕` ➔ `送出外送訂單` 串聯成流程有何嚴重錯誤？
- **2. 關聯方向倒置 (Relationship Direction)：**
  - 仔細觀察 `處理線上扣款` 與 `送出外送訂單` 之間的 `<<include>>` 虛線箭頭方向！
- **3. 系統邊界與內部實體錯置 (Boundary & Actor Placement)：**
  - 檢視 `主廚 (Chef)` 與 `MySQL 資料庫`，兩者如何違反了 UML 系統邊界與參與者定義？

觀念診斷解答：使用案例圖 3 大錯誤剖析與修正



審查診斷結論與架構修正指引：

- **1. 顆粒度太細與流程圖化 (Functional Decomposition)：**
  - 「輸入地址」與「點擊送出」僅是「下單」內部的微小 UI 步驟，不可獨立畫為使用案例。
- **2. 包含關聯方向顛倒 (Inverted <>)：**
  - 基礎案例主動調用共用子程序，箭頭必須由 `下單` 指向 `處理付款`（`Place Food Order` ➔ `Process Payment`）。
- **3. 系統邊界與參與者錯置 (Boundary Violations)：**
  - `Chef` 是人員角色，必須置於邊界方框外；`MySQL Database` 是系統內部儲存，不可畫為外部參與者。

AI 輔助指南：使用案例塑模 (Use Case Modeling)

- **1. 角色設定與系統邊界約束 (Persona & Boundary Priming)：**
  - 指示 AI 扮演「精通 UML 2.5 國際標準的資深需求分析師」。
  - **嚴格要求宣告邊界：** 必須先定義 `rectangle "System Name"`，禁止 AI 將外部雲端服務混入內部邊界。
- **2. 參與者與關聯性嚴格語義規範：**
  - **參與者分類：** 要求 AI 區分「主要參與者 (人)」與「次要外部支援服務 (金流 API、簡訊服務)」。
  - `<<include>>` **規則：** 僅用於「每次下單必執行的強制共用子流程」。
  - `<<extend>>` **規則：** 僅用於「具備特定觸發條件的可選附加行為」，並指明擴充點。
  - **反模式防護：** 嚴禁功能分解（禁止產生「點擊按鈕」、「輸入密碼」等瑣碎泡泡）。
- **3. 架構師人工審查清單 (Verification Checklist)：**
  - [ ] 每個使用案例是否皆以「動詞 + 受詞」命名（如 `下單訂購`、`追蹤外送`）？
  - [ ] 第三方基礎設施（金流閘道、簡訊）是否正確置於系統邊界外側？
  - [ ] include 箭頭是否正確指向被包含的共用功能？

AI 提示詞範例：外送平台使用案例模型

1. 角色設定與任務指令：

> 你是一位資深系統分析架構師。
> 請根據以下需求描述，產生合規且簡練的
> `PlantUML` 使用案例圖與使用案例規格書。

2. 塑模約束條件：

- 使用 `<<extends>>` 與 `<<includes>>` 建構模組化關係。
- 嚴禁功能分解（禁止出現「點擊按鈕」、「輸入密碼」等瑣碎用例）。
- 以 `rectangle "校園外送平台"` 劃定系統邊界。
- 外部金流閘道與簡訊服務置於邊界外作為次要參與者。

3. 輸入需求敘述 (Requirements Statement)：

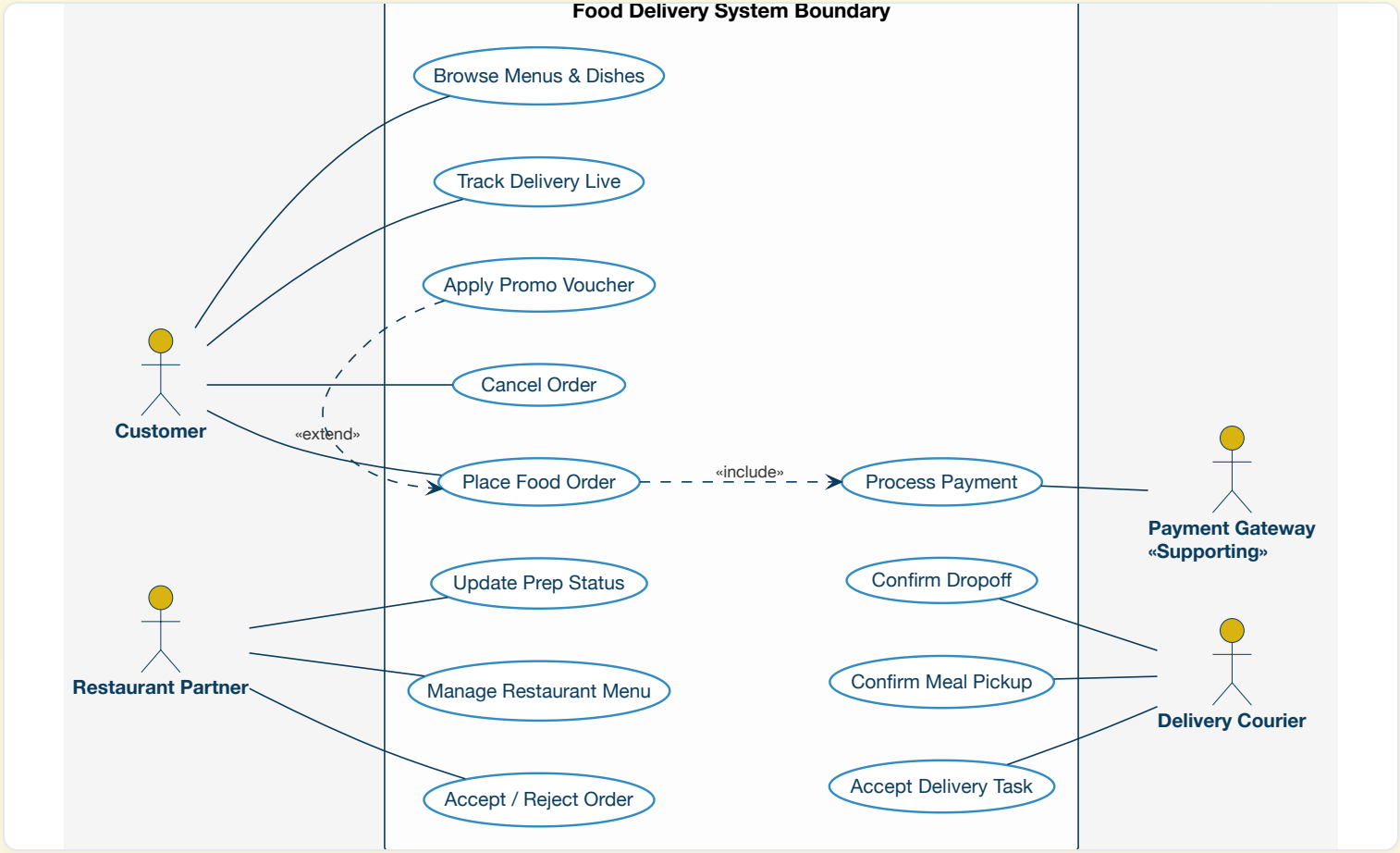
> *「顧客可以在平台瀏覽餐廳菜單、將餐點加入購物車並進行結帳下單。*
> *顧客下單時，系統必須強制連線外部**金流閘道**授權扣款，並驗證外送地址有效性。*
> *結帳時，顧客可以自由選擇是否套用優惠券代碼（`套用折扣優惠`），或勾選放置門口的無接觸配送。*
> ***餐廳員工**可接收並審核進單、更新廚房備餐進度。*
> ***外送員**可承接外送任務、回報餐點取件與最終送達。*
> *當訂單狀態變更時，系統會自動呼叫外部**簡訊服務**向顧客發送即時通知。」*

AI 生成之 PlantUML 與架構圖：外送平台使用案例模型

AI 生成之 PlantUML 規格代碼：

```
@startuml
left to right direction
actor Customer
actor "Restaurant Partner" as Restaurant
actor "Payment Gateway\n«Supporting»" as Payment
actor "Delivery Courier" as Courier

rectangle "Food Delivery System" {
  usecase "Browse Menus" as UC_Browse
  usecase "Place Food Order" as UC_PlaceOrder
  usecase "Apply Promo Voucher" as UC_Voucher
  usecase "Track Delivery Live" as UC_Track
  usecase "Manage Menu" as UC_Menu
  usecase "Accept / Reject Order" as UC_AcceptOrder
  usecase "Process Payment" as UC_Payment
  usecase "Accept Delivery Task" as UC_AcceptTask
}

Customer -- UC_Browse
Customer -- UC_PlaceOrder
Customer -- UC_Track
Restaurant -- UC_Menu
Restaurant -- UC_AcceptOrder

UC_PlaceOrder ..> UC_Payment : <<include>>
UC_Voucher .> UC_PlaceOrder : <<extend>>
UC_Payment -- Payment
```

渲染產出之架構圖：



課堂活動：使用 AI 演進使用案例 (雙人動手實作)

動手實作實驗：變更業務需求並審核 AI 產出 (4 分鐘)

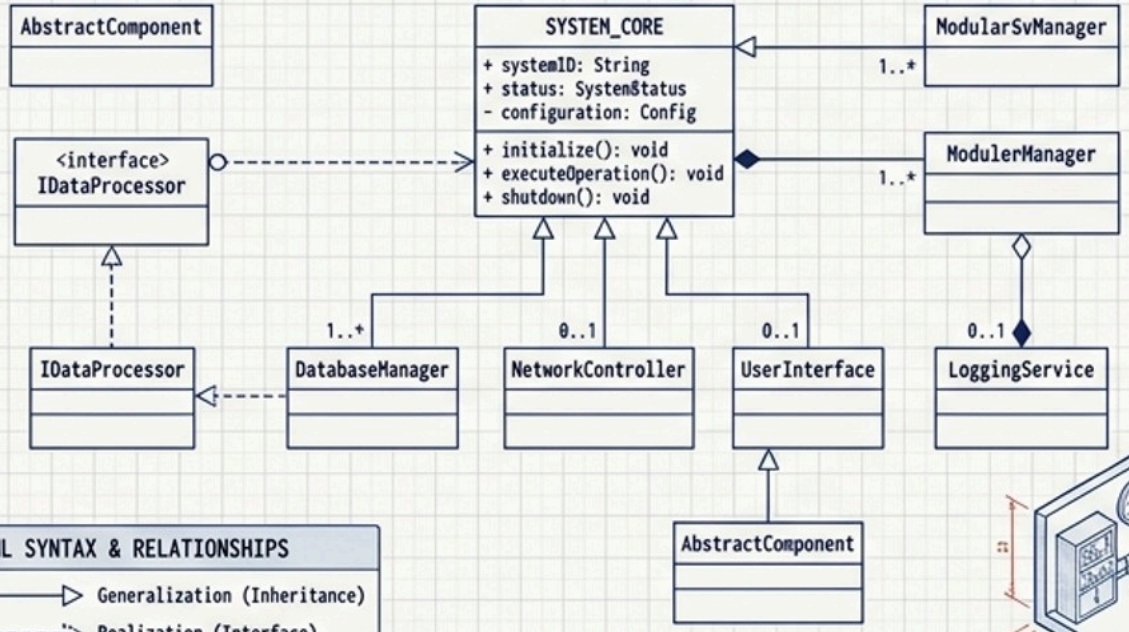
- **1. 在使用案例提示詞中加入一項新需求功能：**
  - *預約定時下單：* 允許 `顧客` 預約未來特定時段的外送服務。
  - *線上即時客服：* 建立 `顧客` 與 `客服人員` 或 `外送員` 的即時對談。
  - *揪團共同訂餐：* 允許多位使用者將餐點加入同一筆匯總購物車。
- **2. 要求 AI 重新生成：** 提示 LLM 更新對應的 PlantUML 使用案例代碼。
- **3. 渲染並嚴格審查：** 貼入 PlantText 渲染並審查：
  - AI 是否將外部系統/金流閘道保持在系統邊界**矩形之外**？
  - AI 是否正確區分**核心必經步驟** (`<<include>>`) 與**可選延伸功能** (`<<extend>>`)？
  - AI 是否成功避開了點擊按鈕等微觀 UI 反模式（例如：避免出現「點擊確認」）？

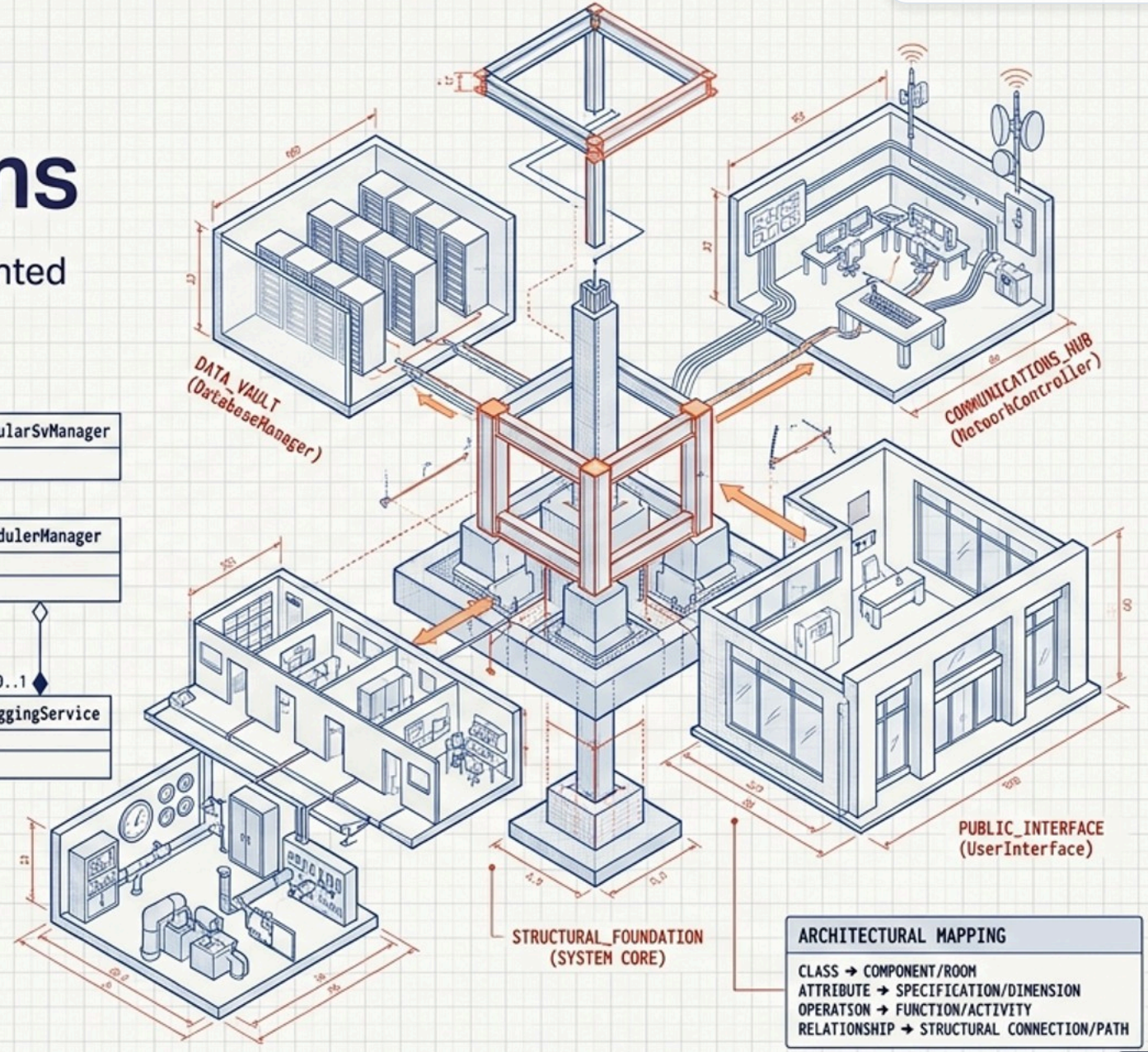
4.4 結構與領域類別模型

> *"類別圖是軟體系統的靜態建築骨幹；如果骨幹畸形，任何敏捷開發都拯救不了系統的崩潰。"*

The Anatomy of UML Class Diagrams

A visual reference guide to mapping object-oriented systems, syntax, and relationships.





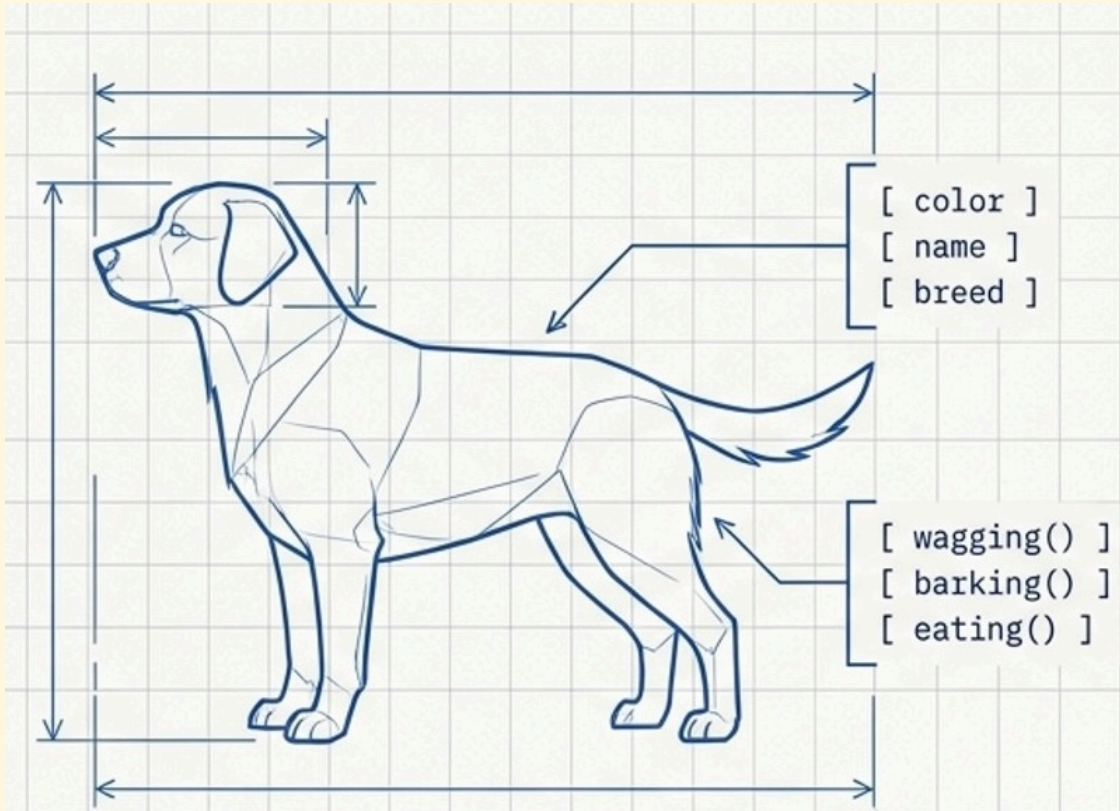
UML SYNTAX & RELATIONSHIPS

- Generalization (Inheritance)
- Realization (Interface)
- Aggregation (Shared) — Shared Ownership
- Composition (Composite) — Strong Ownership
- Association

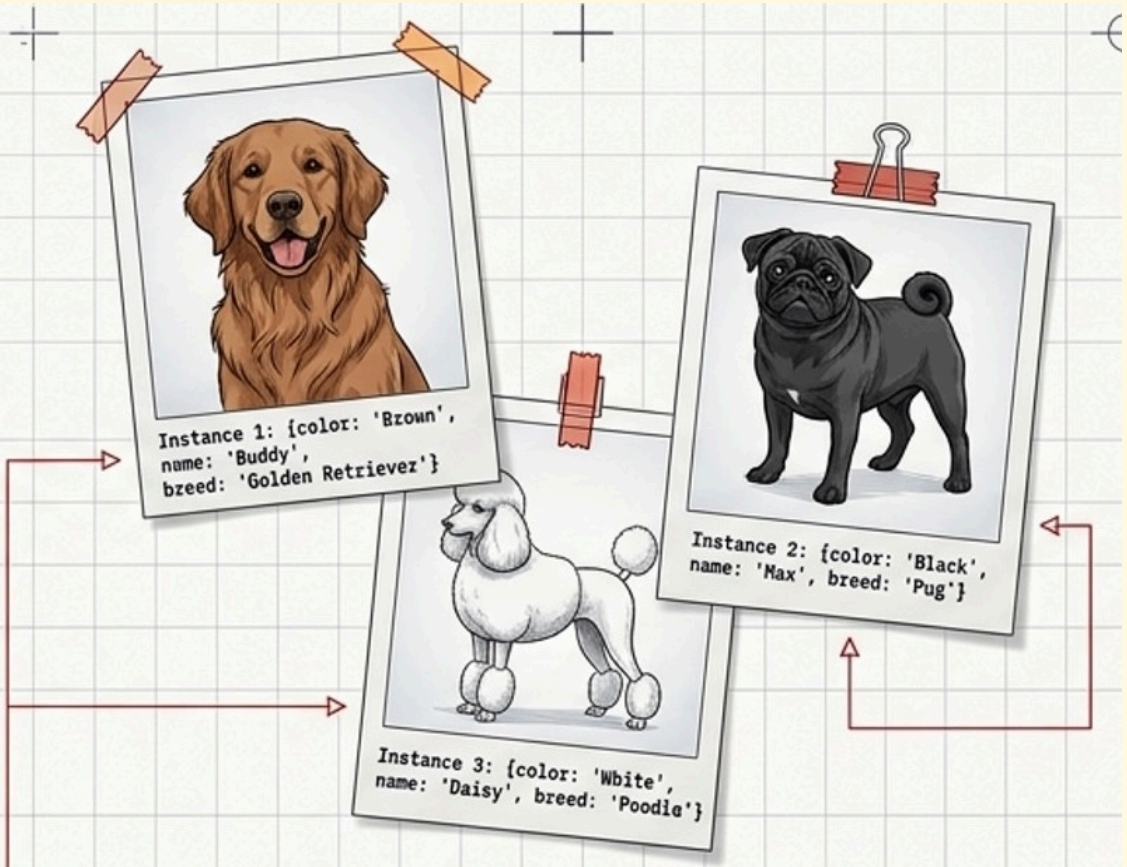
ARCHITECTURAL MAPPING

CLASS → COMPONENT/ROOM
ATTRIBUTE → SPECIFICATION/DIMENSION
OPERATSON → FUNCTION/ACTIVITY
RELATIONSHIP → STRUCTURAL CONNECTION/PATH

藍圖 vs. 實例：物件導向設計基石 (Blueprint vs. Instance)

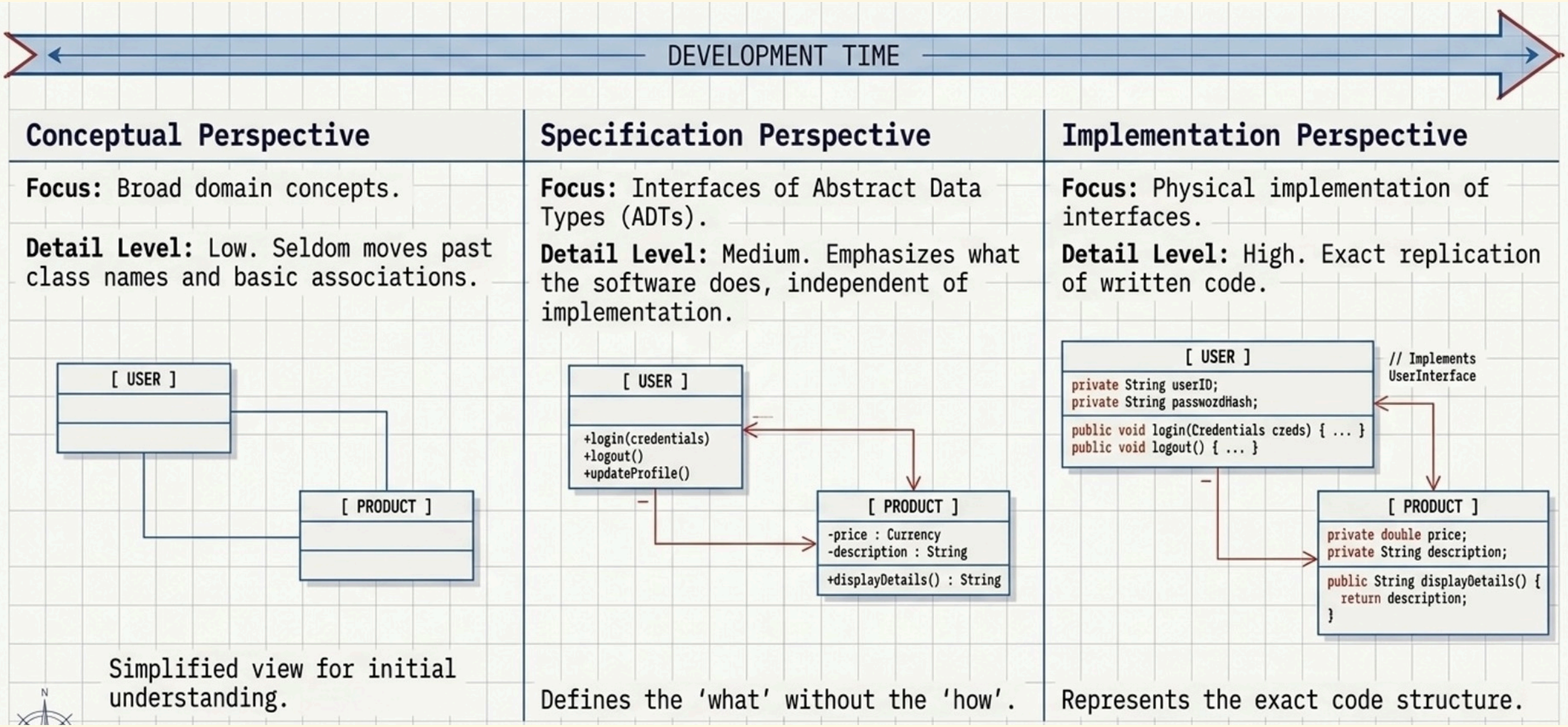


A **Class** is a blueprint. It describes the type of objects and encapsulates state (**attributes**) and behavior (**operations**). It is not the object itself.

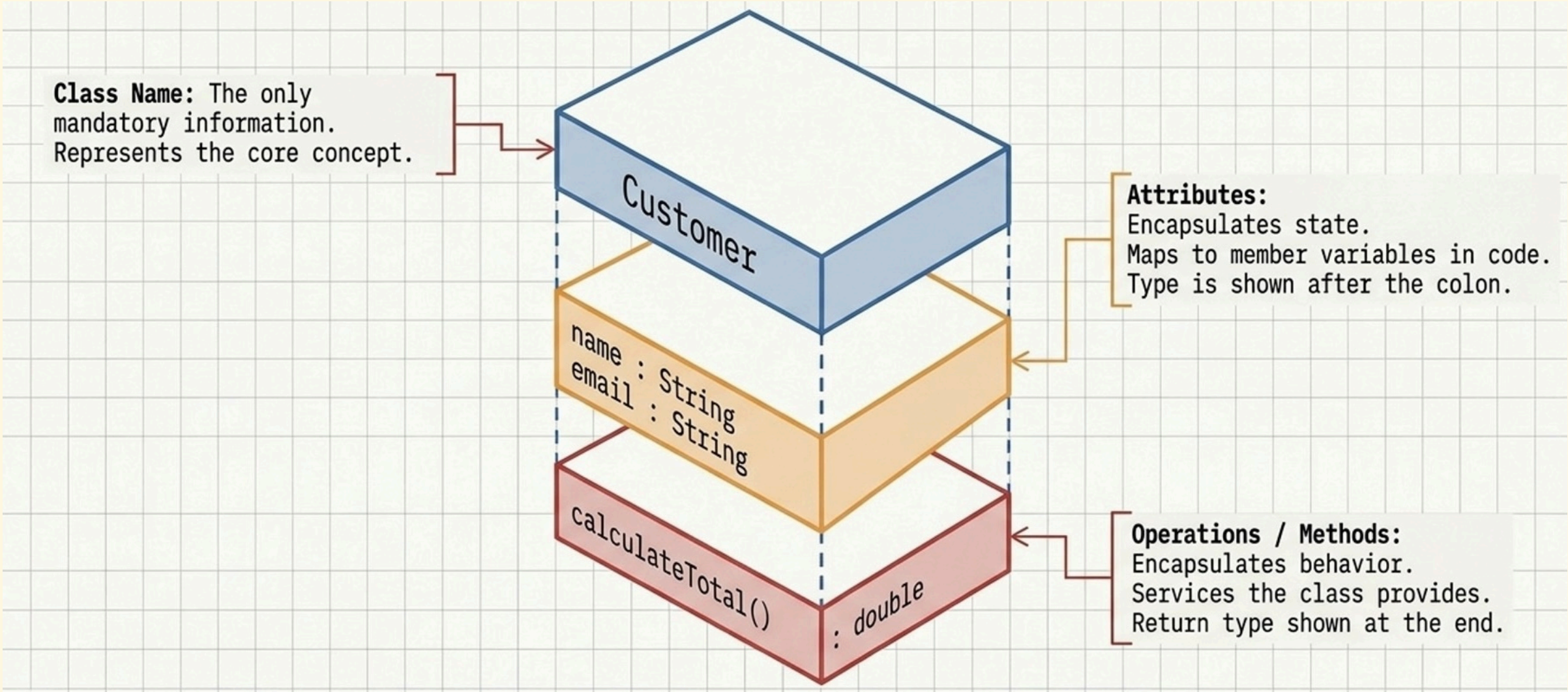


Objects are usable instances of classes. Built from the same blueprint, they contain the same components but hold distinct, real-world states.

類別塑模的三大層次透視 (Three Perspectives)



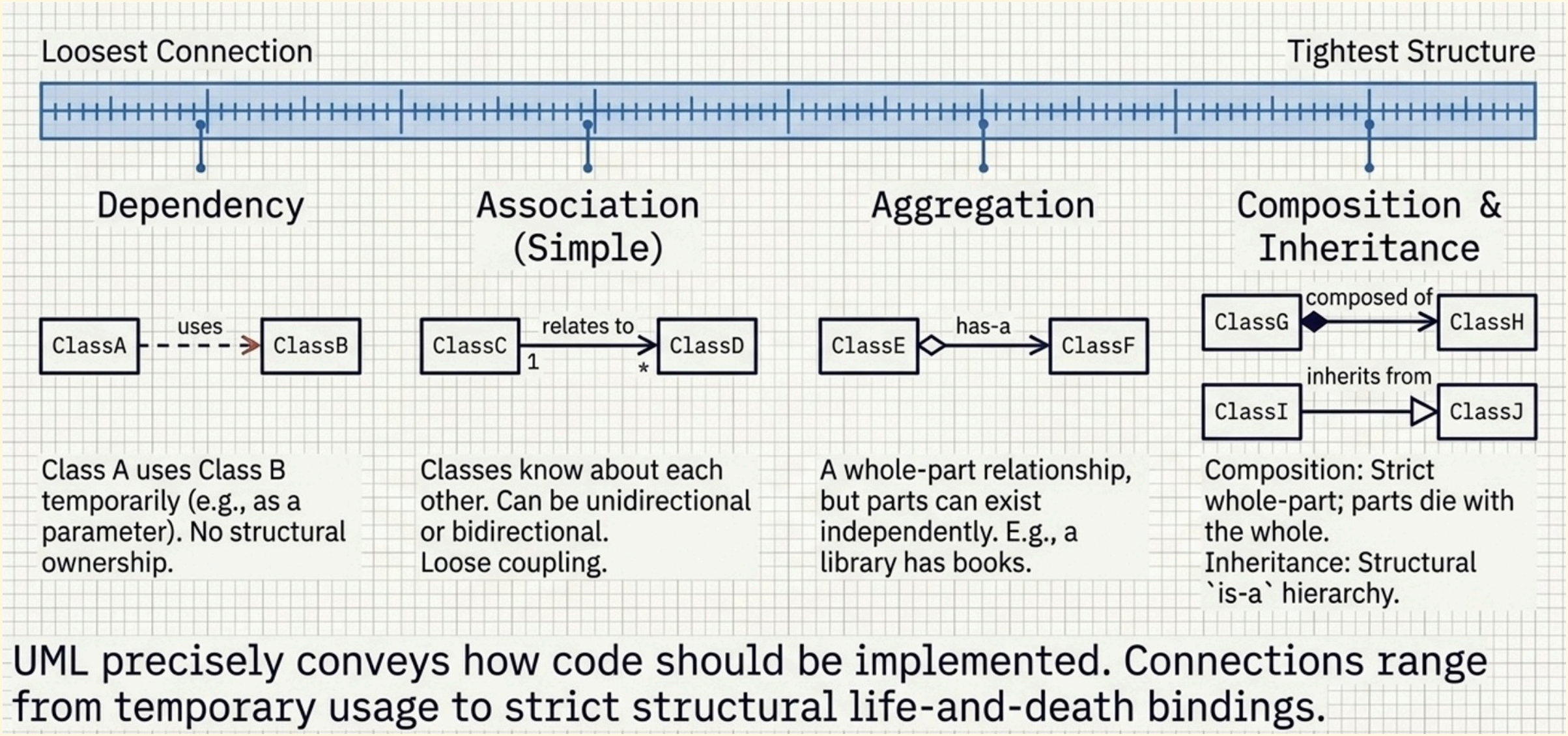
UML 類別方塊解剖學 (Anatomy of the Class Box)



成員可見度矩陣 (+, -, #, ~)

Symbol	Label	Meaning	Code Snippet
+	Public	Accessible from anywhere.	`public String name;`
-	Private	Accessible only within the class itself.	`private int id;`
#	Protected	Accessible within the class and its subclasses.	`protected double balance;`

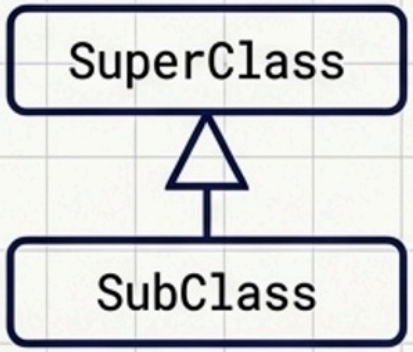
類別關聯分類學 (Taxonomy of Relationships)



UML precisely conveys how code should be implemented. Connections range from temporary usage to strict structural life-and-death bindings.

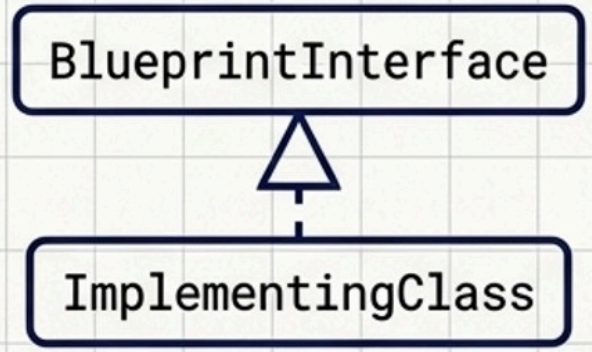
UML 連接線與箭頭速查表 (Connector Cheat Sheet)

Inheritance (Generalization)



Represents an “is-a” relationship. Subclass inherits features.

Realization



Relationship between interface and implementation.

Dependency

A “uses” relationship. Object used in a method, not stored in a field.

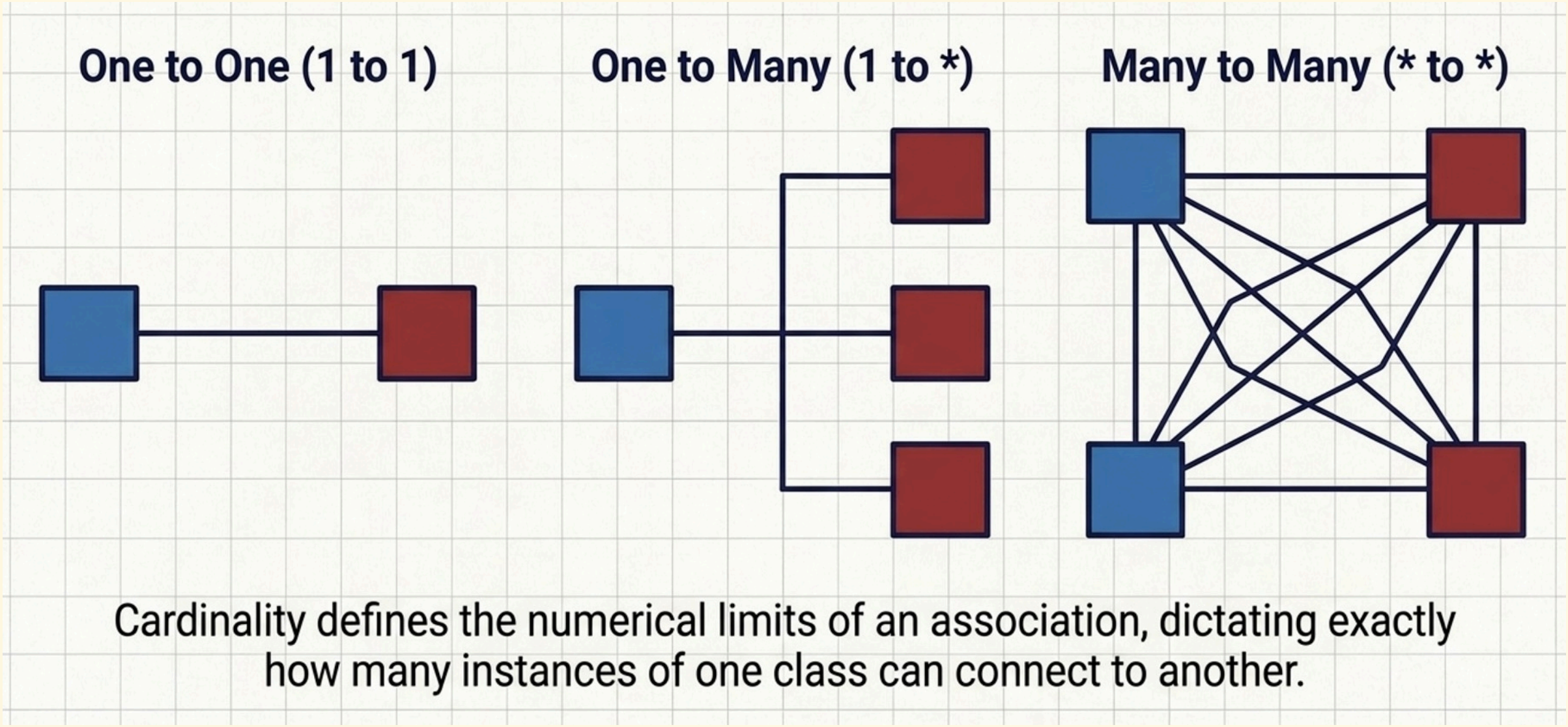
Simple Association

A structural link between two peer classes.

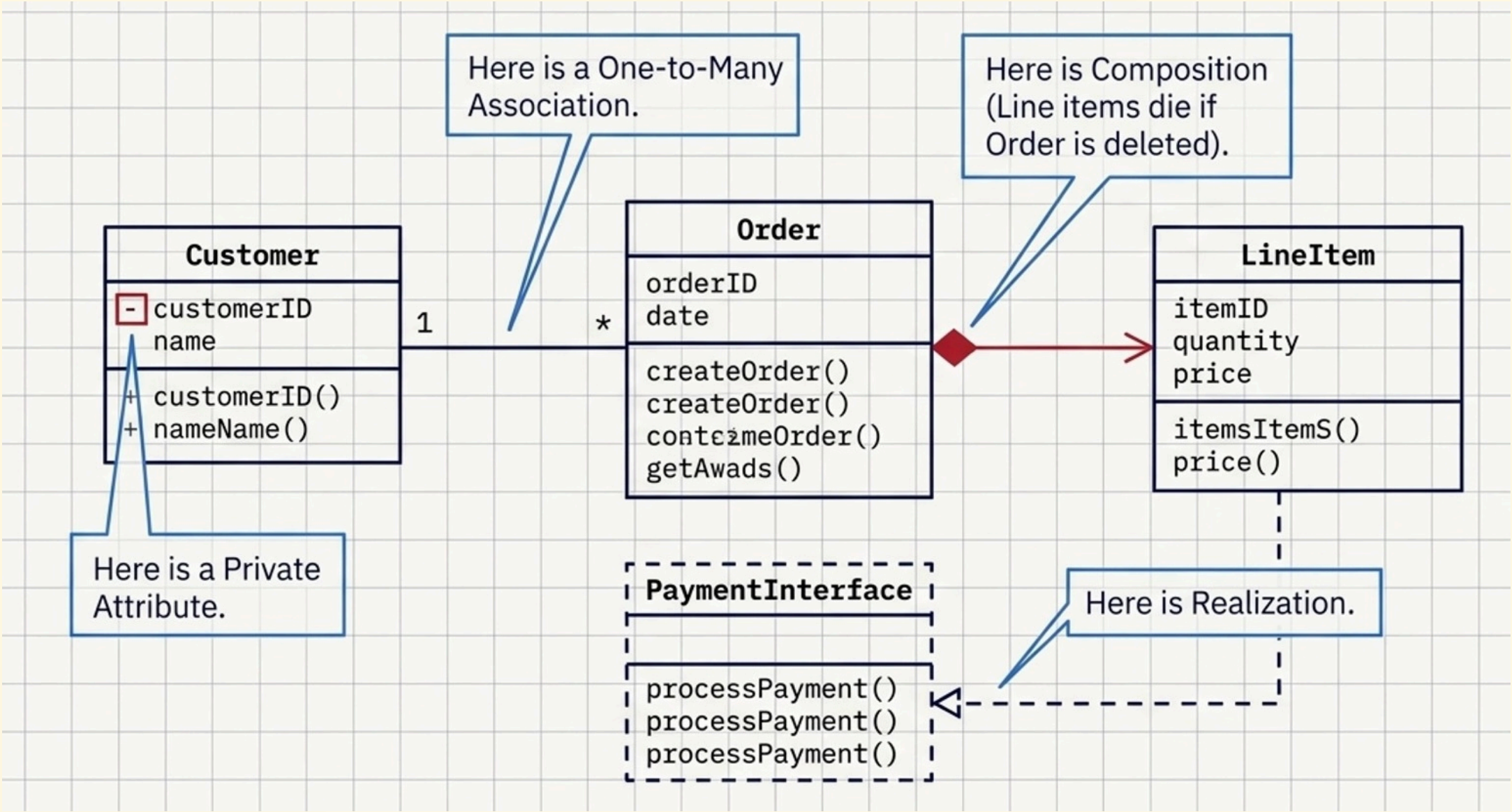
生命週期診斷：聚合 vs. 組合 (Aggregation vs. Composition)

Aggregation	Composition
• **Concept:** A ‘part of’ relationship. • **Lifespan:** Separate lifetimes. • **Analogy:** Team ◇— Player (If the team disbands, players still exist)	• **Concept:** Strict containment. • **Lifespan:** Live and die together. Part cannot stand alone. • **Analogy:** House ◆— Room (If the house is demolished, rooms cease to exist)

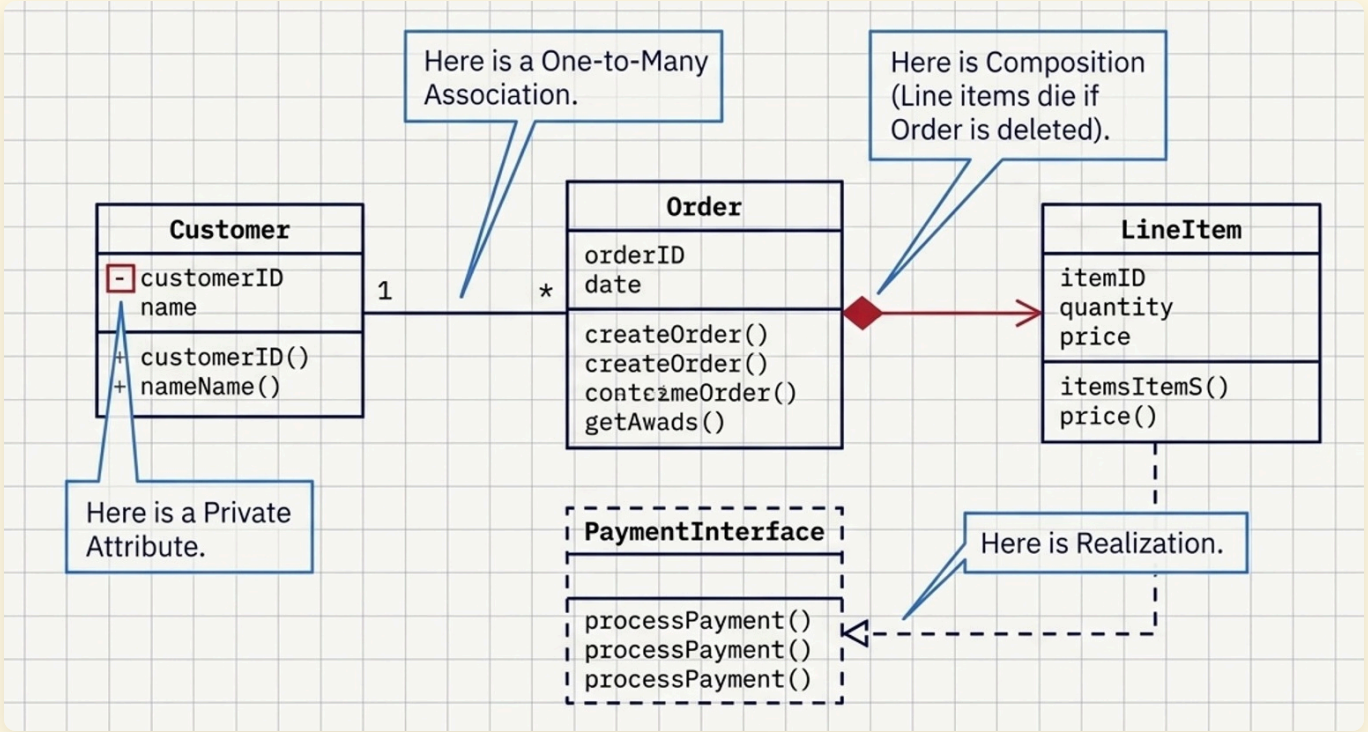
基數、多元性與業務約束 (Cardinality & Constraints)



語法化為系統：電商訂單領域模型 (The Order Model)



觀念檢核：電商訂單模型判讀 (Model Comprehension)



根據右方類別圖，判斷下列 7 個陳述為「正確 (Correct)」或「錯誤 (Incorrect)」：

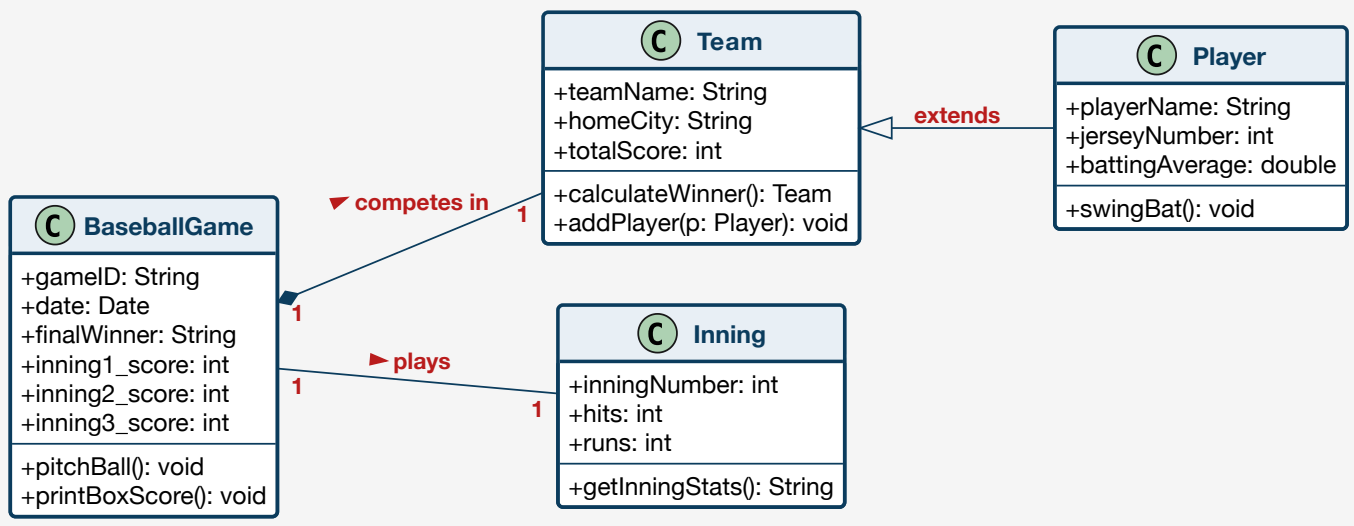
- **1.** 顧客 "Nick" 在系統中可以同時擁有 3 筆訂單。
- **2.** 若一筆 `Order` 被刪除，其所屬的 `LineItem` (訂單明細) 仍會完整保留。
- **3.** 系統中可以存在未關聯任何 `Customer` 的孤立 `Order`。
- **4.** 外部類別可直接透過 `customer.customerID` 存取並修改其值。
- **5.** `LineItem` 實現了 `PaymentInterface` 所定義的介面操作。
- **6.** 新註冊的顧客帳號在系統中可以暫時擁有 0 筆訂單。
- **7.** 一筆 `Order` 可以同時由兩位不同的 `Customer` 共同擁有。

觀念檢核測驗 4 (CCQ 4)

在 UML 類別圖中，若類別 A 與類別 B 之間繪製了一條連線，且在 A 端標示了**實心黑色菱形 (Filled Black Diamond, ◆)** 指向 B，這代表什麼架構意涵？

- **A.** 類別 B 繼承了類別 A 的所有公開方法與屬性。
- **B.** 類別 A 組合 (Composition) 了類別 B；當物件 A 被銷毀時，所屬的物件 B 也將連帶被銷毀。
- **C.** 類別 A 與類別 B 是鬆散聚合關係，B 擁有獨立生存的生命週期。
- **D.** 類別 A 與類別 B 之間僅存在執行期的暫時性依賴。

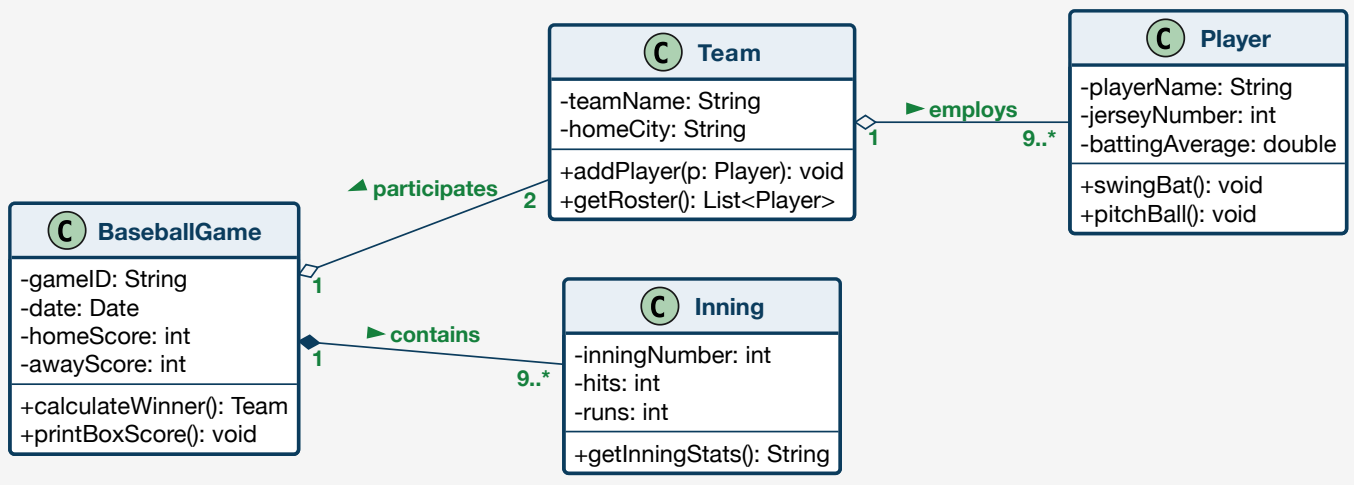
觀念診斷練習：棒球比賽類別模型錯誤審查



審查這份學生繪製的棒球比賽類別圖草稿：
你能從**封裝、繼承、關係（重數與組合）**中找出重大架構謬誤嗎？

- **1. 整體與部分生命週期（聚合 vs. 組合）：**
  - 為何 `BaseballGame` 與 `Team` 使用實心菱形 (`◆`)？若刪除一場比賽紀錄，球隊會跟著消滅嗎？
- **2. 繼承 vs. 成員關聯（is-a vs. has-a）：**
  - 觀察 `Player` 與 `Team` 的連線，`Team <|-- Player` 在觀念上有何嚴重本質錯誤？
- **3. 重數與邊界限制（Multiplicity）：**
  - 一場比賽能只有 `1` 個球隊與 `1` 個局數嗎？棒球比賽實際上有幾隊、幾局？
- **4. 封裝性與職責分配（Encapsulation & Operations）：**
  - 為何所有屬性都是公開的 (`+`)？`pitchBall()` 該放在比賽還是球員？勝負該由單一球隊決定嗎？

觀念診斷解答：棒球比賽類別模型錯誤剖析與修正



審查診斷結論與架構修正指引：

- **1. 組合 vs. 聚合生命週期 (`BaseballGame` & `Team`)：**
  - 實心菱形 (`◆`) 會使比賽刪除時球隊跟著消滅！球隊獨立於比賽存在，應改用聚合 (`◇`)。一場比賽應有 `2` 隊與 `9` 局。
- **2. 濫用繼承 (is-a vs. has-a)：**
  - `Team <|-- Player` 嚴重錯誤。球員是球隊的成員 (has-a)，絕非球隊的一種 (is-a)。
- **3. 破壞封裝與職責分配錯亂：**
  - 屬性應設為私有 (`-`)。`pitchBall()` 應由球員/投手執行。單一球隊不可片面決定 `calculateWinner()`，應由比賽協調判定。

AI 輔助指南：類別塑模 (Class Modeling)

- **1. 角色設定與抽象層次界定 (Persona & Abstraction Level)：**
  - 指示 AI 扮演「物件導向領域模型專家 (Domain Architect)」，並明確要求產出「概念層領域模型」或「實作層設計模型」。
- **2. 整體與部分關聯性嚴格紀律 (Whole-Part Coupling)：**
  - **組合 (實心菱形, ◆)：** 強制 AI 辨識具備生命週期相依性的實體（整體銷毀時，部分連帶 cascade-delete）。
  - **聚合 (空心菱形, ◇)：** 要求部分具備獨立生命週期時使用空心菱形。
  - **禁止濫用泛型關聯：** 嚴禁 AI 全部使用普通實線 (實線) 敷衍了事，必須為每條線指明重數與關聯型態。
- **3. 封裝與雙向重數規範：**
  - 要求強制加上可見度符號（屬性私有 `-`、操作公開 `+`、型態宣告）。
  - 關聯線「兩端」皆必須明確標註重數（如 `1` 對 `1..*`）。
- **4. 架構師人工審查清單 (Verification Checklist)：**
  - [ ] AI 是否誤將繼承 (三角繼承箭頭) 當成組合 (實心菱形)？
  - [ ] 敏感屬性（密碼、金流 token、地址）是否皆設定為 `- private`？
  - [ ] 關聯重數在業務邏輯上是否合理（例如訂單能否包含 0 個項目）？

AI 提示詞範例：外送平台領域類別模型

1. 角色設定與任務指令：

你是一位資深物件導向軟體架構師。
請根據以下領域需求，產出符合 DDD 規範的
標準 PlantUML 類別圖 (Class Diagram)。

2. 塑模約束條件：

- `Order` 與 `OrderItem` 必須使用**組合關聯** (實心菱形，重數 `1` 對 `1..*`)。
- `Order` 與 `Courier` 必須使用**聚合關聯** (空心菱形，重數 `*` 對 `0..1`)。
- `OrderItem` 關聯至 `MenuItem` (`*` 對 `1`)。
- 所有屬性均須宣告 `- private` 與明確資料型態。
- 標註關鍵業務方法 (`+ calculateTotal()`, `+ assignCourier()`)。

3. 輸入需求敘述 (Requirements Statement)：

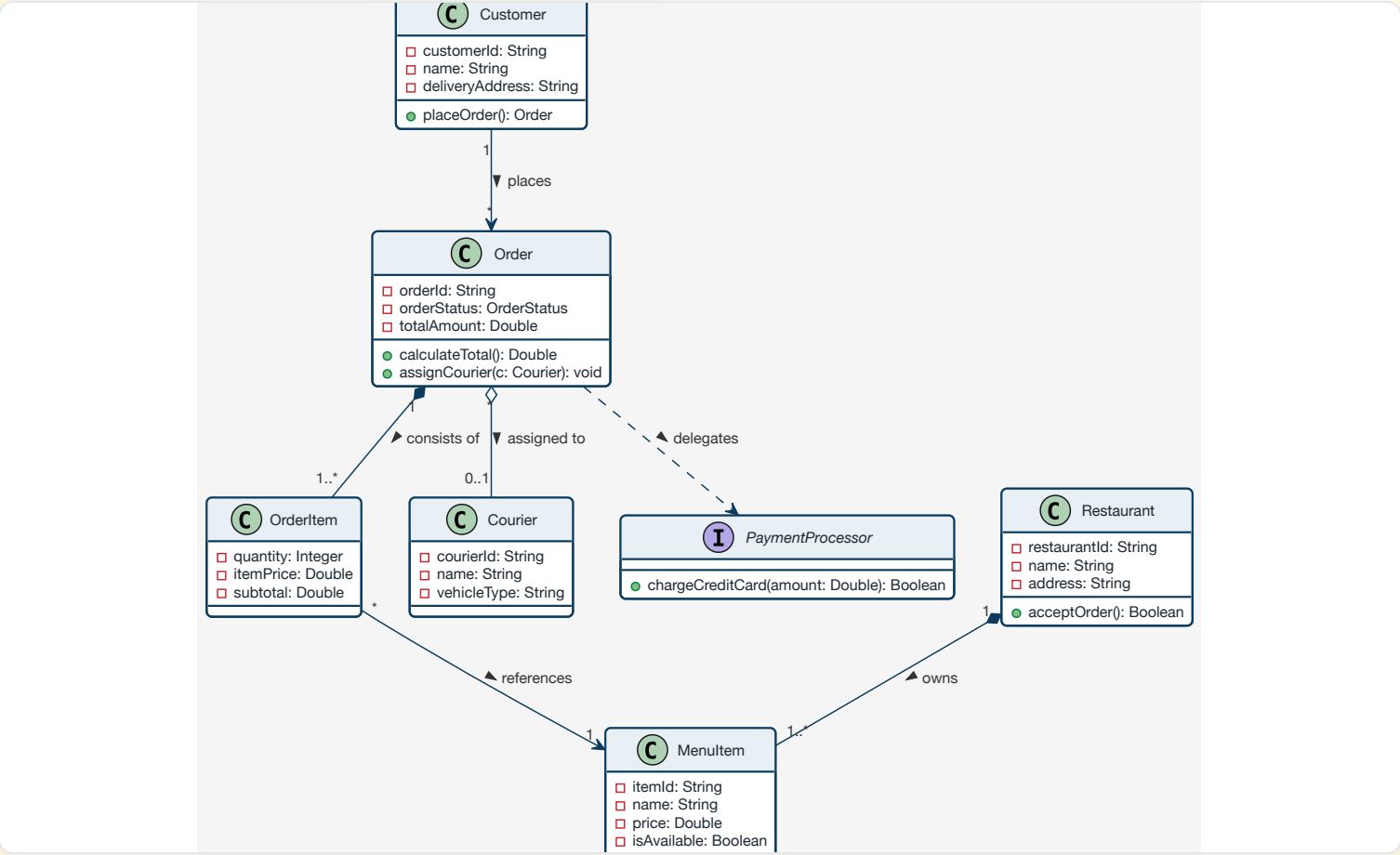
> *「系統包含* ***Customer****（顧客 ID、姓名、Email、電話、外送地址）。*
> ***Restaurant****（餐廳 ID、店名、地址）擁有多項* ***MenuItem****（餐點 ID、品名、單價、是否在庫）。*
> *顧客可建立多筆* ***Order****。每筆訂單包含訂單編號、狀態列舉 (OrderStatus: 待處理、製作中、配送中、已送達)、下單時間與總金額。*
> *一筆訂單由一個或多個* ***OrderItem****（購買數量、購買當下單價、小計）組成；若訂單被取消刪除，其明細項目必須連帶銷毀。*
> *平台可指派一位* ***Courier****（外送員 ID、姓名、電話、交通工具）負責配送訂單。*
> *訂單透過* ***PaymentProcessor*** *介面委託第三方完成扣款。」*

AI 生成之 PlantUML 與架構圖：外送平台領域類別模型

AI 生成之 PlantUML 規格代碼：

```
@startuml
class Customer {
  - customerId: String
  - deliveryAddress: String
  + updateAddress(newAddress: String): void
}
class Restaurant {
  - restaurantId: String
  - cuisineType: String
}
class MenuItem {
  - itemId: String
  - price: Double
}
class Order {
  - orderId: String
  - status: OrderStatus
  + calculateTotal(): Double
  + assignCourier(c: Courier): void
}
class OrderItem {
  - quantity: Integer
  - subtotal: Double
}
class Courier {
  - courierId: String
  - vehicleType: String
}
interface PaymentProcessor {
  + chargeCreditCard(amount: Double): Boolean
}
```

渲染產出之架構圖：



課堂活動：使用 AI 演進領域類別架構 (雙人動手實作)

動手實作實驗：變更架構規則並審核 AI 產出 (4 分鐘)

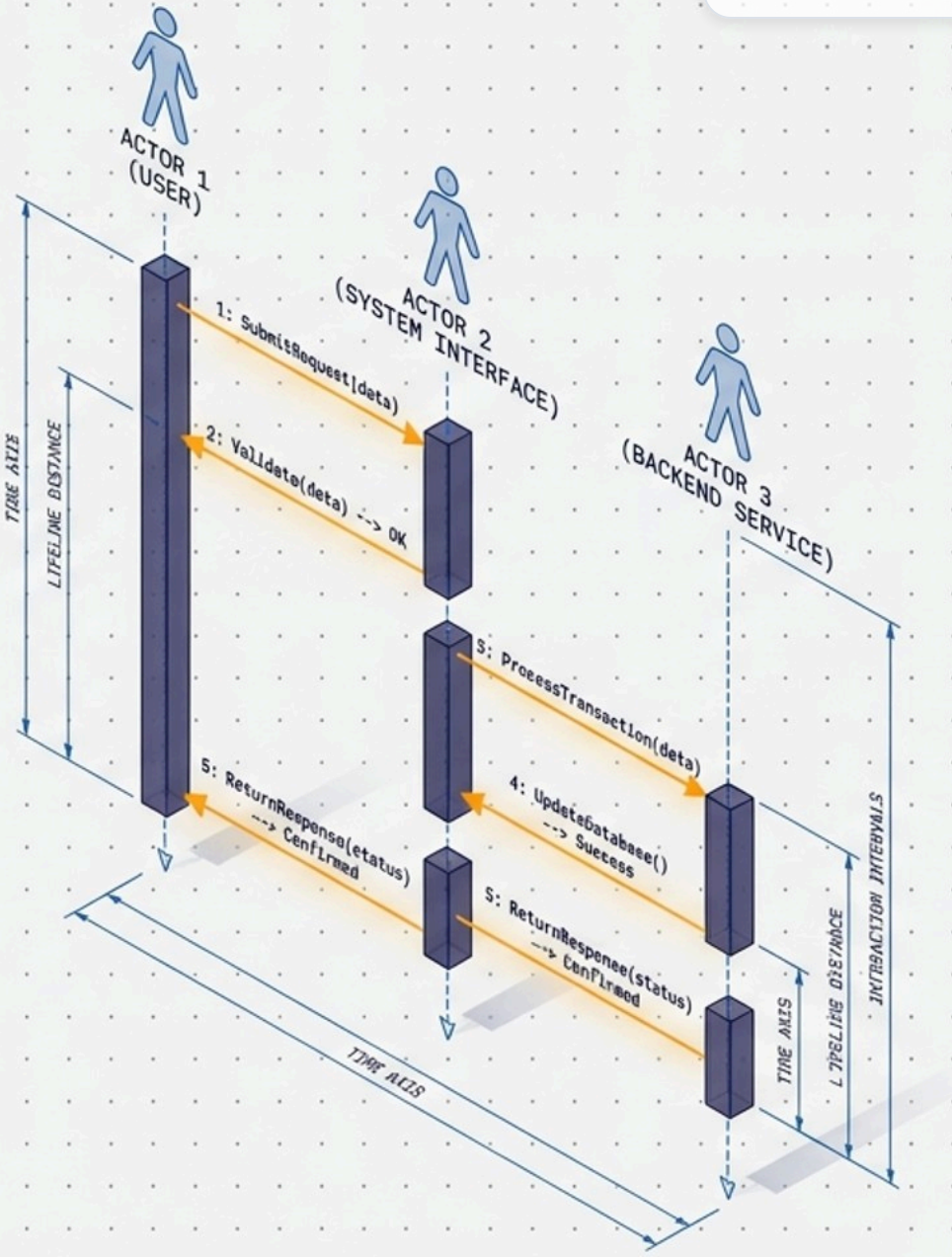
- **1. 在類別模型提示詞中加入一項新架構規則：**
  - *多店合併結帳：* 允許單一筆訂單跨多家餐廳合併下單結帳。
  - *外送小費與評價：* 增加對外送員的直接打賞與雙向服務評分。
  - *VIP 會員訂閱制：* 導入每月固定扣款的會員免運機制。
- **2. 要求 AI 重新生成：** 提示 LLM 更新對應的 PlantUML 類別模型代碼。
- **3. 渲染並嚴格審查：** 貼入 PlantText 渲染並審查：
  - AI 是否正確維持**複合 (◆)** 與**聚合 (◇)** 的生命週期差異？
  - **重數 (Multiplicities)** 是否合乎邏輯（跨店訂單需不需要拆分子訂單）？
  - 關鍵欄位是否維持 `- private` 封裝，且未被設計模式過度複雜化？

4.5 互動與循序圖模型

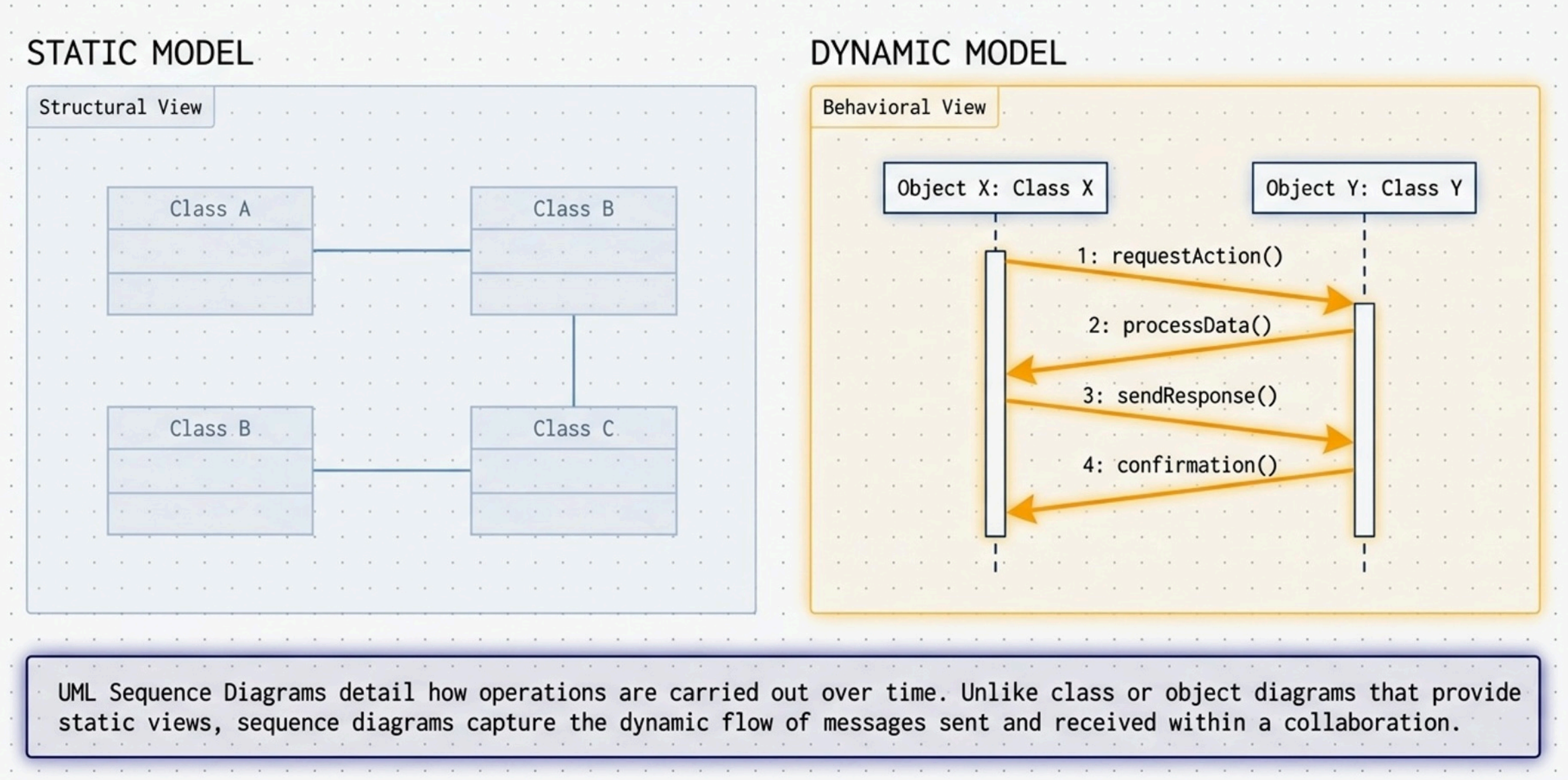
> *"時間是軟體架構中最嚴苛的考驗；循序圖將時間的流逝具象化為可驗證的訊息契約。"*

Sequence Diagrams: The Architectural Blueprint

A visual reference for modeling system interactions and logic.

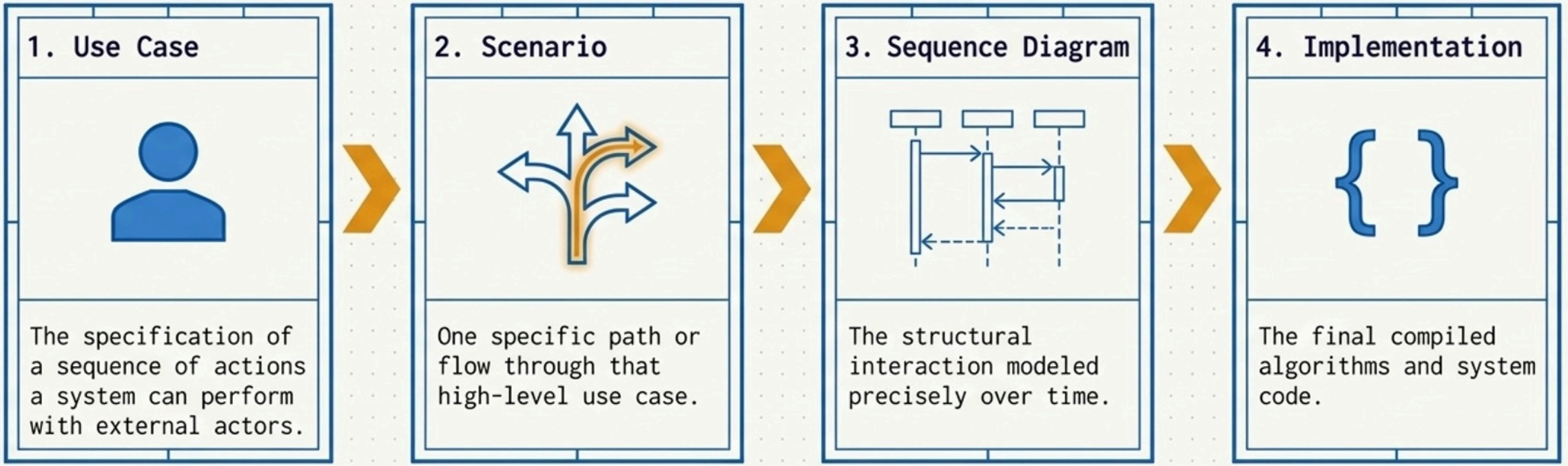


靜態結構 vs. 動態協作 (Static Structure vs. Dynamic Collaboration)



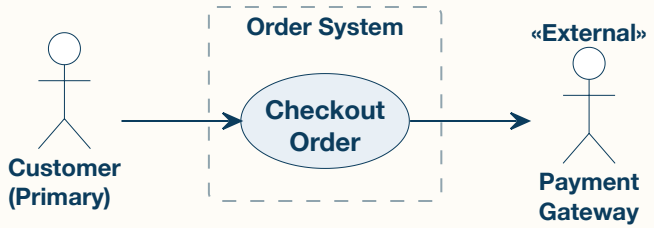
UML Sequence Diagrams detail how operations are carried out over time. Unlike class or object diagrams that provide static views, sequence diagrams capture the dynamic flow of messages sent and received within a collaboration.

從需求到程式碼管線：由故事至實作 (Requirements-to-Code Pipeline)



上游模型支撐：使用案例、場景描述與類別模型

1. 使用案例模型



- **主要參與者：** 顧客 (Customer)
- **次要參與者：** 第三方金流 (Payment Gateway)
- **系統邊界：** 訂單系統 (Order System)
- **業務目標：** 結帳並購買購物車餐點

2. 使用案例情境描述

UC-01： 結帳送出訂單

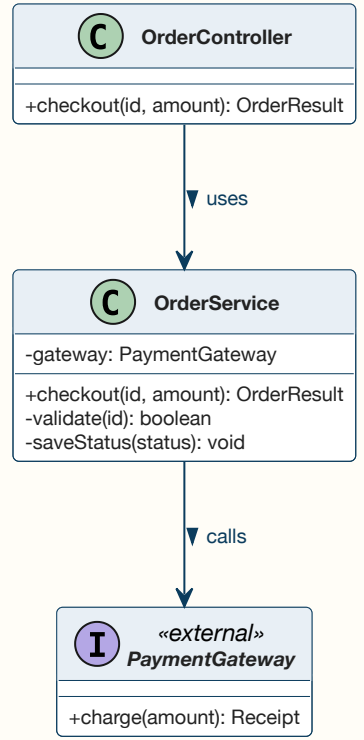
前置條件： 購物車商品已確認且庫存有效

主要成功場景：

1. 顧客送出結帳並提交付款資訊。
2. 系統驗證訂單項目與品項有效性。
3. 系統向第三方金流請求授權扣款。
4. 金流閘道確認交易並回傳扣款收據。
5. 系統將訂單狀態更新為 `PAID`。
6. 系統向顧客顯示結帳成功確認頁。

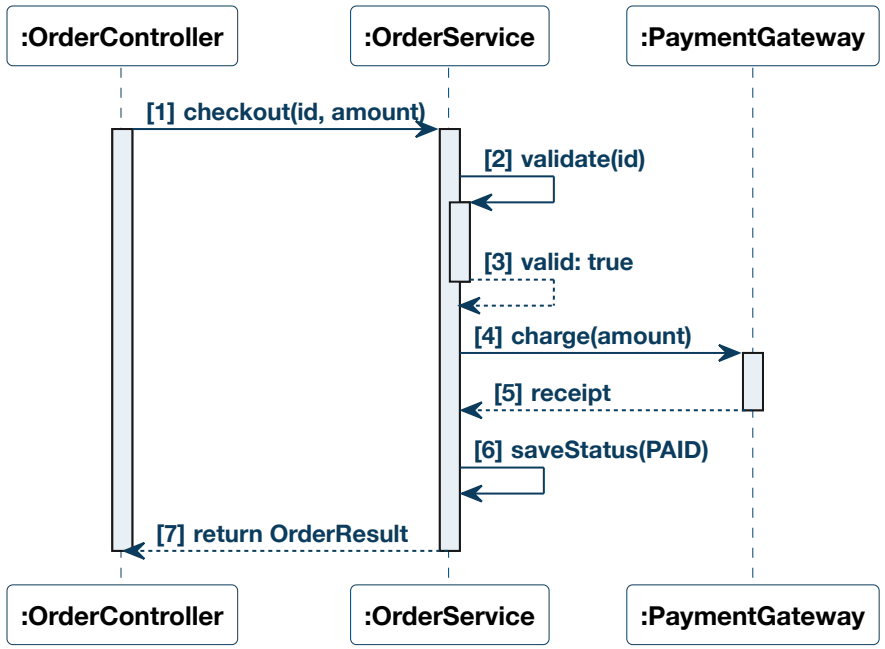
後置條件： 訂單狀態持久化儲存為 `PAID`

3. 設計類別模型



互動畫布：程式碼至循序圖映射

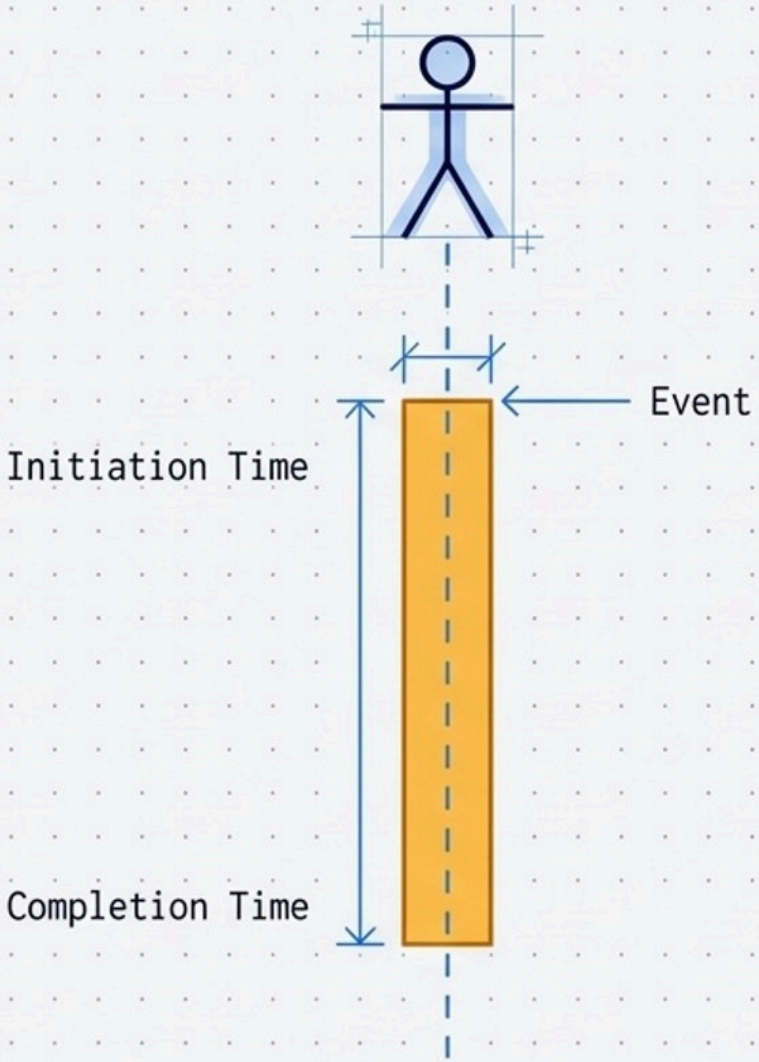
UML 循序圖：3 個物件與 7 大事件時序映射



原始程式碼 (Java)：執行時序流程

```java
class OrderService {
    private PaymentGateway gateway;

    // [1] Controller 調用結帳方法
    public OrderResult checkout(String id, double amount) {
        // [2] 內部驗證訂單有效性（自呼叫）
        boolean ok = this.validate(id); // [3] 回傳 true

        // [4] 調用第三方金流扣款
        Receipt receipt = gateway.charge(amount);
        // [5] 金流回傳付款收據

        // [6] 將狀態持久化為 PAID（自呼叫）
        this.saveStatus(OrderStatus.PAID);

        // [7] 回傳結帳結果給 Controller
        return new OrderResult(true, receipt);
    }
}
```

結構解剖學：生命線、活化條與時間流 (Structural Anatomy)



Actor

External entity interacting with the subject. A role, not necessarily a physical person.

Lifeline

An individual participant in the interaction.

Focus of Control

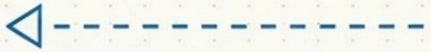
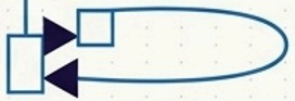
The period during which an element performs an operation. Visualized by the activation rectangle.

Event

Any precise point where an occurrence happens (e.g., sending or receiving a message).

訊息矩陣：同步呼叫、非同步訊號與生命週期 (Messaging Matrix)

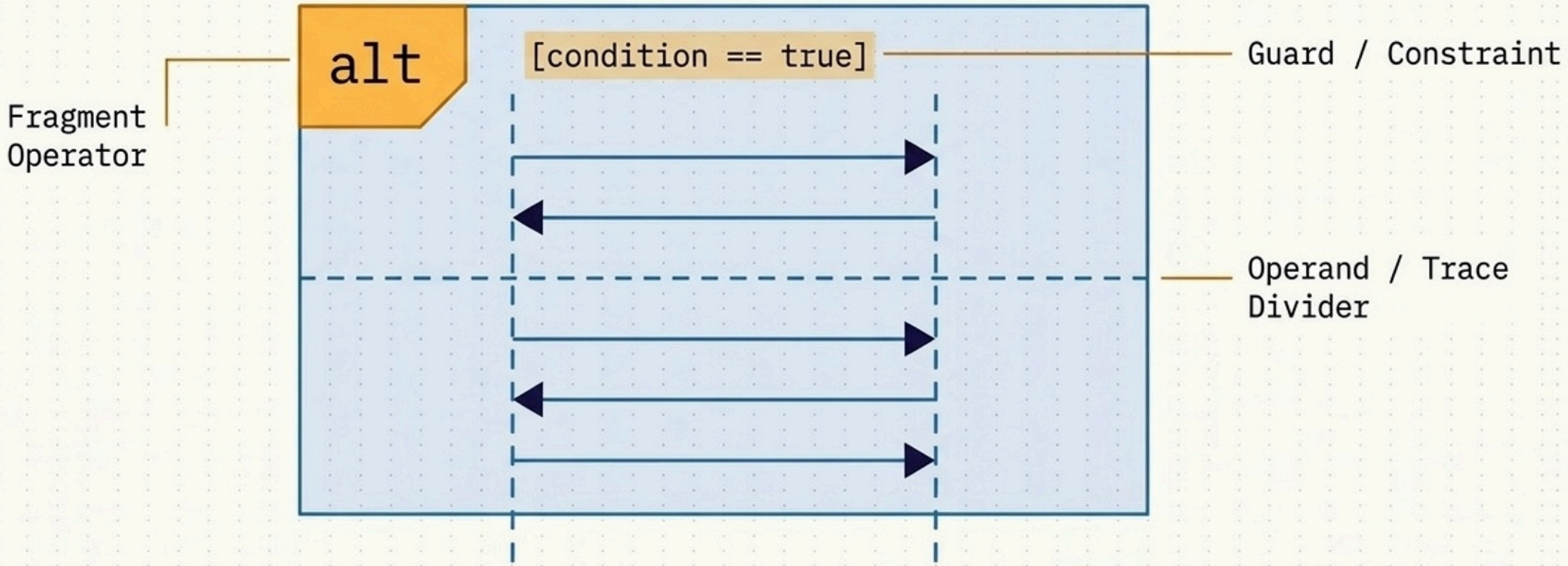
The Messaging Matrix

Message Type	Visual Notation	Semantic Meaning
Call		Invokes an operation on a target lifeline.
Return		Passes information back to the caller of a former message.
Self		Invokes a message on the same lifeline.
Recursive		Invokes a message targeting a new activation stacked on the caller.
Create		Instantiates a new target lifeline.
Destroy		Requests the destruction of the target lifeline's lifecycle.
Duration		Shows the distance between two time instants for an invocation.

Note: A comment carries no semantic force but attaches useful remarks.

UML 2.0 複合片段：alt, opt, loop, par

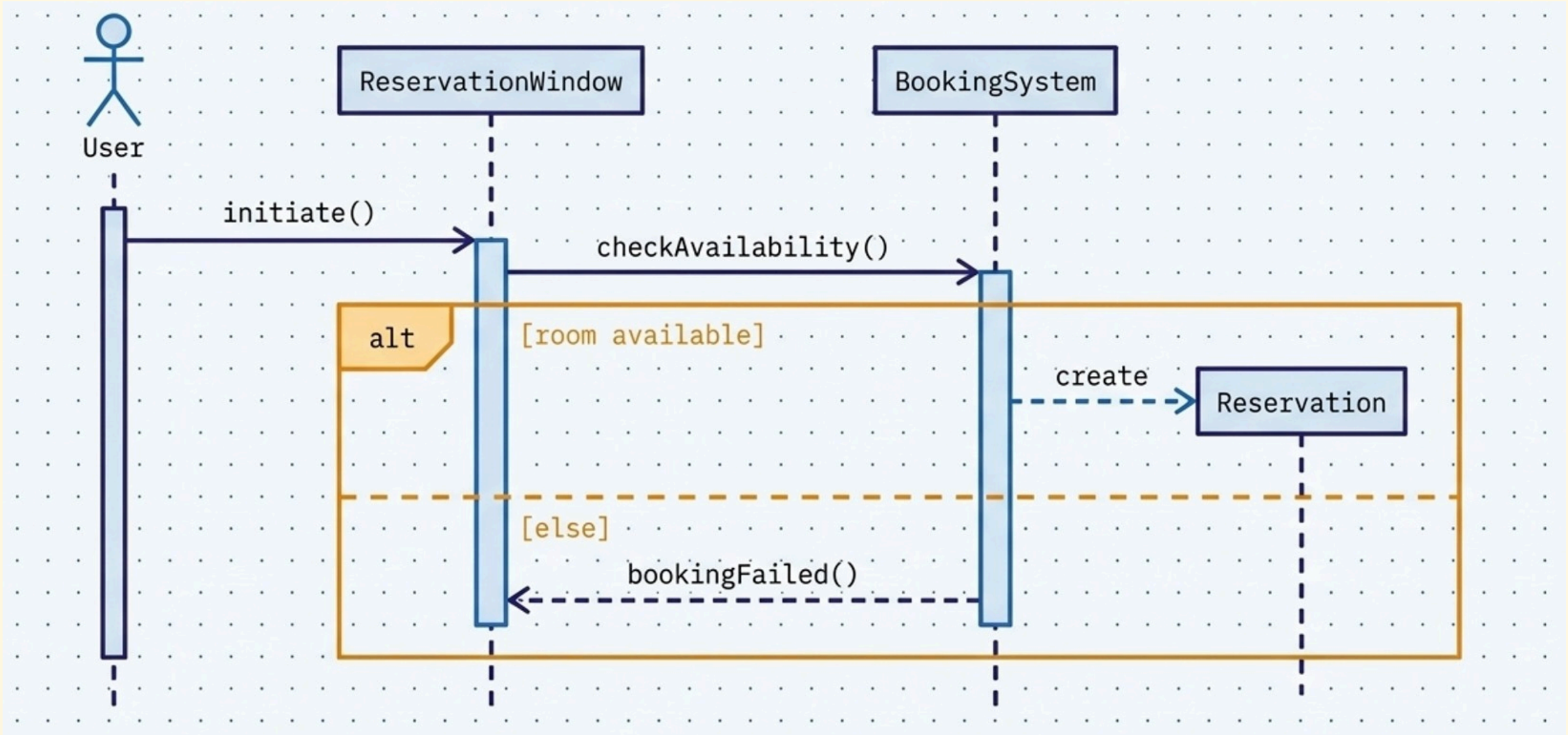
Combined Fragments enclose a portion of the interaction to capture branching, loops, and parallel execution, moving beyond single linear paths.



複合片段運算子轉換至正式程式碼邏輯 (Fragment Operators to Code)

Operator	Definition	Code Equivalent
`alt`	**Alternative:** Multiple fragments; only the one with a true condition executes.	`if / else if / else`
`opt`	**Optional:** Executes only if the supplied condition is true. Single trace.	`if (without else)`
`loop`	**Loop:** Executes multiple times based on a guard condition.	`while / for loop`
`par`	**Parallel:** Each fragment runs concurrently.	`multi-threading / async`
`region`	**Critical Region:** Only one thread can execute this fragment at once.	`Mutex lock / sync`
`neg`	**Negative:** Shows an explicitly invalid interaction.	`Error path / Exception`
`ref`	**Reference:** Refers to an interaction defined on another diagram.	`Function / Module call`
`sd`	**Sequence Diagram:** Surrounds an entire root sequence diagram.	`Root program`

實戰案例研討：飯店客房預訂互動流程 (Hotel Reservation Flow)

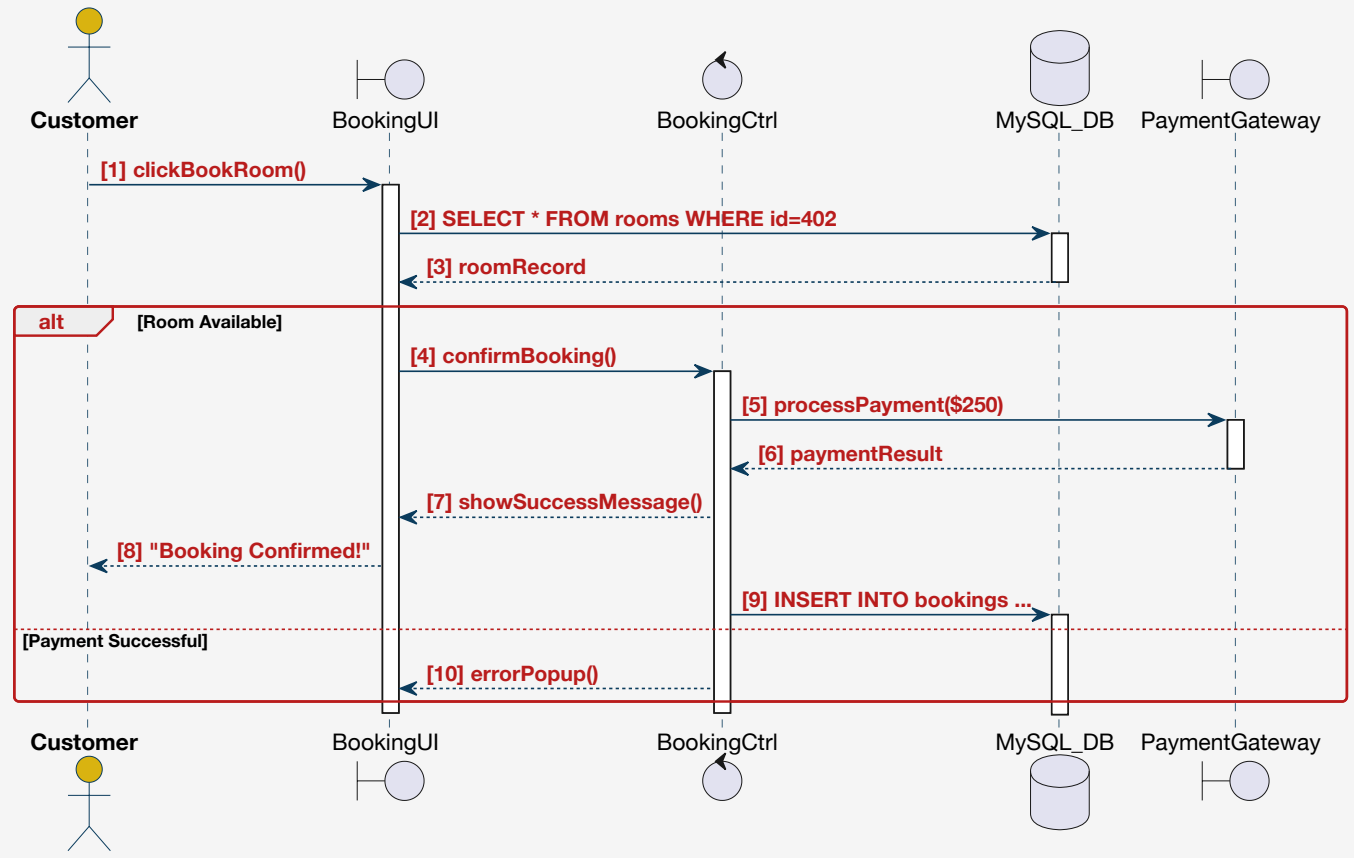


觀念檢核測驗 5 (CCQ 5)

在 BCE (Boundary-Control-Entity) 架構循序圖中，當顧客透過結帳前端點擊『確認下單』按鈕時，該事件應先傳遞給哪一種類型的物件處理？

- **A.** 直接傳遞給 `Order` 實體物件立即寫入資料庫
- **B.** 直接呼叫 `PaymentGatewayAPI` 邊界物件立即刷卡扣款
- **C.** 傳遞給 `OrderController` 控制物件協調業務邏輯驗證與調用
- **D.** 直接傳遞給 `Restaurant` 實體物件確認餐廳接單

觀念診斷練習：找出這張循序圖的 4 大致命錯誤！

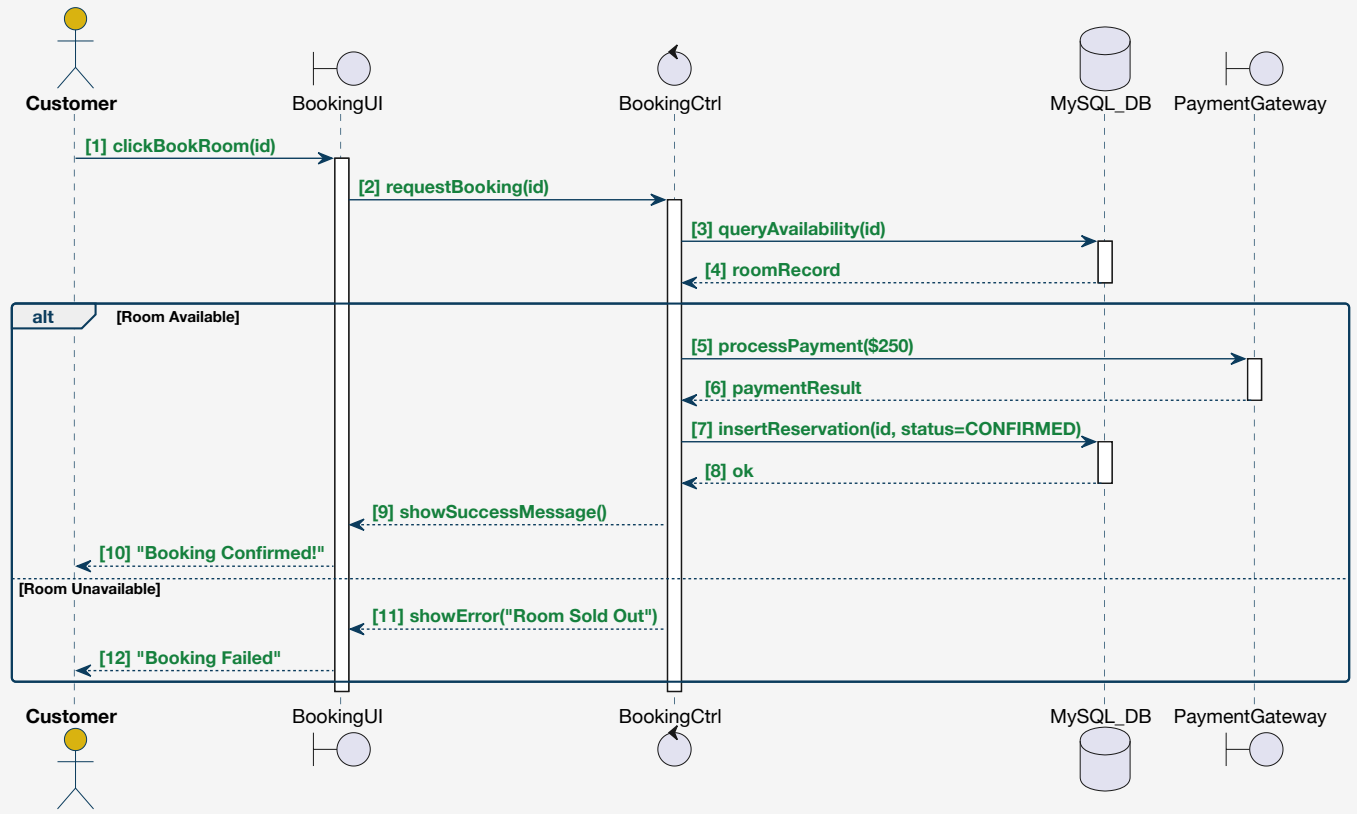


審查這份學生繪製的客房預訂循序圖草稿：

你能從**架構分層**、**訊息語意**、**時序因果**中找出 4 大致命錯誤嗎？

- **1. 架構分層與封裝破壞 (Layering Violation)：**
  - 檢視 `BookingUI` 與 `MySQL_DB`，訊息 `[2]` (`SELECT ...`) 違反了什麼基本架構原則？
- **2. 同步 vs. 非同步訊息語意 (Message Syntax)：**
  - 觀察訊息 `[6]`，既然 `[5]` 是同步阻塞呼叫，回傳值為何誤用非同步虛線開放箭頭？
- **3. 複合片段 (`alt`) 守衛條件邏輯錯亂 (Invalid Guards)：**
  - 檢查 `alt` 方塊中的守衛條件：`[Room Available]` 與 `[else Payment Successful]` 有何荒謬之處？
- **4. 時序因果矛盾與活化條懸掛 (Chronological Paradox)：**
  - 訊息 `[7]`、`[8]` 的「預訂成功」與 `[9]` 的「寫入資料庫」順序為何顛倒？DB 活化條有閉合嗎？

觀念診斷解答：循序圖 4 大致命錯誤剖析與修正



審查診斷結論與架構修正指引：

- **1. 違反架構分層原則 (Layering Violation)：**
  - 前端 `BookingUI` 絕不可越權直接向 `MySQL_DB` 下達 SQL 查詢，必須經由 Controller/Service 業務邏輯層中介。
- **2. 複合片段 (`alt`) 守衛邏輯錯亂 (Invalid Guards)：**
  - `alt` 分支必須針對同一條件互斥 (如 `[Room Available]` 與 `[else / Unavailable]`)，不可混雜付款狀態。
- **3. 時序因果顛倒與過早確認 (Chronological Paradox)：**
  - 資料庫寫入 (`INSERT INTO bookings`) 必須成功後，才可向使用者顯示預訂成功訊息。
- **4. 活化條懸掛遺漏閉合 (Dangling Activation)：**
  - `MySQL_DB` 的活化長條在操作完成後必須立即終止閉合，不可無限延伸。

AI 輔助指南：循序圖塑模 (Sequence Modeling)

- **1. 角色設定與架構模式約束 (Persona & Architecture Pattern)：**
  - 指示 AI 扮演「分散式系統與 API 整合架構師」。
  - **強制引入 BCE 模式：** 嚴格要求區分 Boundary (介面)、Control (業務邏輯)、Entity (資料庫實體)，嚴禁 UI 直接繞過控制器存取資料庫。
- **2. 訊息語法與複合片段嚴格規範：**
  - **同步呼叫 (`->`) 與返回 (`-->`)：** 強制要求對稱繪製返回虛線與回傳型態，避免啟動條無限懸空。
  - **非同步派發 (`->>`)：** 對於背景事件排隊、推播通知，強制使用開放式箭頭。
  - **複合片段 (`alt`)：** 強制要求 AI 必須針對業務失敗情境（如扣款失敗、超時）繪製 `alt [成功] else [失敗]` 區塊。
- **3. 架構師人工審查清單 (Verification Checklist)：**
  - [ ] 是否存在違背分層架構的「越級呼叫」（例如前端 UI 直接調用資料庫實體）？
  - [ ] `alt` 片段中是否具備互斥的守衛條件 `[guard]`？
  - [ ] 啟動條 (activate/deactivate) 的生命區間是否與方法呼叫週期完全吻合？

AI 提示詞範例：外送平台結帳循序模型

1. 角色設定與任務指令：

你是一位資深分散式系統後端架構師。
請根據以下結帳情境，採用 BCE 模式
產出標準 PlantUML 循序圖 (Sequence Diagram)。

2. 塑模約束條件：

- 宣告生命線職責：
  - `boundary CheckoutUI as ":結帳介面"`
  - `control OrderCtrl as ":訂單控制器"`
  - `boundary StripeAPI as ":金流服務"`
  - `entity OrderEntity as ":訂單實體"`
  - `queue KitchenQueue as ":廚房事件佇列"`
- 必須使用 `alt` 片段處理 `[扣款成功]` 與 `[扣款失敗]`。
- 廚房通知必須使用非同步箭頭 `->>` 發送至事件佇列。

3. 輸入需求敘述 (Requirements Statement)：

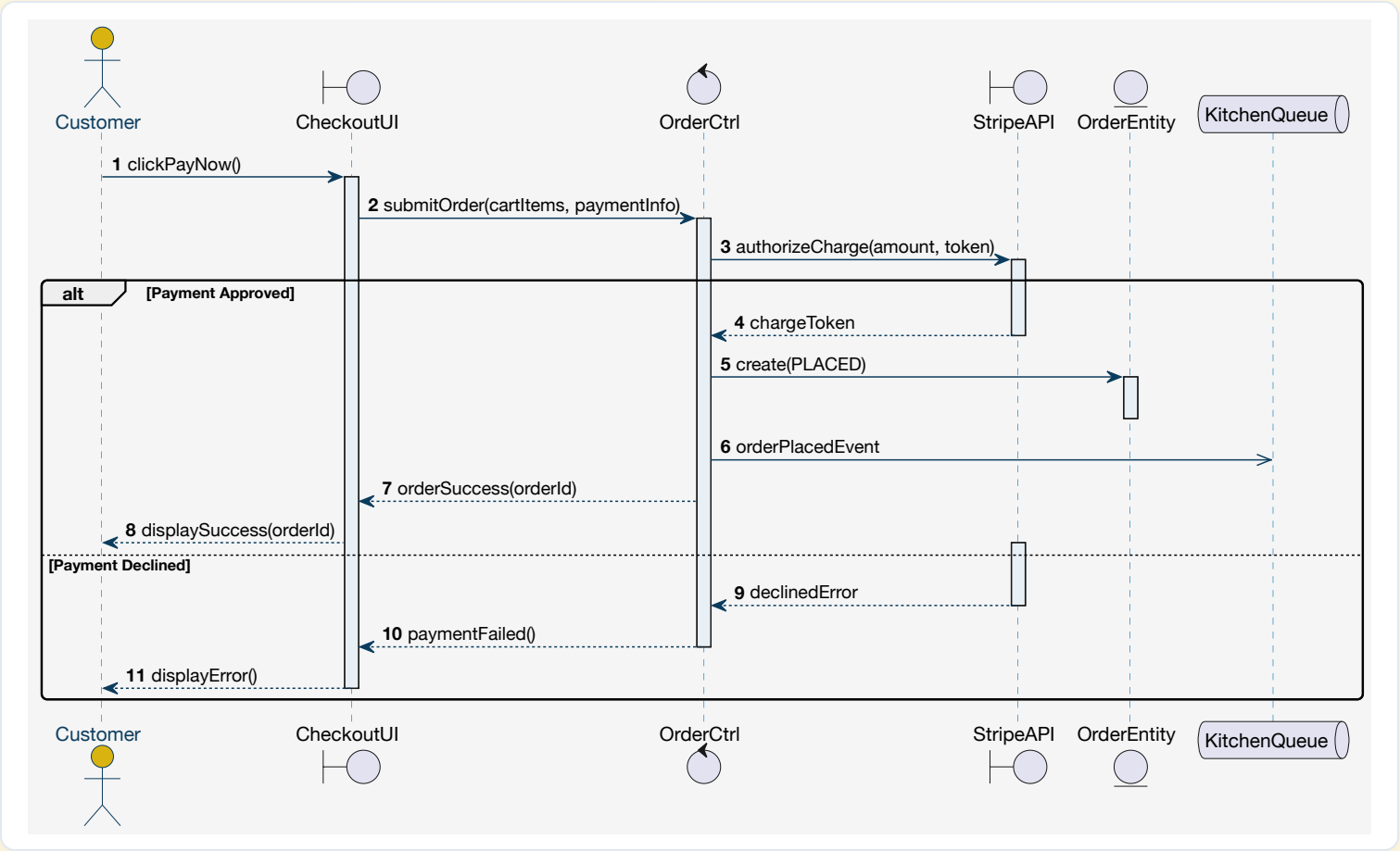
> *「顧客在 **CheckoutUI** 點擊『立即付款』按鈕。*
> ***CheckoutUI** 轉發 `submitOrder(購物車清單, 支付憑據)` 至 **OrderCtrl**。*
> ***OrderCtrl** 呼叫 **StripeAPI** 執行 `authorizeCharge(金額, 憑據)`。*
> *若授權成功：*
> 1. *StripeAPI 回傳授權碼 `chargeToken`。*
> 2. *OrderCtrl 呼叫 **OrderEntity** 建立持久化訂單紀錄（狀態設為 `PLACED`）。*
> 3. *OrderCtrl 以非同步訊息發布 `orderPlacedEvent` 至 **KitchenQueue**。*
> 4. *OrderCtrl 回傳 `orderSuccess(訂單編號)` 給 CheckoutUI。*
>
> *若授權失敗：*
> 5. *StripeAPI 回傳 `declinedError`。*
> 6. *OrderCtrl 記錄失敗日誌，不建立訂單，直接回傳 `paymentFailed()`。」*

AI 生成之 PlantUML 與架構圖：外送平台結帳循序模型

AI 生成之 PlantUML 規格代碼：

```
@startuml
autonumber
actor Customer
boundary CheckoutUI
control OrderCtrl
boundary StripeAPI
entity OrderEntity
queue KitchenQueue

Customer -> CheckoutUI: clickPayNow()
activate CheckoutUI
CheckoutUI -> OrderCtrl: submitOrder(cart, payment)
activate OrderCtrl
OrderCtrl -> StripeAPI: authorizeCharge(amount, token)
activate StripeAPI

alt Payment Approved
  StripeAPI --> OrderCtrl: chargeToken
  deactivate StripeAPI
  OrderCtrl -> OrderEntity: create(PLACED)
  OrderCtrl ->> KitchenQueue: orderPlacedEvent
  OrderCtrl --> CheckoutUI: orderSuccess(orderId)
  CheckoutUI --> Customer: displaySuccess()
else Payment Declined
  StripeAPI --> OrderCtrl: declinedError
  OrderCtrl --> CheckoutUI: paymentFailed()
  deactivate OrderCtrl
  CheckoutUI --> Customer: displayError()
```

渲染產出之架構圖：



課堂活動：使用 AI 演進循序模型 (雙人動手實作)

動手實作實驗：變更動態時序並審核 AI 產出 (4 分鐘)

- **1. 在結帳流程中加入一個極端邊界情境 (Edge Case)：**
  - *金流逾時 (Gateway Timeout)：* 外部付款閘道超過 5 秒未回應（觸發重試或補償）。
  - *拆分付款 (Split Payment)：* 允許顧客使用電子錢包點數折抵搭配信用卡差額支付。
  - *庫存售罄 (Out of Stock)：* 在付款授權過程中，餐廳剛好宣告餐點售罄。
- **2. 要求 AI 重新生成：** 提示 LLM 更新對應的 PlantUML 循序圖代碼。
- **3. 渲染並嚴格審查：** 貼入 PlantText 渲染並審查：
  - AI 是否維護**分層架構**（UI → 控制器），杜絕 UI 越權直連資料庫？
  - **成功與異常分支** (`alt`) 是否皆具備完整明確的回傳訊息？
  - 非同步後台通知（如通知廚房）是否確實與同步付款解耦？

4.6 流程與活動圖模型

> *"複雜的業務流程就像多條河流匯聚；活動圖能讓我們精確掌控每一道支流的並行與匯合。"*

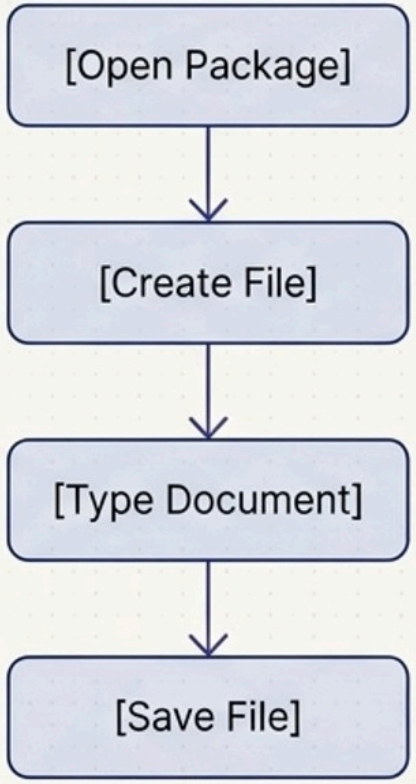
視覺化邏輯：超越靜態結構的動態流程 (Visual Logic)

An Activity Diagram is a crucial behavioral diagram in the Unified Modeling Language (UML). It is an advanced version of a flowchart designed to describe the dynamic aspects of a system, mapping the precise flow from one activity to the next.

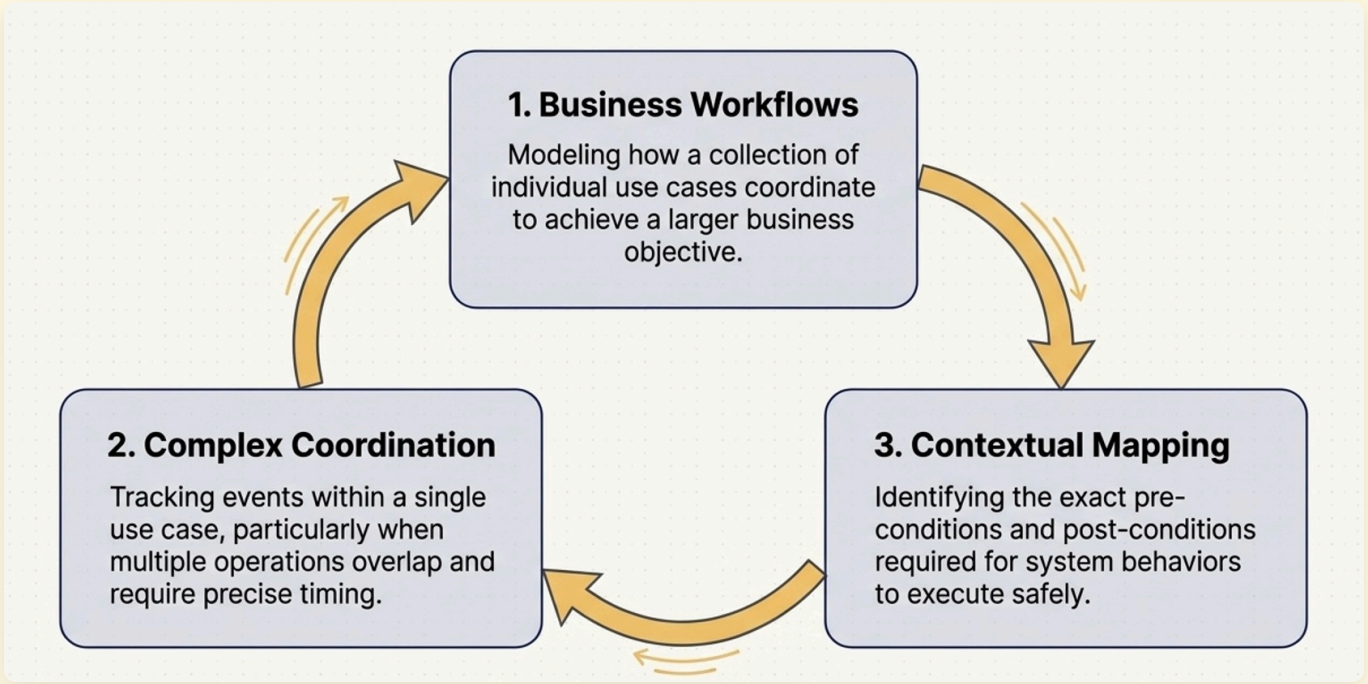
The Chaos: Text-Based Processes

Process starts when the user opens the package and then needs to initialize the system, following which they must create must create a new file and ensure all settings are correct, after which they proceed to type the document, remembering to format it according to the guidelines, and finally, after reviewing, they must save the file in the appropriate directory before closing the application and moving on to the next task.

The Clarity: Visual Logic



採用活動圖塑模的三大觸發契機 (Three Triggers)



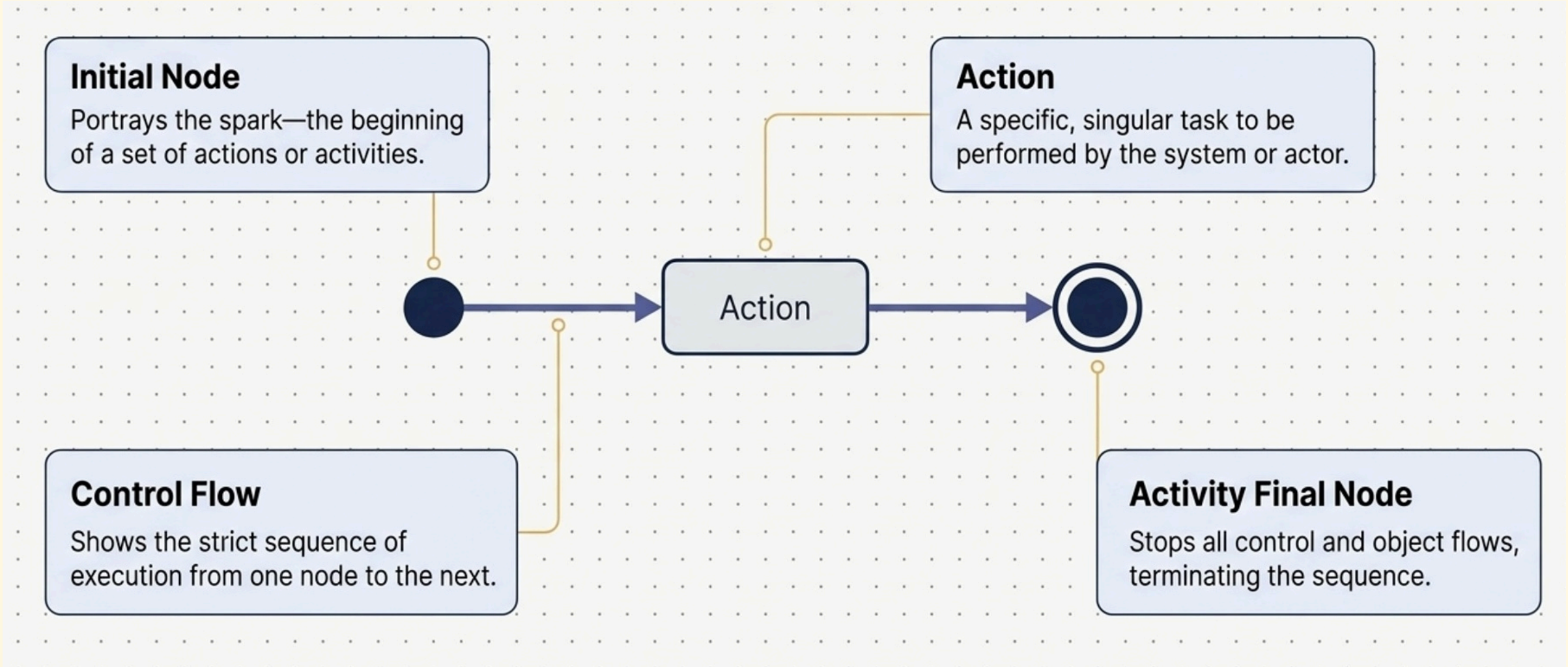
何時該動手繪製 UML 活動圖？

- **1. 跨領域商業工作流 (Business Workflows)：**
  - **端到端價值鏈：** 串聯協調多個獨立的使用案例，達成完整的端到端業務目標。
  - **團隊認知對齊：** 讓業務利害關係人與工程團隊對整體運作全貌達成共識。
- **2. 複雜內部協同調度 (Complex Coordination)：**
  - **單一功能深度分析：** 追蹤單一使用案例內部重疊的多項作業、分支路徑與精確時序。
  - **並行任務管理：** 安全調度多執行緒平行任務，避免競態條件。
- **3. 情境狀態映射 (Contextual Mapping)：**
  - **前後置條件契約：** 明確界定動作執行前的必備前提，以及完成後的狀態保證。
- **核心價值：** 活動圖是連結高階使用案例與底層程式碼實作的最佳思維橋樑。

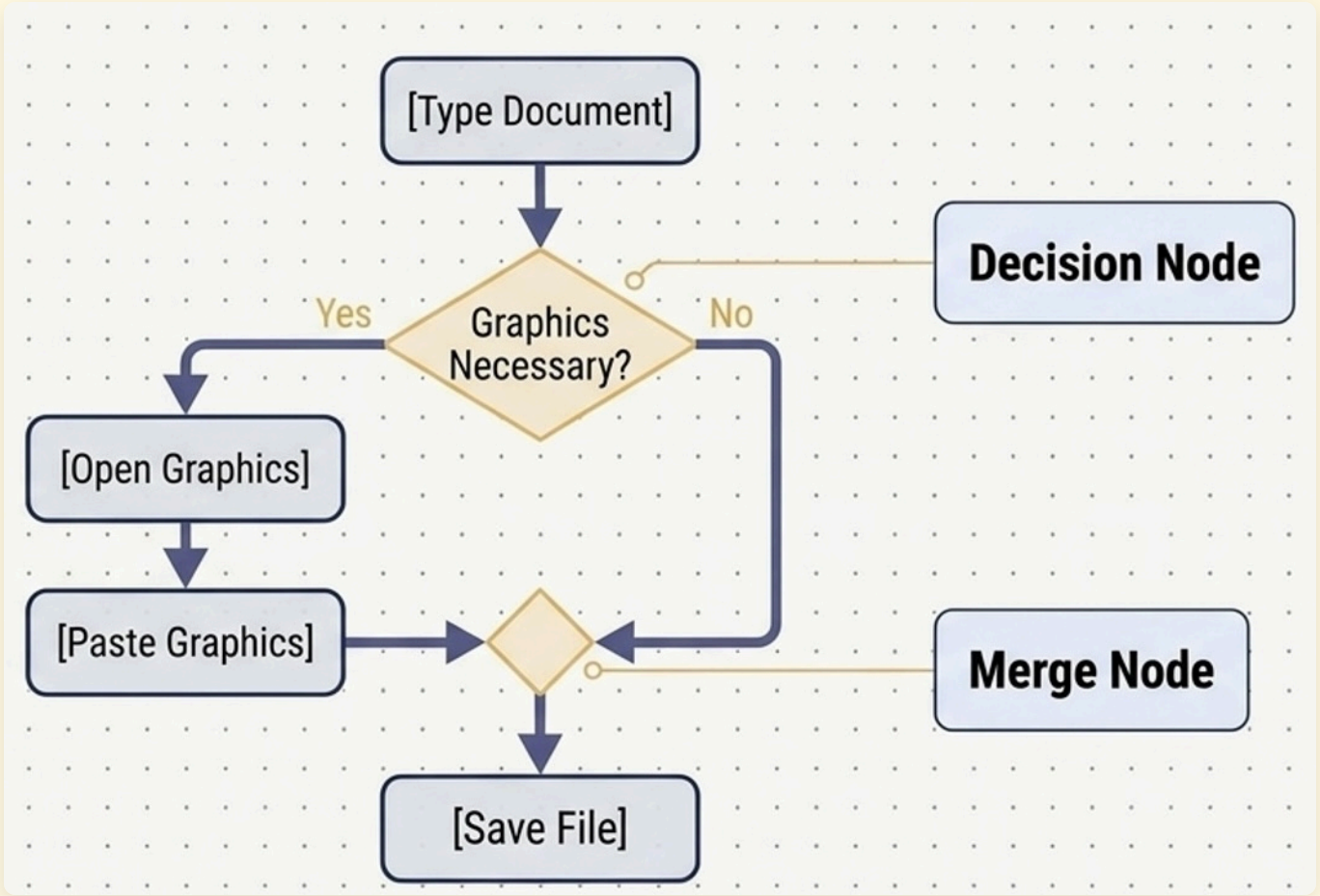
傳統流程圖 vs. UML 活動圖核心差異 (Flowcharts vs. Activity Diagrams)

塑模能力 / 核心維度	傳統流程圖 (Flowchart)	UML 活動圖 (Activity Diagram)
循序執行邏輯 (Sequential Logic)	✓	✓
條件分支判定 (Conditional Splits)	✓	✓
平行並行處理 (Parallel / Concurrency)	✗	✓
角色責任劃分 (Swimlanes / Partitions)	✗	✓
資料物件狀態流 (Object Flow & States)	✗	✓

系統流程核心語彙：起點、動作與終點 (Core Vocabulary)



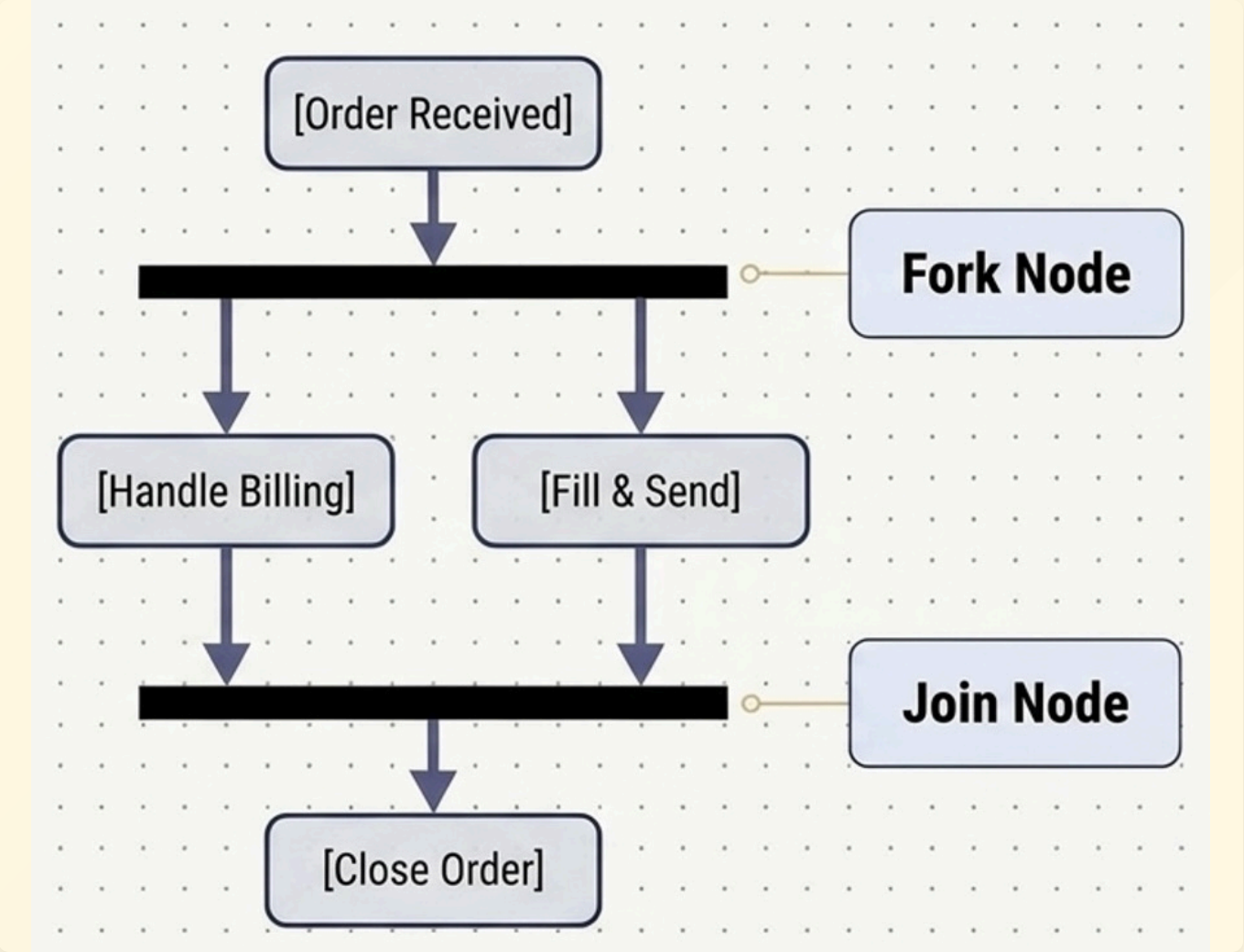
條件分支邏輯：判定與合併節點 (Decision & Merge)



工作流程中的互斥條件路徑管理：

- **決策節點 (上方菱形 ◇)：**
  - **條件分支分流點：** 評估外出控制流上的守衛條件（例如 `[需要圖片？ / 是]` vs. `[否]`）。
  - **嚴格互斥性：** 執行期根據布林條件判定，**恰好只有一條分支**會取得執行權。
- **合併節點 (下方菱形 ◇)：**
  - **替代路徑重新匯合：** 將互斥的分支重新收攏至單一控制流（如匯合後執行 `儲存檔案`）。
  - **無需同步等待：** **不等待**多個標記到達；哪一條分支先完成，就直接放行往下繼續。
- **核心要訣：** 菱形專門處理 **if-else** 互斥選擇，安全地切分並重聚邏輯路徑。

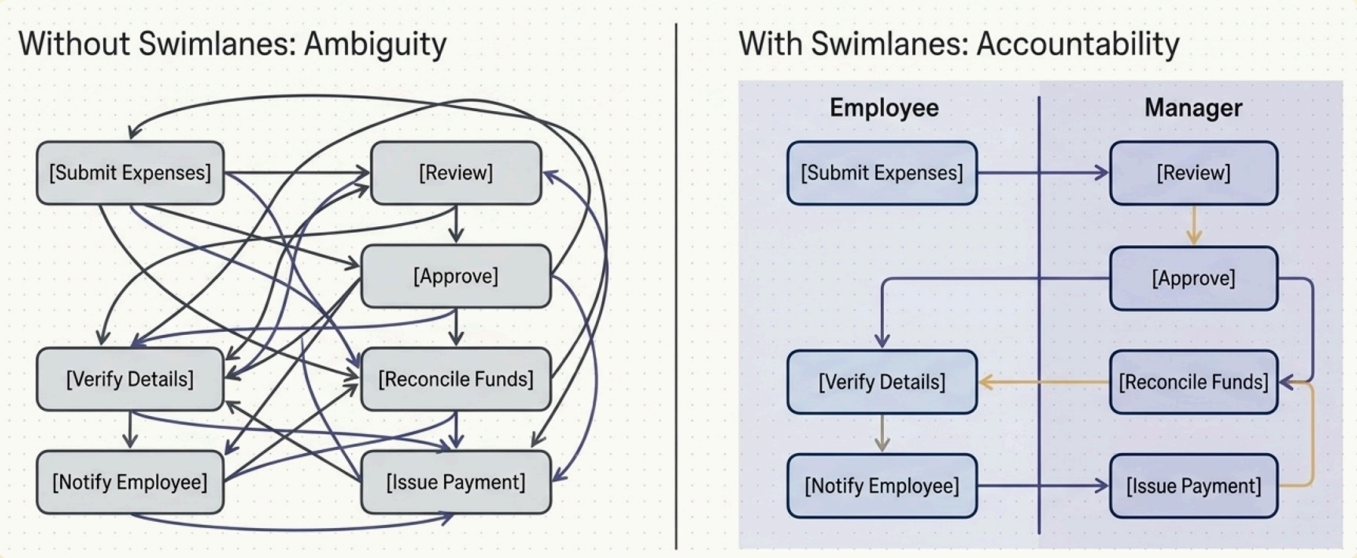
並行協作處理：分支與結合節點 (Fork & Join)



編排多執行緒平行任務與同步屏障：

- **分岔節點 (上方粗黑橫棒 ═)：**
  - **1 對多並行分流：** 接收單一輸入控制流，將其切分為**多條平行同時執行的執行緒**。
  - **同時並發執行：** `處理請款` 與 `備貨寄送` 同步展開，彼此互不阻塞。
- **結合節點 (下方粗黑橫棒 ═)：**
  - **同步等待屏障：** 所有匯入的平行分支皆必須執行完畢，輸出控制流 (`關閉訂單`) 才會放行。
  - **等待保證：** 若請款 1 秒搞定、包裝需 10 秒，請款執行緒會在結合棒前掛起等待，直到兩者齊全。
- **核心要訣：** Fork 啟動平行並發執行緒；Join 擔任屏障，等待所有執行緒全數抵達才放行。

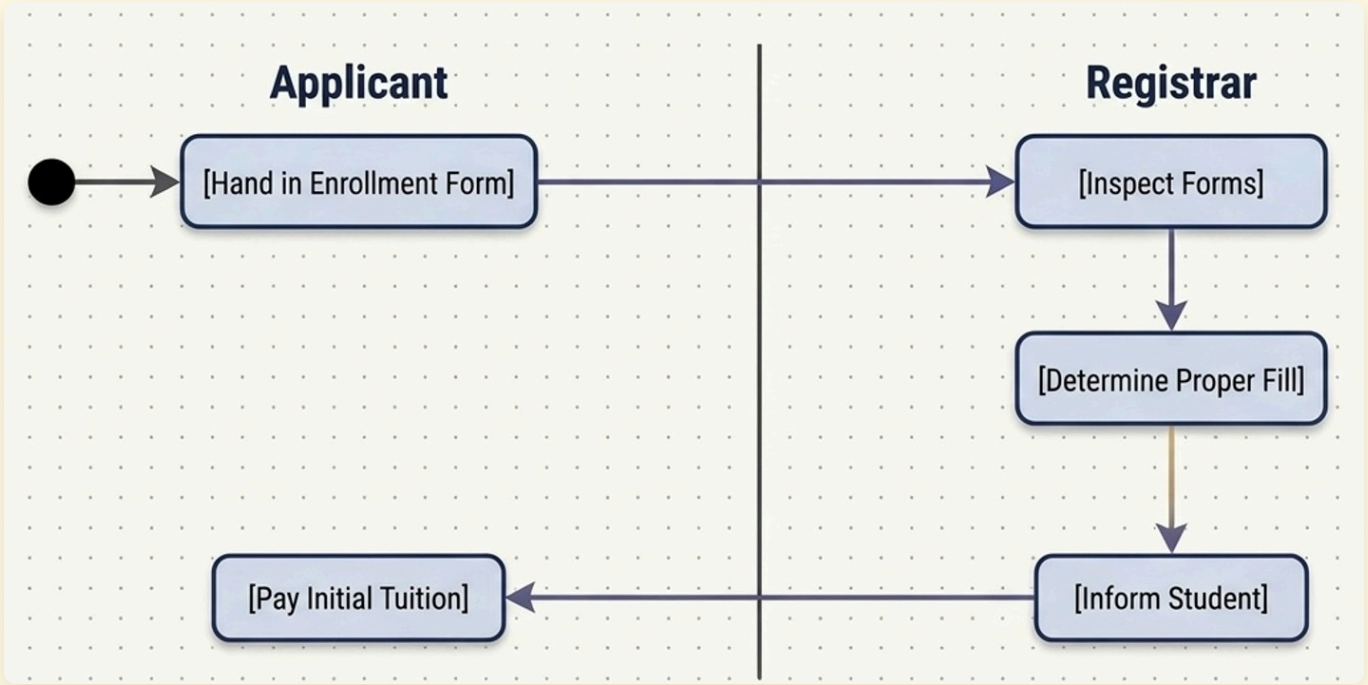
多角色責任歸屬困境 (The Multi-Actor Accountability Problem)



明確的角色權責劃分消除流程模糊：

- **核心困境：缺乏界線的流程混亂 (左側)：**
  - **無主任務交織：** 只知道「要做什麼」卻不知道「由誰負責」，流程勢必淪為一團混亂。
  - **責任真空盲點：** 義大利麵般的交錯箭頭，使員工與主管間的交接點模糊，導致工作踢皮球。
- **解法之道：泳道建立責任邊界 (右側)：**
  - **垂直分欄確立權責：** 以清楚的縱向泳道將職責切分為 `員工 (Employee)` 與 `經理 (Manager)`。
  - **精確跨欄交接：** 箭頭跨越泳道即代表一次清晰的責任移交（如員工申請 → 經理審核與撥款）。
- **核心法則：** 泳道將混亂的作業糾纏，轉化為權責清晰、邊界分明的工業級運作骨幹。

泳道與分割區：明確職責邊界 (Swimlanes & Partitions)



實務上的跨角色無縫交接塑模：

- **角色分區 (泳道 Swimlanes)：**
  - 將動作依執行主體進行群組：左側為 `申請學生 (Applicant)`，右側為 `註冊組長 (Registrar)`。
  - 清晰界定不同組織、部門或子系統間的協同邊界。
- **註冊流程的交接生命週期：**
  - **1. 發起動作：** 申請學生填寫並繳交入學表格 (`Hand in Enrollment Form`)。
  - **2. 跨界審核：** 控制流跨越邊界至註冊組進行資料查驗 (`Inspect Forms` → `Determine Proper Fill`)。
  - **3. 結果通知與回傳：** 註冊組通知學生審核結果 (`Inform Student`)，執行權交還學生以完成繳費 (`Pay Initial Tuition`)。
- **核心要訣：** 每條跨越泳道的分隔線箭頭，都是不同角色或系統模組間最關鍵的介面合約。

活動圖完整視覺語法分類表 (Visual Taxonomy of Activity Diagrams)

The Actors

Swimlane / Partition

Groups activities by actor or execution thread.

The Actions

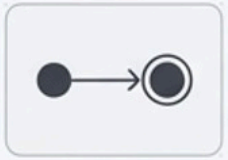
Action Node

A single task to be performed.

Object Node

Represents a connected physical or digital artifact.

The Directors



Initial & Final Nodes

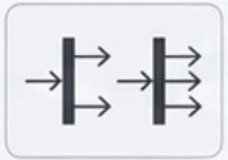
Start and stop all system flows.

Control & Object Flows

Sequence execution and data movement.

Decision & Merge Nodes

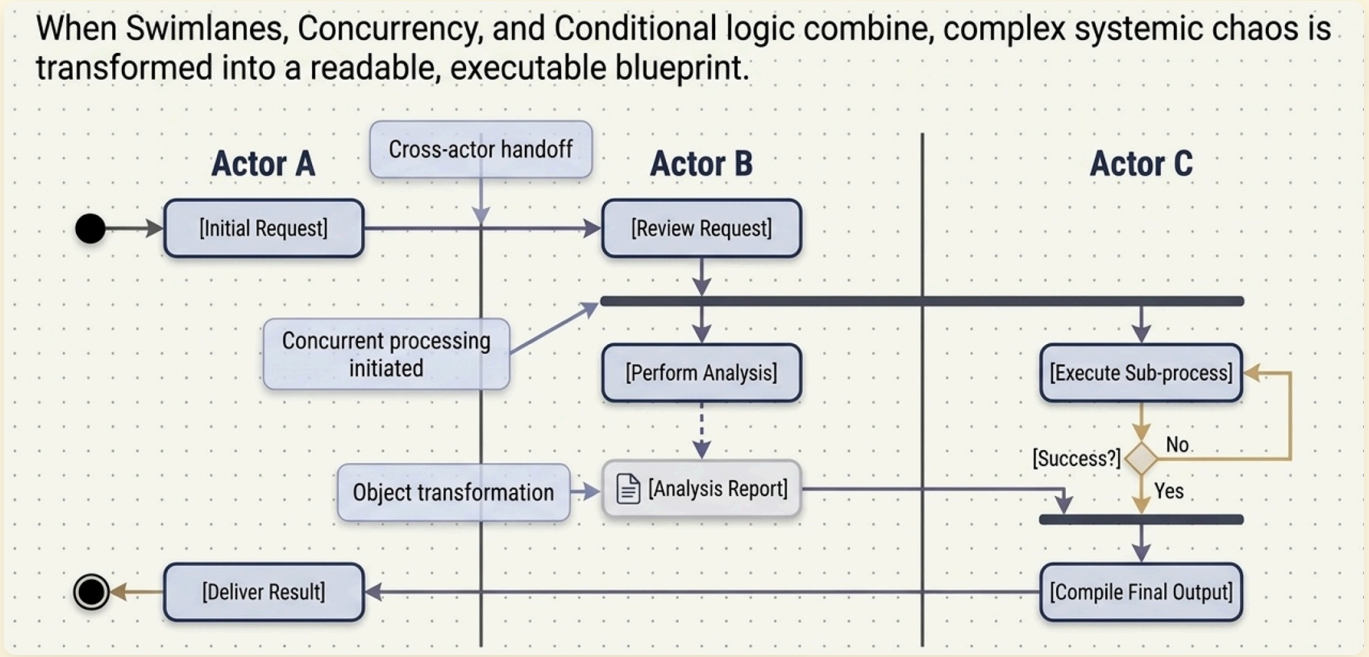
Split and return conditional paths.



Fork & Join Nodes

Split and return concurrent parallel paths.

統整工作流程模型：泳道、並行與條件邏輯 (Unified Workflow Model)



完美融合泳道、平行並行與條件邏輯：

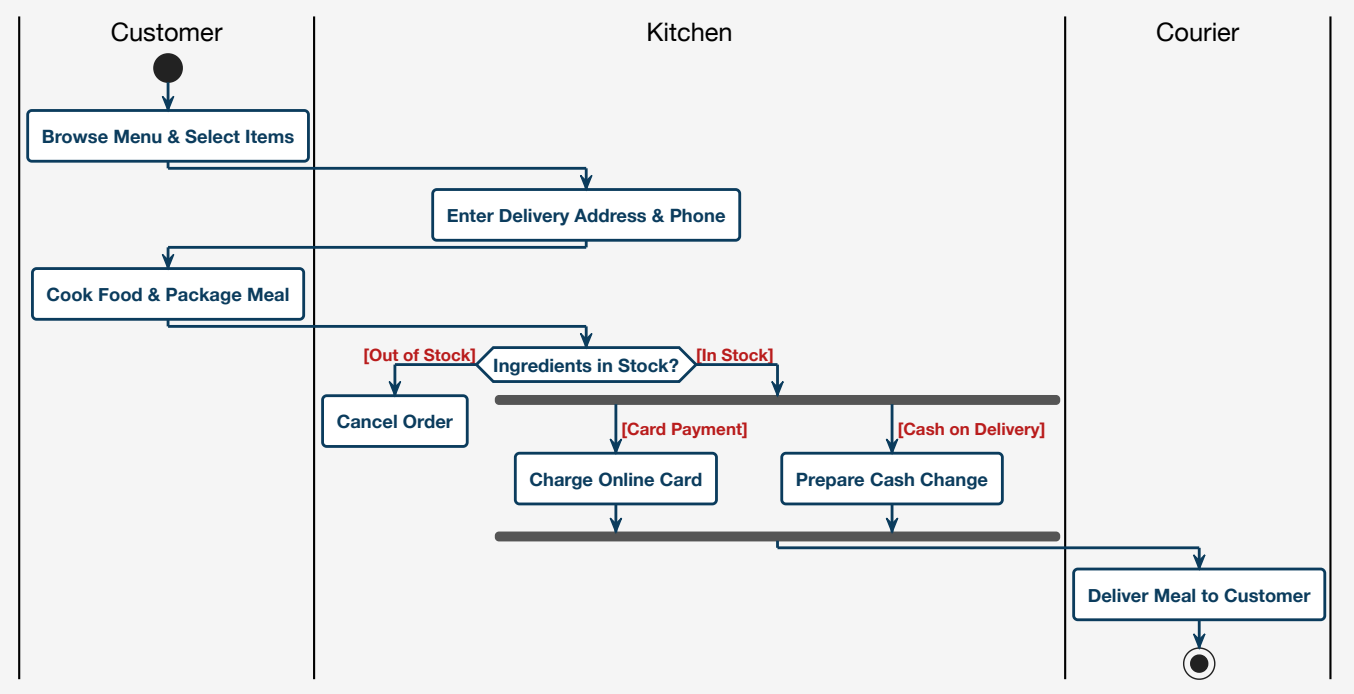
- **1. 跨角色交接 (Cross-Actor Handoff)：**
  - `角色 A` 發起初始請求 (`Initial Request`)；控制流跨越泳道交由 `角色 B` 進行審查 (`Review Request`)。
- **2. 並行處理與物件資料流 (Concurrency & Object Flow)：**
  - **Fork 分岔棒：** 同步並行觸發 `角色 B` (執行分析) 與 `角色 C` (執行子程序)。
  - **物件狀態轉換：** 角色 B 分析後產出實體資料物件 `[Analysis Report]`。
- **3. 條件重試迴圈與同步等待屏障 (Join Barrier)：**
  - **決策菱形：** 角色 C 依 `[Success?]` 判定；失敗則迴圈重試，成功才放行。
  - **Join 結合棒：** 嚴格等待「分析報告」與「子程序成功」兩者皆抵達，才編譯最終產出並遞交結果。
- **核心要訣：** 將泳道權責、並行執行緒、資料物件與迴圈收攏為工業級標準工程藍圖。

觀念檢核測驗 6 (CCQ 6)

在 UML 活動圖中，**分岔/結合 (Fork/Join 同步條)** 與 **決策/合併 (Decision/Merge 菱形)** 的核心差異為何？

- **A.** Fork/Join 用於將單一流程切分為多條「並行同時執行」的執行緒；Decision/Merge 則依據布林條件由多條路徑中「嚴格選擇其一 (互斥)」執行。
- **B.** Fork/Join 用於類別繼承；Decision/Merge 用於物件實體化。
- **C.** Fork/Join 只能處理資料流；Decision/Merge 只能處理錯誤例外。
- **D.** Fork/Join 必須等待人工確認；Decision/Merge 由系統計時器自動觸發。

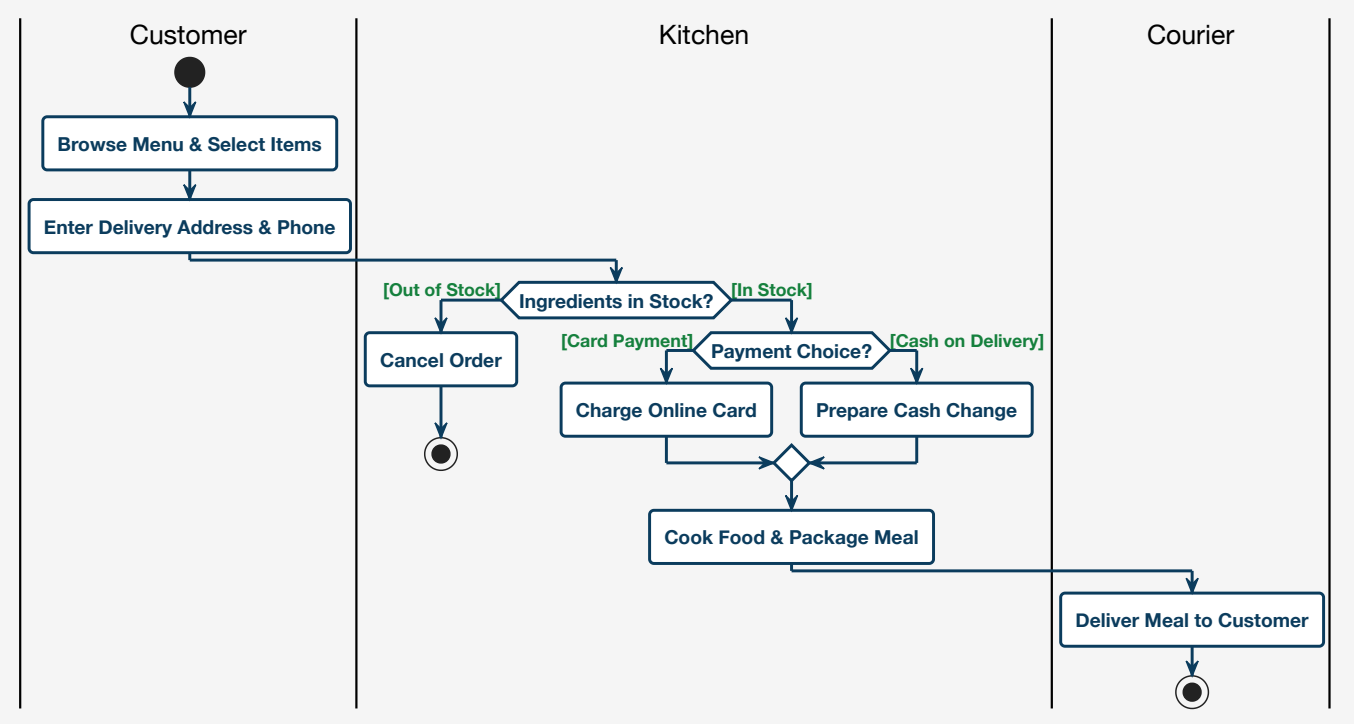
觀念診斷練習：找出這張活動圖的 4 大致命錯誤！



審查這份學生繪製的外送活動流程圖草稿：
你能從**並行控制**、**條件分支**、**泳道責任歸屬**中找出重大流程謬誤嗎？

- **1. 並行分支 vs. 條件判斷 (Fork vs. Decision)：**
  - 仔細觀察 Kitchen 泳道內的 `fork` 分支條，在並行分支上標註 `[Card Payment]` 與 `[Cash on Delivery]` 條件守衛有何本質錯誤？
- **2. 泳道責任與角色錯置 (Swimlane Inversion)：**
  - 檢視 `Customer` 與 `Kitchen` 泳道內的動作，哪些職責被荒謬地顛倒了？
- **3. 懸空流程與終止節點遺漏 (Dangling Flow)：**
  - 追蹤 `[Out of Stock]` 判定後的 `Cancel Order` 路徑，該流程結束時缺少了什麼關鍵節點？
- **4. 流程死鎖與同步語意 (Deadlock Hazard)：**
  - 若將互斥的分支誤用並行結合條 (Join Bar) 匯合，在執行期會引發什麼致命系統問題？

觀念診斷解答：活動圖 4 大致命錯誤剖析與修正



審查診斷結論與架構修正指引：

- **1. 並行分支條 vs. 條件決策菱形 (Fork vs. Decision)：**
  - `fork` 分支條會同時並行觸發所有路徑！信用卡與貨到付款是互斥選項，必須使用決策菱形 (Decision Diamond)。
- **2. 泳道角色責任完全顛倒 (Swimlane Inversion)：**
  - 角色責任配置荒謬：顧客負責輸入地址與電話；廚房負責烹調食物與打包餐點。
- **3. 懸空流程與終止節點遺漏 (Dangling Flow)：**
  - `Cancel Order` 後流程懸空終止，所有結束路徑必須明確連至活動終止節點 (◉)。
- **4. 匯合條引發執行期死鎖 (Deadlock Hazard)：**
  - 結合條 (Join Bar) 必須等待所有並行路徑的 Token 全數到達才可推進。將互斥分支接至 Join 會導致系統永久等待而死鎖。

AI 輔助指南：活動圖塑模 (Activity Modeling)

- **1. 角色設定與現代語法標準 (Persona & Syntax Standard)：**
  - 指示 AI 扮演「分散式工作流與業務流程架構師 (Workflow Architect)」。
  - **強制要求使用現代語法：** 指定採用 **PlantUML Activity Beta 新版語法** (`start`, `stop`, `if/else`, `fork/fork again/end fork`)，避免過時的舊版標記。
- **2. 泳道當責邊界嚴格分配 (Swimlane Accountability)：**
  - 要求明確定義泳道分區（如 `|顧客|`, `|餐廳廚房|`, `|調度引擎|`, `|外送員|`），動作必須歸屬在對應欄位中。
- **3. 並行與同步工程紀律：**
  - **分岔 (`fork` / `fork again`)：** 明確指示哪些流程必須平行運作（例如廚房做菜與系統配對司機）。
  - **同步結合 (`end fork` / `join`)：** 強制要求在取餐交件前必須同步，防止產生單獨無人外送的競爭條件 (Race Condition)。
- **4. 架構師人工審查清單 (Verification Checklist)：**
  - [ ] 每個 `fork` 是否都有對應的 `end fork` 結合屏障？
  - [ ] 跨泳道的控制流轉移是否合理且具備最小依賴？
  - [ ] 是否存在流程在結合棒無限等待、造成業務死結的缺陷？

AI 提示詞範例：外送平台廚房與外送並行活動模型

1. 角色設定與任務指令：

> 你是一位分散式工作流程架構師。
> 請根據以下外送履約流程，使用 `PlantUML activity beta` 語法產生具備泳道與並行的活動圖。

2. 塑模約束條件：

- 建立 4 個泳道：
  - `|顧客|`，`|餐廳廚房|`，
  - `|調度引擎|`，`|外送員|`
- 訂單付款後立即插入 `fork` 並行分岔：
  - 分支一：廚房接單、備餐並包裝餐點。
  - 分支二：調度系統規劃路徑、媒合司機、司機開車抵達店家。
- 使用 `end fork` (結合) 同步「包裝完成」與「司機抵達」。
- 結合後進行交餐配送，驗證 OTP 後 `stop`。

3. 輸入需求敘述 (Requirements Statement)：

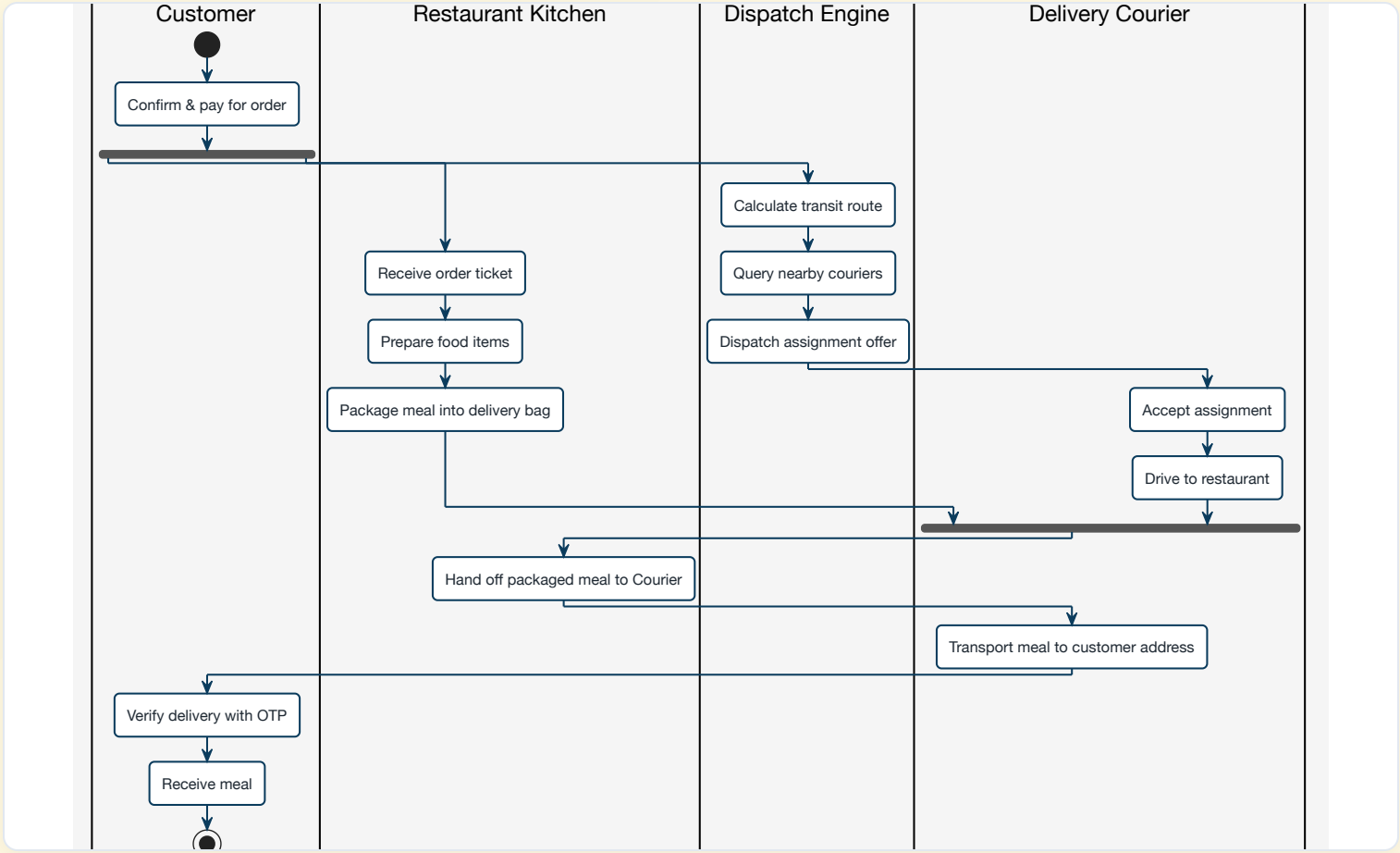
> *「顧客在 **|顧客|** 泳道送出訂單並完成信用卡付款。*
> *付款確認後，系統立即啟動兩條平行並行工作流：*
>
> - *在 **|餐廳廚房|** 泳道：廚房列印工單、廚師開始烹調餐點、完成後將食物裝盒並貼上封條。*
> - *同時間，在 **|調度引擎|** 泳道：系統計算配送里程、搜尋附近空閒外送員並派送接單推播；在 **|外送員|** 泳道，外送員點擊接單並騎車前往餐廳。*
>
> ***同步里程碑：** 外送員抵達櫃檯時，必須等待『餐點包裝完成』且『外送員已經抵店』兩項條件皆達成，才能進行實體交餐。*
> *交餐後，外送員騎車前往顧客地址，顧客輸入 OTP 驗證簽收，流程結束。」*

AI 生成之 PlantUML 與架構圖：外送平台活動流程模型

AI 生成之 PlantUML 規格代碼：

```
@startuml
|Customer|
start
:Confirm & pay for order;

fork
  |Restaurant Kitchen|
  :Receive order ticket;
  :Prepare food items;
  :Package meal into delivery bag;
fork again
  |Dispatch Engine|
  :Calculate transit route;
  :Query nearby couriers;
  :Dispatch assignment offer;
  |Delivery Courier|
  :Accept assignment;
  :Drive to restaurant;
end fork

|Restaurant Kitchen|
:Hand off packaged meal to Courier;

|Delivery Courier|
:Transport meal to customer address;
```

渲染產出之架構圖：



課堂活動：使用 AI 演進活動工作流 (雙人動手實作)

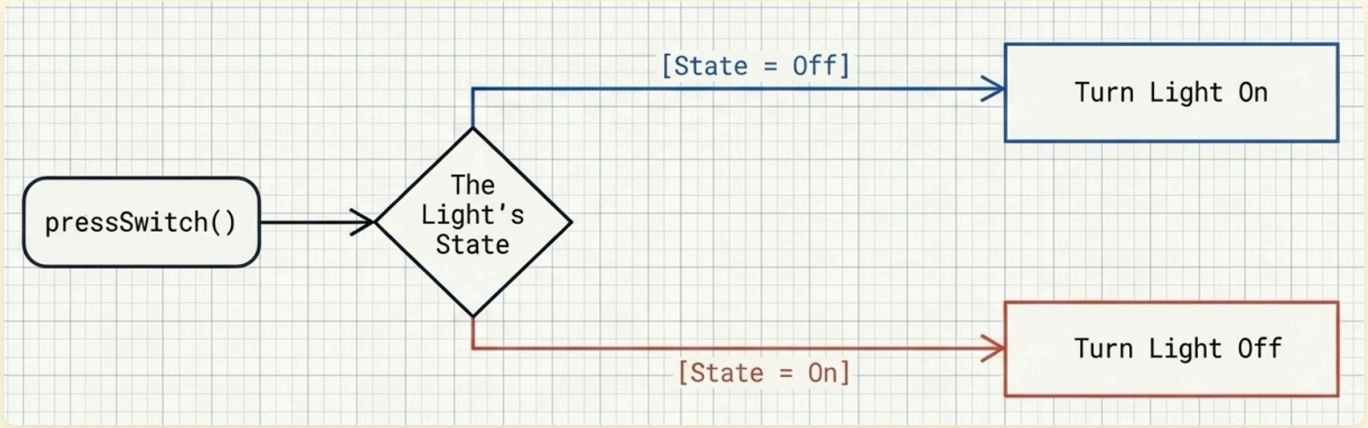
動手實作實驗：變更流程邏輯並審核 AI 產出 (4 分鐘)

- **1. 在活動圖提示詞中加入一條流程規則：**
  - *外送員重新派單：* 若原配對之外送員拒絕接單，系統自動重新搜尋媒合下一位司機。
  - *品管出餐檢驗：* 廚房品檢若發現餐點焦掉，必須重新下鍋製作後方能打包。
  - *顧客自取模式：* 若顧客選擇自取，完全略過外送員調度與派送泳道。
- **2. 要求 AI 重新生成：** 提示 LLM 更新對應的 PlantUML 活動圖代碼。
- **3. 渲染並嚴格審查：** 貼入 PlantText 渲染並審查：
  - 各項活動步驟是否皆正確坐落於所屬角色的**職責泳道**？
  - **平行分岔 (Fork) 與匯聚 (Join)** 是否成對匹配，杜絕死結 (Deadlock)？
  - **決策菱形 (Decision Diamonds)** 是否標註了完整互斥的轉移防護條件？

4.7 行為與狀態機模型

> *"軟體系統在動態執行中的行為，取決於它當下所處的狀態，以及觸發狀態轉移的事件。"*

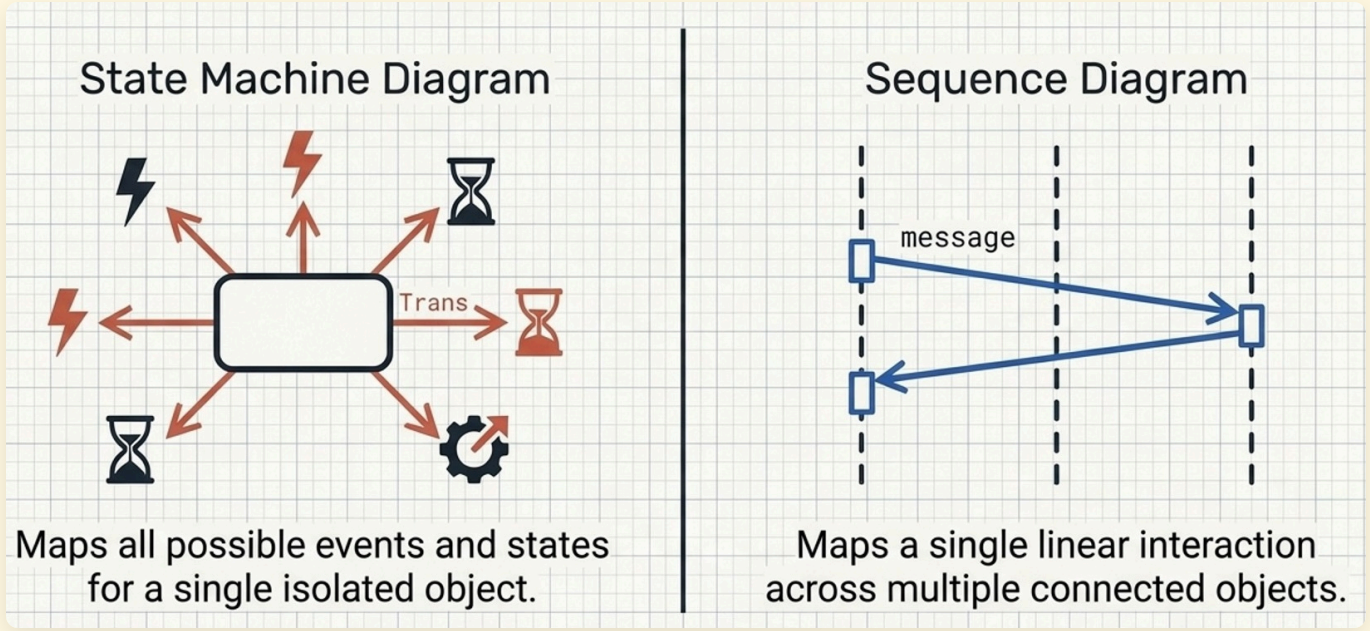
相同事件，因狀態不同而產生不同結果 (Same Event Different Results)



情境依存的動態行為響應：

- **相同刺激，截然不同的結果：**
  - 輸入完全相同的觸發事件 (`pressSwitch()`)，卻因系統當前內部狀態而產生互斥結果。
- **內部狀態條件分流：**
  - **當前狀態 = Off (藍色路徑)：** 觸發開關閉合電路，系統轉移至 `On` 狀態。
  - **當前狀態 = On (紅色路徑)：** 觸發開關切斷電路，系統轉移至 `Off` 狀態。
- **核心架構定律：** 反應式系統的運作行為由 `（當前狀態，輸入事件）→（目標狀態，動作執行）` 嚴格決定。

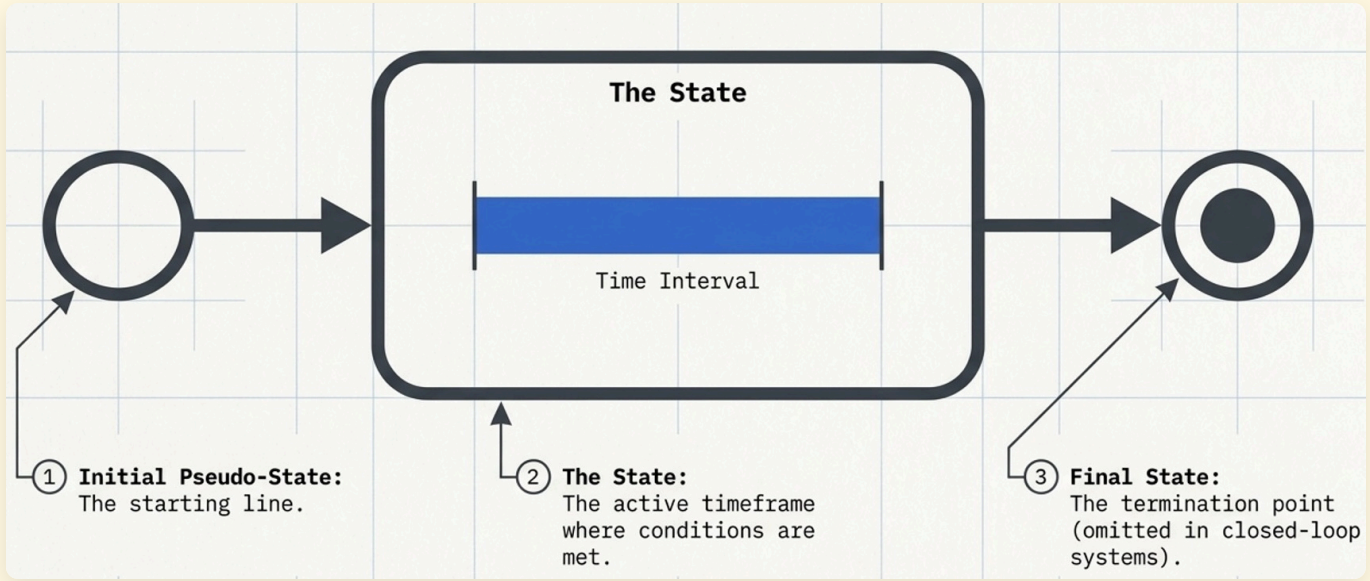
狀態機在 UML 生態系中的架構定位 (Positioning in UML)



行為塑模的兩大互補視角：

- **循序圖 (跨物件互動時序)：**
  - **塑模範圍：** 多個物件沿著生命線彼此協同互動。
  - **覆蓋度：** 專注追蹤**單一執行場景**（如標準成功路徑或特定例外）。
- **狀態機圖 (單一物件生命週期全貌)：**
  - **塑模範圍：** 聚焦**單一核心實體**（如 `Order` 或 `Connection`）。
  - **覆蓋度：** 窮舉該實體在整個生命週期中**所有可能的狀態、事件與轉移**。
- **架構黃金法則：** 跨模組協議用循序圖；核心實體的狀態約束與不變量用狀態機。

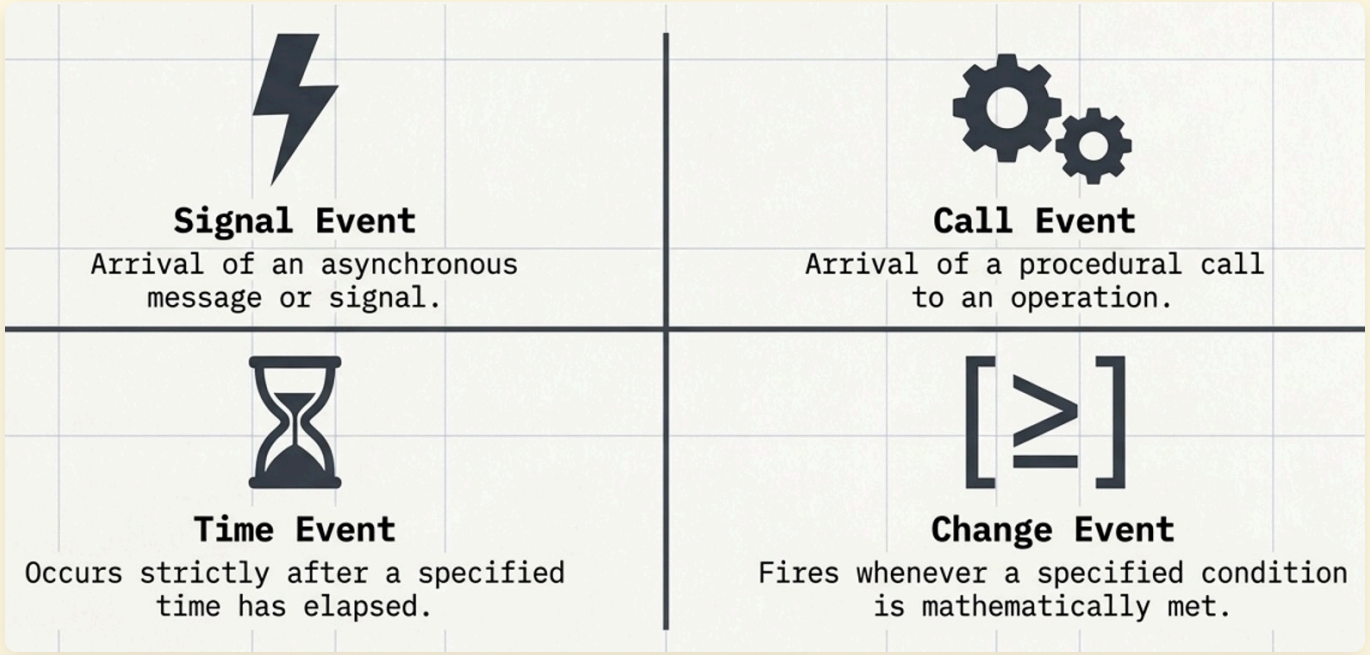
定義狀態：物件生命週期中的時間區間 (State Defined)



狀態的實質定義與生命週期區間：

- **1. 初始偽狀態 (● Initial Pseudo-State)：**
  - 非實質狀態；標記物件生命週期被點燃的預設進入起點。
- **2. 狀態本體 (圓角方塊 The State)：**
  - **持續性時間區間：** 物件滿足特定不變量、持續執行常駐活動，或等待外部事件的期間。
  - **狀態 vs. 動作：** 狀態具備時間長度，與瞬間完成的不可分割動作截然不同。
- **3. 終止狀態 (◉ Final State)：**
  - 靶心符號標記生命週期正式結束，進行資源回收與銷毀。
- **核心法則：** 伺服器常駐常態系統通常省略終點；商業單據實體則必有終止靶心。

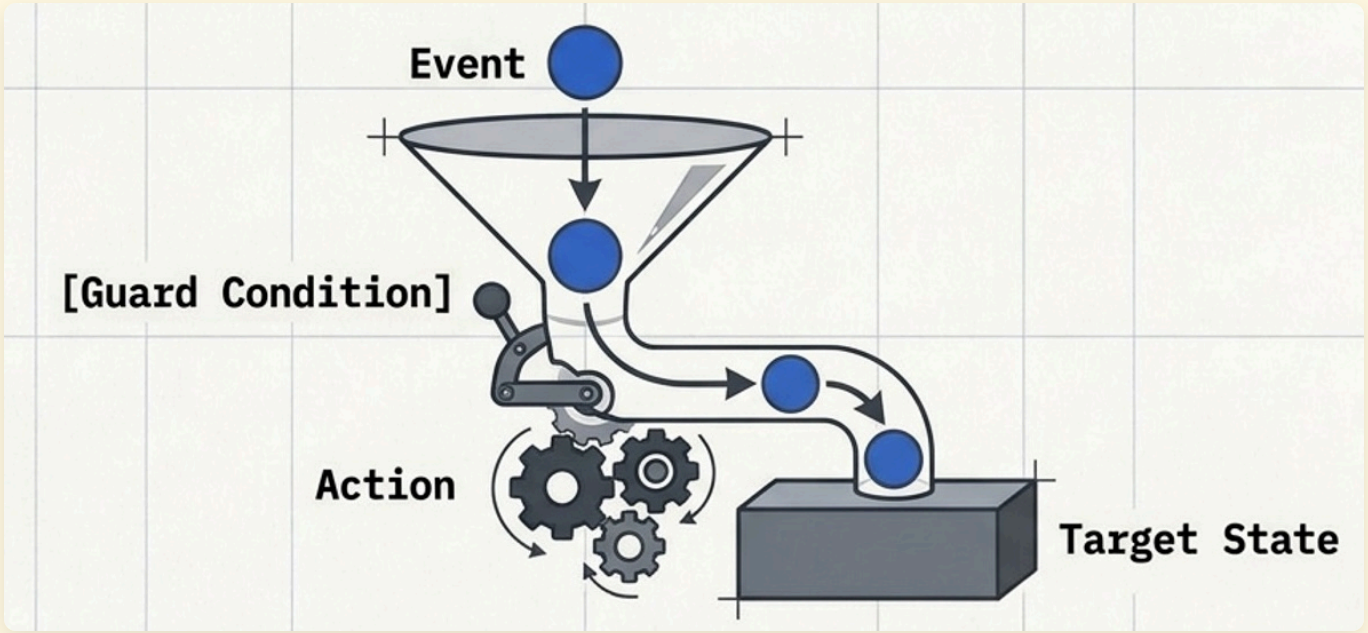
觸發狀態轉移的四大事件類型 (Four Triggers)



驅動狀態轉移的四大觸發事件：

- **1. 信號事件 (⚡ Signal Event)：**
  - 外部角色或子系統發送的非同步事件封包或訊息。
- **2. 呼叫事件 (⚙ Call Event)：**
  - 外部呼叫端同步調用該物件介面上的方法或操作。
- **3. 時間事件 (⏳ Time Event)：**
  - 基於時間流逝的計時器觸發（例如逾時機制 `after(30s)`）。
- **4. 變化事件 ([≥] Change Event)：**
  - 當布林條件運算式由偽轉真時瞬間引爆（例如 `when(temp > 100)`）。
  - **核心要訣：** 狀態機中所有的主動轉移，皆由這四種標準事件之一所點燃。

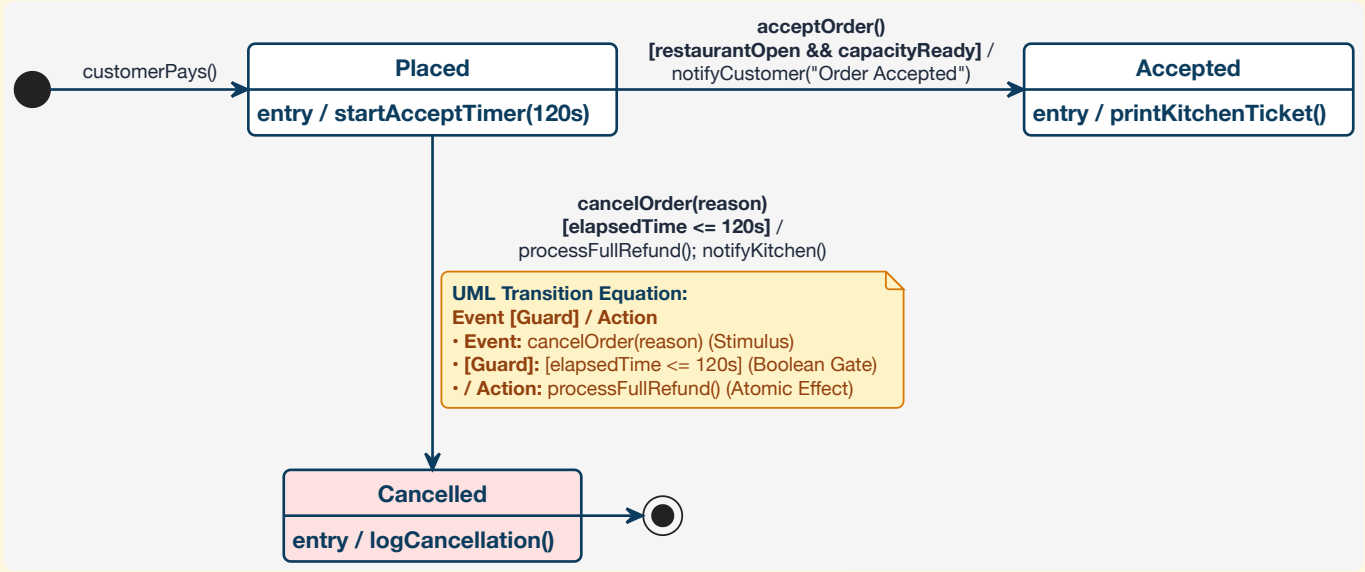
狀態轉移機制：事件、守衛條件與動作 (Mechanics of Transition)



狀態轉移漏斗管線運作機制：

- **標準轉移語法方程式：**
  - UML 標準格式：`事件名稱 ［守衛條件］ / 執行動作`。
- **步步拆解轉移漏斗：**
  - **1. 事件觸發：** 傳入事件如同滾珠落入狀態處理漏斗。
  - **2. 守衛閘門驗證：** 布林條件閘門判定；若為 false 則事件被忽略，不發生轉移。
  - **3. 動作齒輪運轉：** 觸發不可中斷之原子動作，於瞬間執行完畢。
  - **4. 目標狀態著陸：** 離開來源狀態，正式進入目標狀態。
  - **無觸發完成轉移：** 無事件標註之轉移，於內部活動完成後自動放行。

狀態轉移實例：外送訂單的事件、防衛與動作



外送訂單生命週期的轉移管線實例：

- **標準轉移語法結構：**
  - `cancelOrder(reason) [elapsedTime <= 120s] / processFullRefund(); notifyKitchen()`
- **1. 事件刺激 (`cancelOrder`)：**
  - 顧客從手機 App 發送主動取消請求。
- **2. 守衛條件 (`[elapsedTime <= 120s]`)：**
  - 嚴格限制下單後 120 秒內方可取消。若逾時（guard 為 false），轉移直接被阻擋，訂單保持在 `Placed`。
- **3. 動作執行 (`/ processFullRefund()`)：**
  - 轉移發生時瞬間原子性執行全額退款並通報餐廳廚房。
- **目標狀態著陸 (`Cancelled`)：**
  - 成功移轉至 `Cancelled` 並觸發 `entry / logCancellation()`。

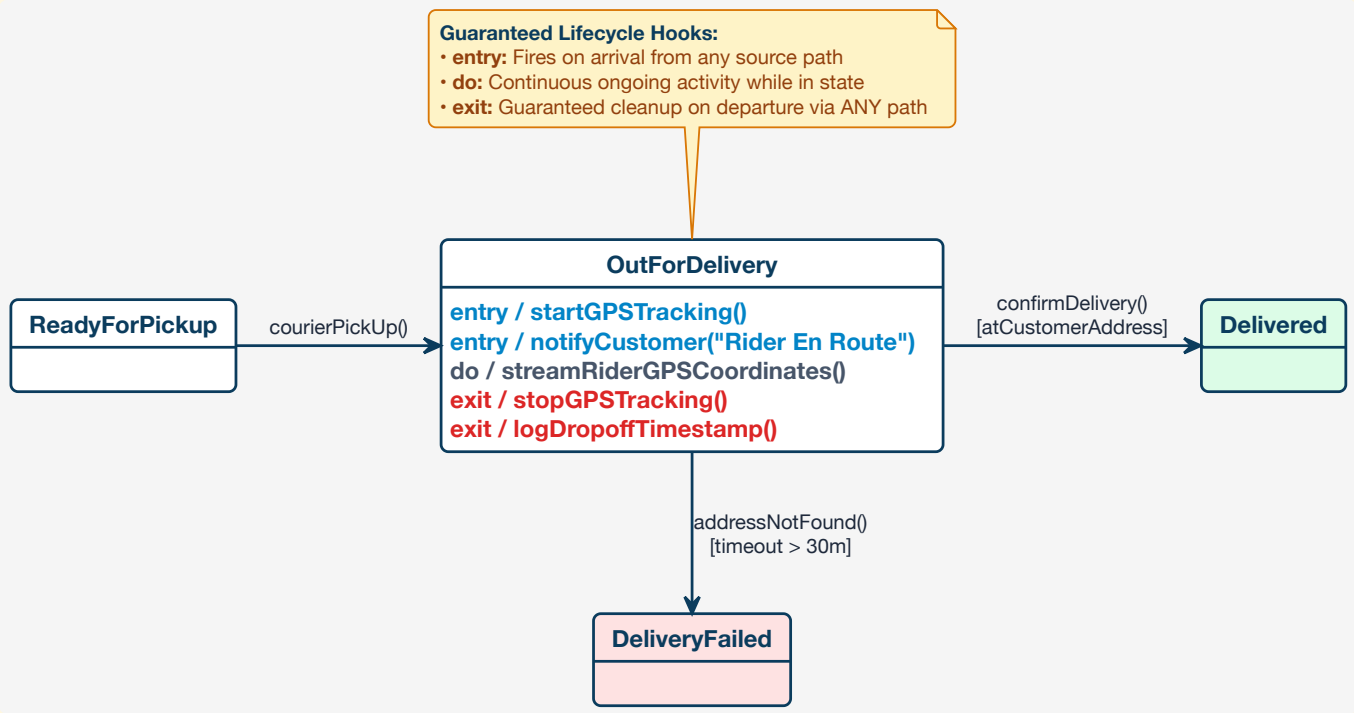
執行引擎：動作 (Action) vs. 活動 (Activity)

Feature	Action	Activity
Nature	Atomic Computation (Operation call, destroy object)	Non-atomic computation
Duration	Instantaneous	Ongoing (Runs to completion or indefinitely)
Interruptibility	NEVER interruptible	YES, can be interrupted by events
Triggers	On Entry, Do, On Event, On Exit	Sustained state presence

瞬間原子動作 vs. 持續可中斷活動：

- **動作 (Action，不可中斷原子運算)：**
  - **執行時間：** 瞬間完成（邏輯耗時為零）。
  - **中斷特性：** **絕對不可中斷**；一旦觸發必執行至結束，方能處理下個事件。
  - **放置位置：** 標註於轉移箭頭、`entry /` 或 `exit /`。
- **活動 (Activity，持續可中斷任務)：**
  - **執行時間：** 具時間跨度，狀態活躍期間持續運算。
  - **中斷特性：** **隨時可被中斷**；一旦外向轉移事件抵達立即強制中止。
  - **放置位置：** 宣告於狀態內部 `do / [activity]`。
  - **架構鐵律：** 嚴禁將耗時長任務置於 Action，否則將導致狀態機嚴重阻塞。

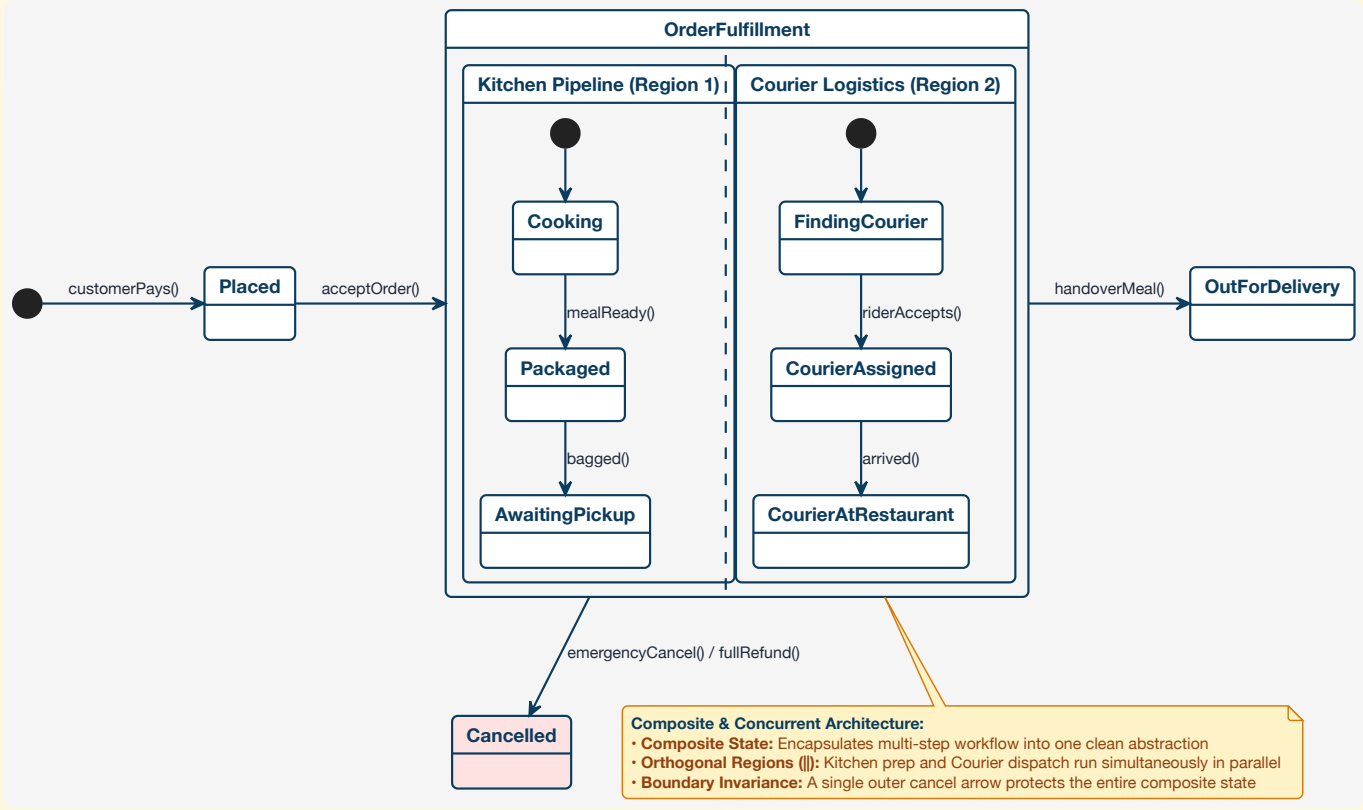
邊界執行：進入動作與離開動作 (Entry & Exit Actions)



外送配送狀態的生命週期保證機制：

- **進入動作 (`entry / startGPSTracking()` 等)：**
  - 外送員取餐跨入 `OutForDelivery` 的瞬間無條件觸發。
  - 立即啟動騎手手機 GPS 定位並推播通知「外送員已出發前往配送」。
- **持續活動 (`do / streamRiderGPSCoordinates()` 等)：**
  - 配送路程中持續串流即時座標；抵達顧客門口或逾時即刻中斷。
- **離開動作 (`exit / stopGPSTracking()` 等)：**
  - 無論外送成功 (`Delivered`) 或失敗 (`DeliveryFailed`)，離開時保證執行！
  - 關閉高耗電 GPS 晶片追蹤並寫入送達時間戳記。
- **架構價值：** 確保生命週期初始與清理邏輯完全內聚，免於在多條離開箭頭重複撰寫代碼。

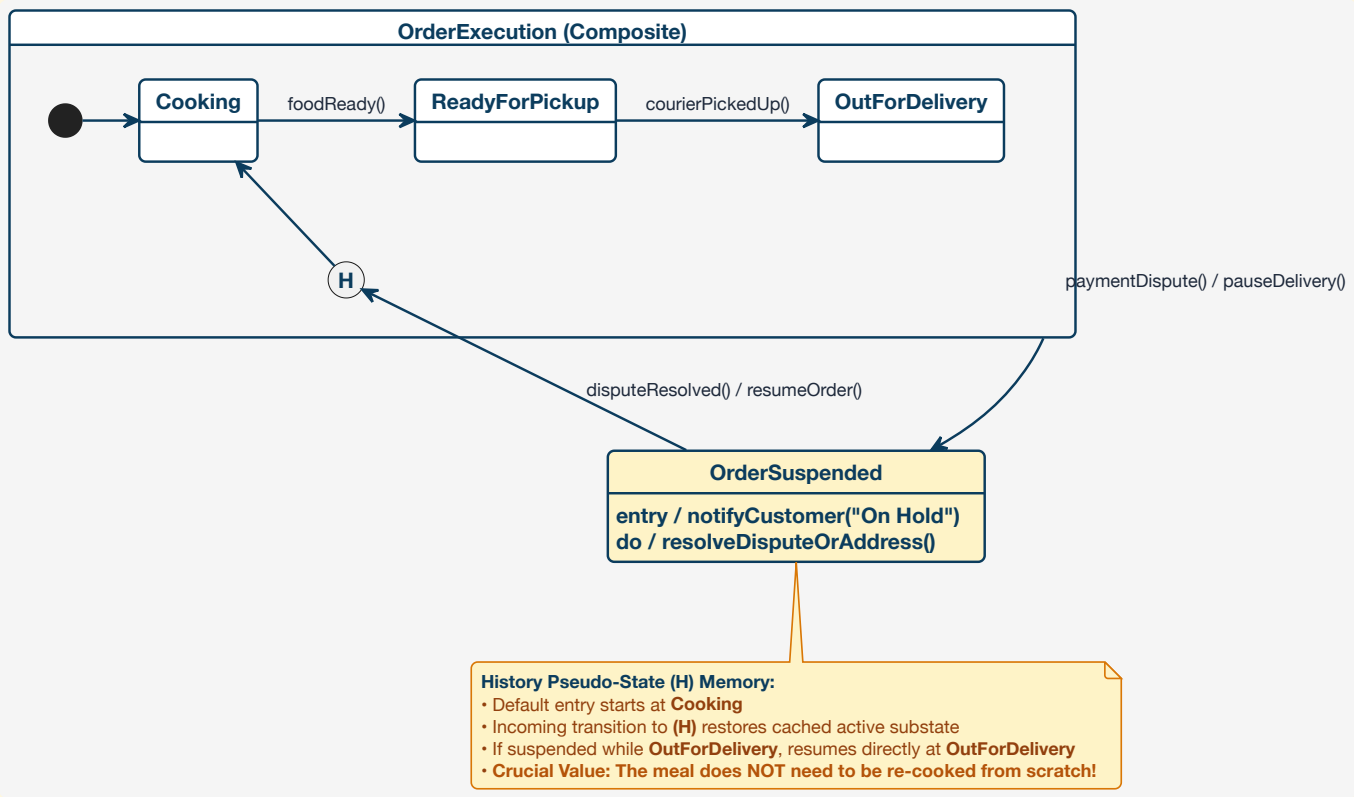
擴展架構：複合狀態與平行狀態 (Scaling Complexity)



以階層結構駕馭外送履約的狀態複雜度：

- **1. 簡單狀態 (`Placed`)：**
  - 單一原子狀態，等待餐廳確認接單。
- **2. 複合狀態 (`OrderFulfillment`)：**
  - 高階超狀態，封裝內部繁瑣之備餐與外送派件流程。
  - 外邊界轉移（如 `emergencyCancel() / fullRefund()`）統一適用於全體子狀態，大幅消除重複箭頭。
- **3. 正交並行區域 (`||` 並行軌道)：**
  - **廚房備餐管線 (Region 1)：** `Cooking` → `Packaged` → `AwaitingPickup`。
  - **外送員調度物流 (Region 2)：** `FindingCourier` → `CourierAssigned` → `CourierAtRestaurant`。
  - 雙管線同步且獨立並行運作！

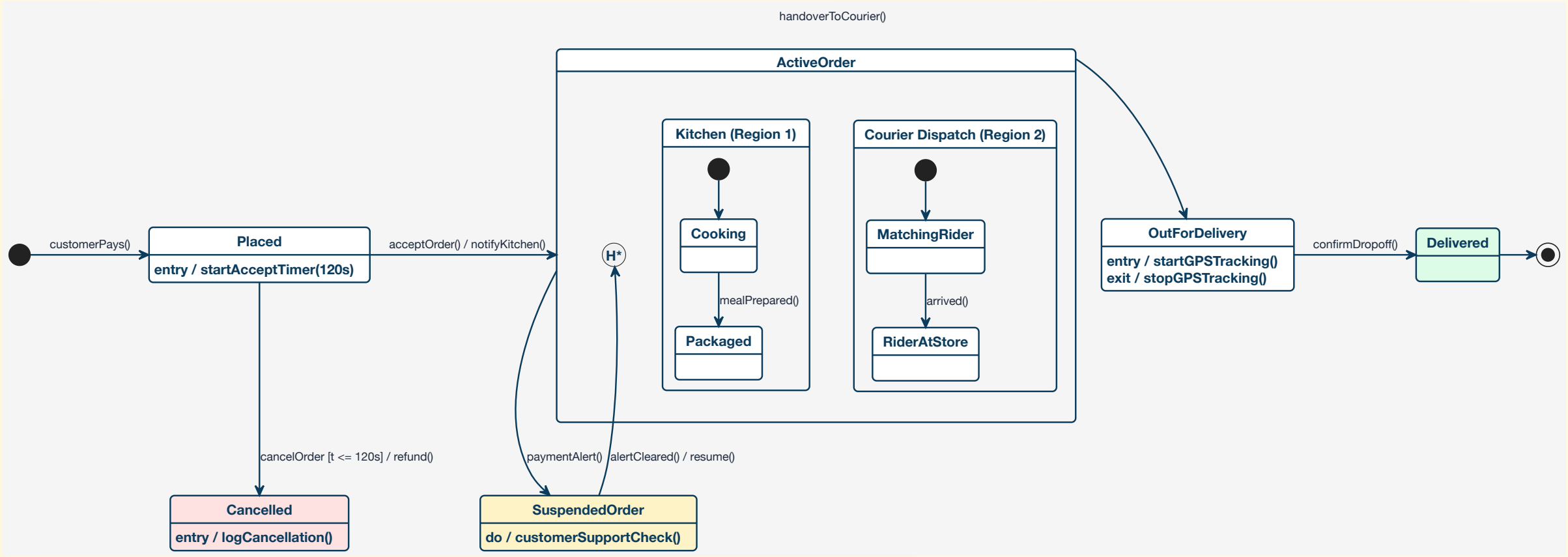
複合狀態與系統記憶 (History State)



歷史記憶節點 (H)：外送中斷後的精確復原：

- **預設進入流程：**
  - 新訂單正常進入 `OrderExecution` 時，一律從起點 `Cooking` 依序啟動。
- **歷史偽節點 `(H)` 的狀態快取：**
  - **中斷記憶：** 當訂單遭遇突發暫停（如信用卡盜刷查核、地址疑義），離開前自動記錄當下活躍狀態。
  - **精確跳轉復原：** 疑義排除後（`disputeResolved()`），箭頭指入 `(H)`，直接精準還原至 `OutForDelivery`！
- **極為關鍵的商業價值：**
  - **若沒有 (H)：** 系統將退回 `Cooking` 重新開煮，餐廳將浪費食材並重新備餐！
  - **具備 (H)：** 外送員直接在路上接續配送已經煮好的餐點。

整合藍圖：外送訂單完整生命週期狀態圖



動態模型對照：狀態機圖 vs. 活動圖 (State Machine vs. Activity)

> *兩者皆為 UML 動態行為模型，但從根本上回答了完全不同的架構問題。*

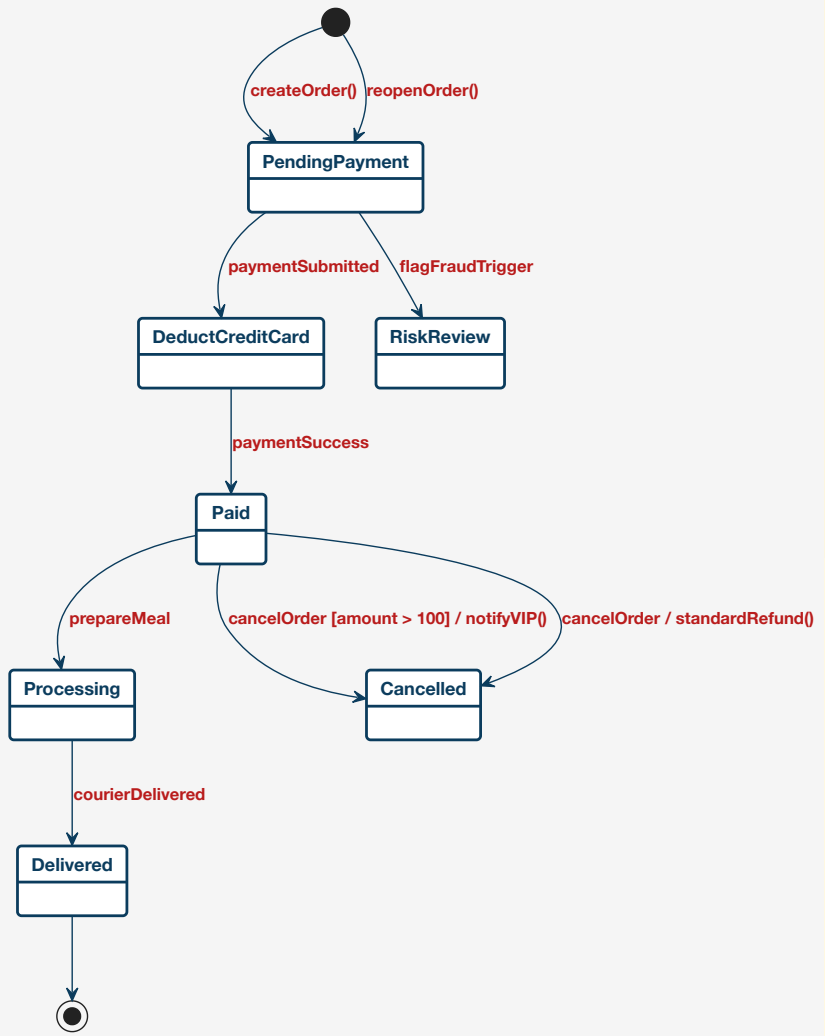
比較維度	狀態機圖 (State Machine)	活動圖 (Activity Diagram)
核心視角	**實體導向 / 被動反應 (Reactive)：** 聚焦於「單一實體」從生到死的生命週期。	**流程導向 / 程序推進 (Procedural)：** 聚焦於跨越「多個參與者與系統」的端到端工作流程。
核心節點	**狀態 (State)：** 實體處於等待事件或維持運作的靜止/持續模式。	**動作 (Action / Activity)：** 系統或人員正在執行的主動計算或業務操作。
轉移驅動方式	**事件驅動 (Event-Driven)：** 轉移必須等待外部或內部觸發事件 (`Event [Guard] / Action`)。	**完成驅動 (Completion-Driven)：** 動作執行完畢後自動沿著控制流/物件流推進下一步。
平行並行模型	**正交區域 (Orthogonal Regions)：** 複合狀態內的平行子狀態區塊 (並行軌道)。	**分岔與結合節點 (Fork & Join)：** 跨越泳道 (Swimlanes) 的明確流程分流與同步。
最佳應用場景	領域實體生命週期（如外送訂單、帳戶狀態）、通訊協定、IoT 設備控制。	跨系統業務流程（如結帳到出貨流程）、ETL 批次管線、使用案例流程展開。

觀念檢核測驗 7 (CCQ 7)

在 UML 狀態機圖中，**動作 (Action，如 transition 上的 `/ refund()` 或 `entry /`) 與 活動 (Activity，如 `do /`)** 最根本的執行語義差異為何？

- **A.** Action 是瞬間完成、不可中斷的原子性操作；Activity 是耗時進行的持續運算，且可被外來事件隨時中斷。
- **B.** Action 只能寫 Python；Activity 只能寫 SQL。
- **C.** Action 只能在進入狀態時執行；Activity 只能在離開狀態時執行。
- **D.** Action 專門處理例外失敗；Activity 專門處理成功路徑。

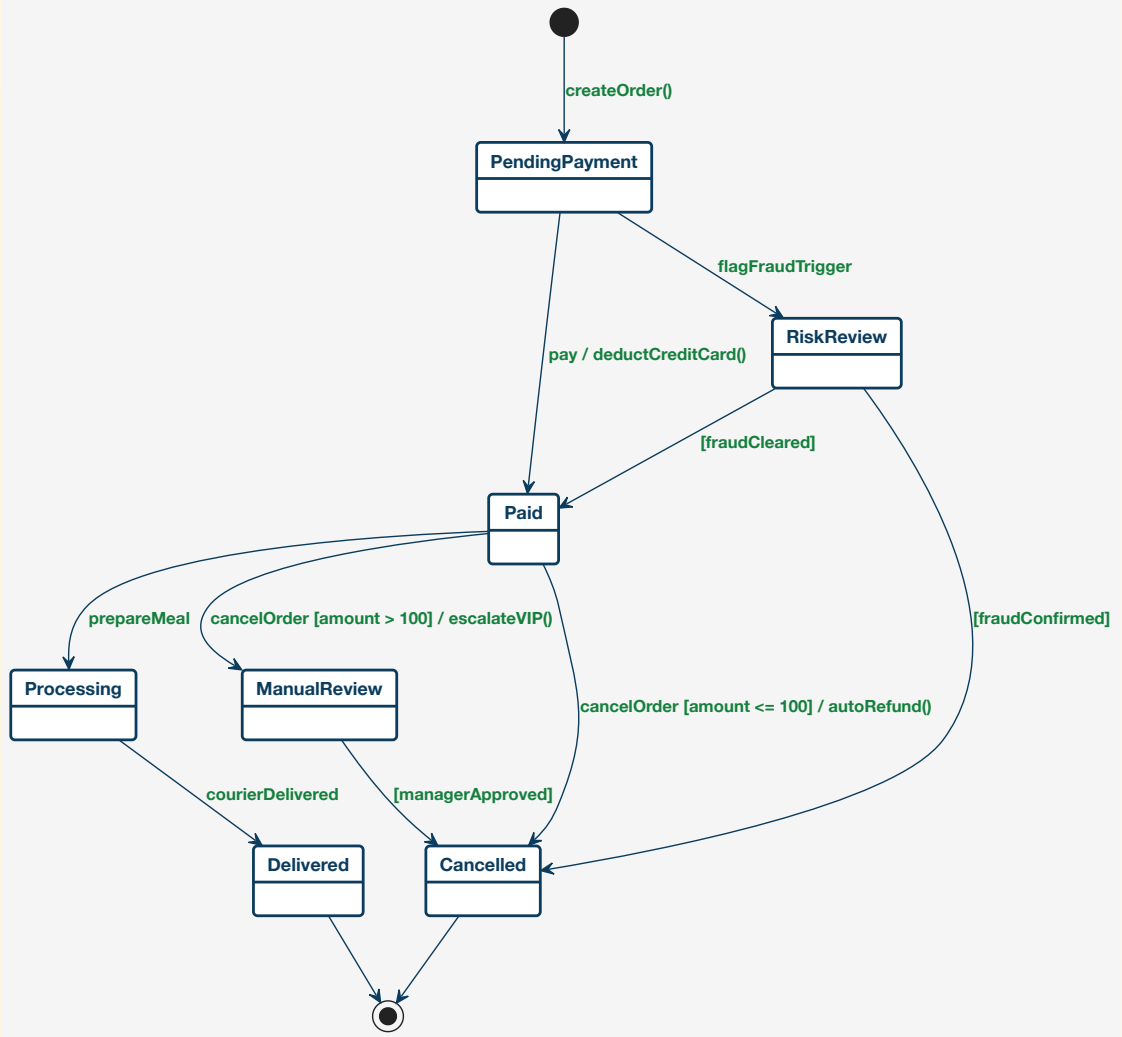
觀念診斷練習：找出這份狀態機模型的生命週期謬誤



審查這份學生繪製的訂單生命週期狀態機圖草稿：
你能從**狀態語意**、**轉移條件**、**終止節點約束**中找出重大架構謬誤嗎？

- **1. 狀態 vs. 瞬時動作 (State vs. Action)：**
  - 檢視 `DeductCreditCard`，將一個耗時為零的瞬時動作繪製為持久狀態存在什麼語意錯誤？
- **2. 黑洞陷阱狀態 (Black Hole State)：**
  - 觀察 `RiskReview` 狀態，訂單一旦進入該狀態後會遭遇什麼致命問題？
- **3. 非確定性轉移衝突 (Non-Deterministic Conflict)：**
  - 檢視離開 `Paid` 狀態的兩條 `cancelOrder` 轉移線：當 `amount == 150` 時，系統該執行哪一條？
- **4. 終止狀態不可逆性 (Final State Invariant)：**
  - 從終止狀態 `[*]` 伸出箭頭 `reopenOrder()` 重新回到 `PendingPayment`，違反了什麼核心不變量？

觀念診斷解答：狀態機模型生命週期謬誤剖析與修正



審查診斷結論與架構修正指引：

- **1. 持久狀態 vs. 瞬時動作 (State vs. Action)：**
  - `DeductCreditCard` 是耗時為零的瞬時動作，應作為轉移線上的效果 (`pay / deductCard()`)，絕不可畫為狀態方塊。
- **2. 黑洞陷阱狀態 (Black Hole State)：**
  - `RiskReview` 只有進入箭頭、零離開路徑，訂單一旦進入審查即永久卡死，缺少核准或拒絕出口。
- **3. 非確定性轉移衝突 (Non-Deterministic Conflict)：**
  - 離開 `Paid` 的兩條 `cancelOrder` 線未設定互斥守衛，當 `amount == 150` 時兩條皆滿足，造成不可預期的競態衝突。
- **4. 終止狀態不可逆約束 (Final State Invariant)：**
  - 從終止狀態 `[*]` 拉出 `reopenOrder()` 嚴重違反 UML 規範；終止狀態代表生命週期結束，不可有離開箭頭。

AI 輔助指南：狀態機塑模 (State Machine Modeling)

- **1. 角色設定與形式化檢驗紀律 (Persona & Formal FSM Discipline)：**
  - 指示 AI 扮演「形式化系統驗證與有限狀態機 (FSM) 專家」。
  - **狀態命名嚴格紀律：** 狀態名稱必須代表物件的「客觀穩態條件」，一律採用形容詞或過去分詞（如 `Placed`, `Preparing`, `Delivered`），嚴禁使用動詞當作狀態名稱！
- **2. 轉換語法與防禦性不變量約束：**
  - **法定語法公式：** 強制要求轉換連線必須遵循 `事件名稱 [守衛條件] / 執行動作`。
  - **進入/離開動作：** 對於具備資源配置或通知的關鍵狀態，明確定義 `entry /` 與 `exit /`。
  - **嚴格封鎖非法路徑：** 明確宣告哪些狀態之間「絕對禁止跳轉」（例如配送中嚴禁直接點擊取消）。
- **3. 架構師人工審查清單 (Verification Checklist)：**
  - [ ] 是否存在無法抵達的「孤島狀態」或進得去出不來的「死結陷阱」？
  - [ ] 守衛條件 `[guard]` 是否互斥且覆蓋所有可能？
  - [ ] 終止狀態 (`[*]`) 是否正確對應至業務的終結？

AI 提示詞範例：外送平台訂單生命週期狀態機模型

1. 角色設定與任務指令：

> 你是一位形式化驗證工程師。
> 請根據以下訂單生命週期規則，產生標準合規的
> PlantUML 狀態機圖 (State Machine Diagram)。

2. 塑模約束條件：

- 狀態清單：`Placed`, `Accepted`, `Preparing`, `ReadyForPickup`, `OutForDelivery`, `Delivered`, `Cancelled`。
- 轉換標籤必須遵循 `Event [Guard] / Action`。
- `OutForDelivery` 狀態必須包含 `entry / startGPSTracking()`。
- 僅允許在 `Placed` 與 `Accepted` 狀態下取消 `[cancellationWindow <= 120s]`；進入 `Preparing` 後嚴禁取消。

3. 輸入需求敘述 (Requirements Statement)：

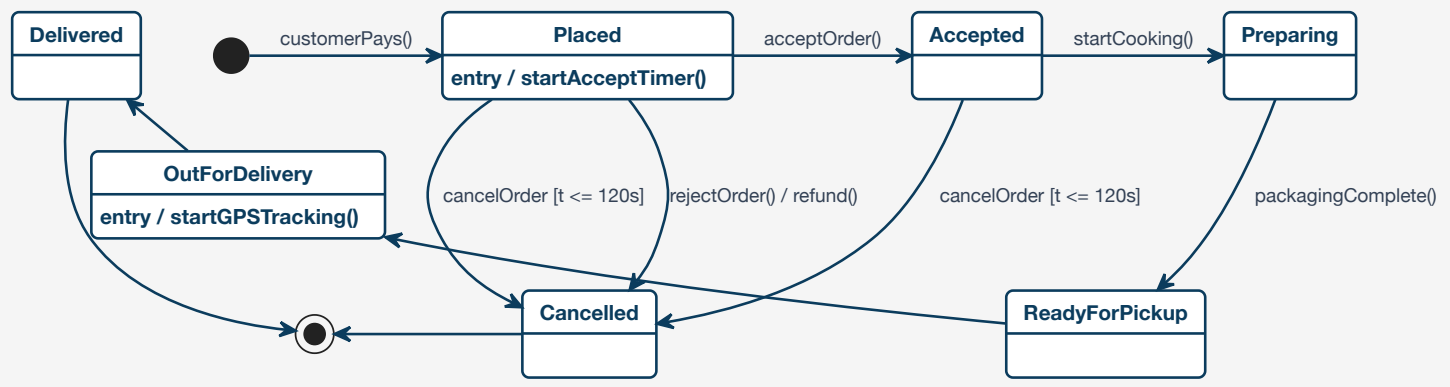
> 「**Order** 生命週期從 `[*]` 開始，顧客完成扣款後進入 **Placed** (已下單)。
> 在 **Placed** 狀態下，餐廳點擊接受則觸發 `acceptOrder` 進入 **Accepted**；若餐廳拒絕則觸發 `rejectOrder` 進入 **Cancelled** `/ issueFullRefund()`。
> 顧客在 **Placed** 或 **Accepted** 狀態下可主動觸發 `cancelOrder`，但必須滿足守衛條件 `[timeElapsed <= 120s]` 才能轉入 **Cancelled**。
> 當廚房點擊開始做菜，觸發 `startCooking` 進入 **Preparing**；此時取消路徑完全鎖死。
> 餐點打包完成後觸發 `packagingComplete` 進入 **ReadyForPickup**。
> 外送員抵達掃描條碼後觸發 `courierPickedUp` 進入 **OutForDelivery**，同時執行 `entry / startGPSTracking()`。
> 顧客輸入 OTP 簽收後觸發 `confirmDropoff` 進入終止狀態 **Delivered**，最後轉入 `[*]`。」

AI 生成之 PlantUML 與架構圖：外送平台狀態機模型

AI 生成之 PlantUML 規格代碼：

```
@startuml
[*] -right-> Placed : customerPays()

state Placed {
  Placed : entry / startAcceptTimer()
}
state OutForDelivery {
  OutForDelivery : entry / startGPSTracking()
}

Placed -right-> Accepted : acceptOrder()
Accepted -right-> Preparing : startCooking()
Preparing -down-> ReadyForPickup : packagingComplete()
ReadyForPickup -left-> OutForDelivery : courierPickedUp()
OutForDelivery -left-> Delivered : confirmDropoff()
Delivered -down-> [*]

Placed -down-> Cancelled : rejectOrder() / refund()
Placed -down-> Cancelled : cancelOrder [t <= 120s]
Accepted -down-> Cancelled : cancelOrder [t <= 120s]
Cancelled -left-> [*]
@enduml
```

渲染產出之架構圖：



課堂活動：使用 AI 演進狀態機生命週期 (雙人動手實作)

動手實作實驗：變更生命週期並審核 AI 產出 (4 分鐘)

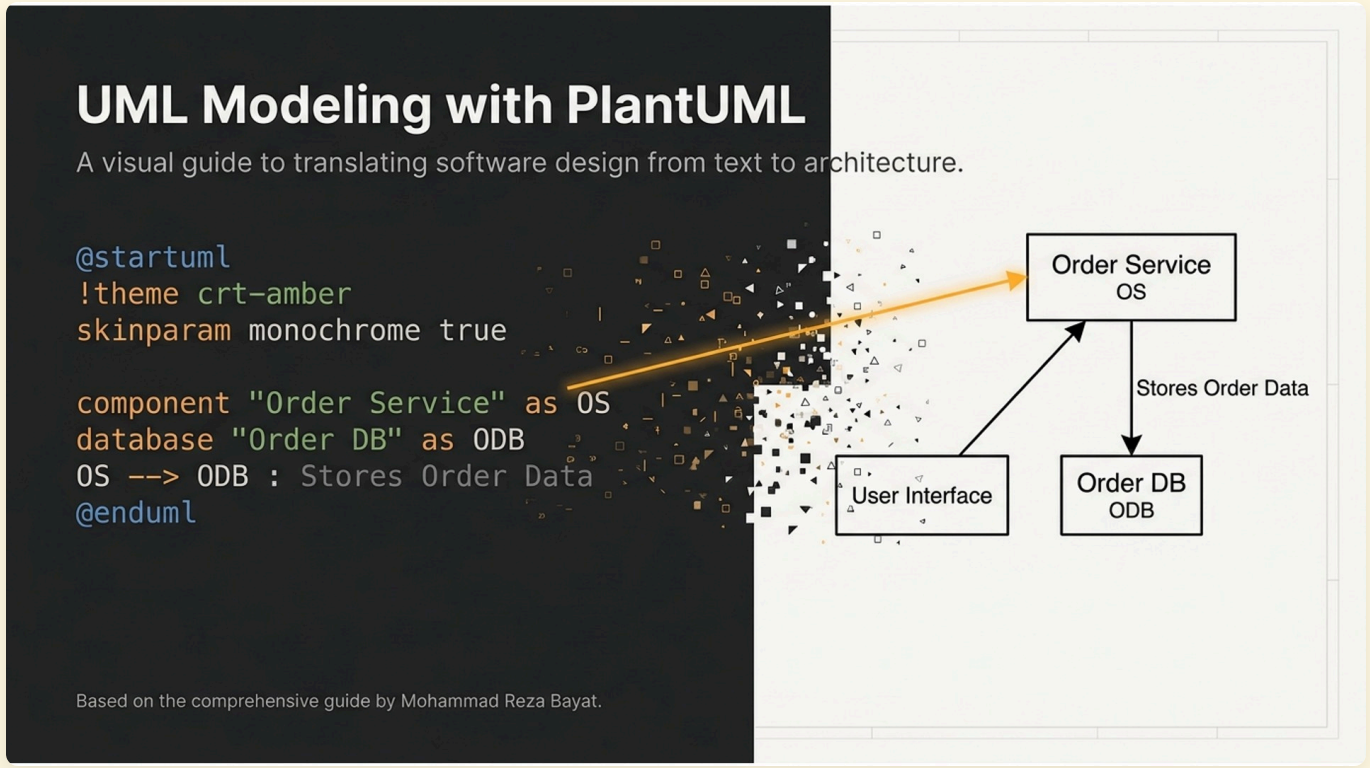
- **1. 在狀態機提示詞中加入一個生命週期不變量 (Invariant)：**
  - *司機途中棄單：* 若外送員中途車輛故障棄單，訂單狀態退回 `ReadyForPickup` 重新派單。
  - *顧客延期配送：* 允許在 `Preparing` 備餐中申請延後外送指定時段。
  - *外送失敗處理：* 抵達門前若聯絡不上顧客超過 10 分鐘，轉移至 `FailedDelivery` 並原車退回。
- **2. 要求 AI 重新生成：** 提示 LLM 更新對應的 PlantUML 狀態機代碼。
- **3. 渲染並嚴格審查：** 貼入 PlantText 渲染並審查：
  - AI 是否維護**狀態命名規範**（使用被動式/形容詞，不使用動詞）？
  - **非法狀態轉移**（如餐點在路上卻被隨意取消）是否被嚴格防護？
  - **進入/離開動作**與轉移事件之邏輯前後是否完全一致？

4.8 PlantUML 宣告式文字塑模

> *"告別手動拖拉圖框的痛苦。架構即代碼，讓軟體模型與程式碼在 Git 倉儲中一同進化。"*

線上編輯器 PlantText 線上實時沙盒 (planttext.com) →

PlantUML：以代碼驅動架構 (Code-Driven Architecture)



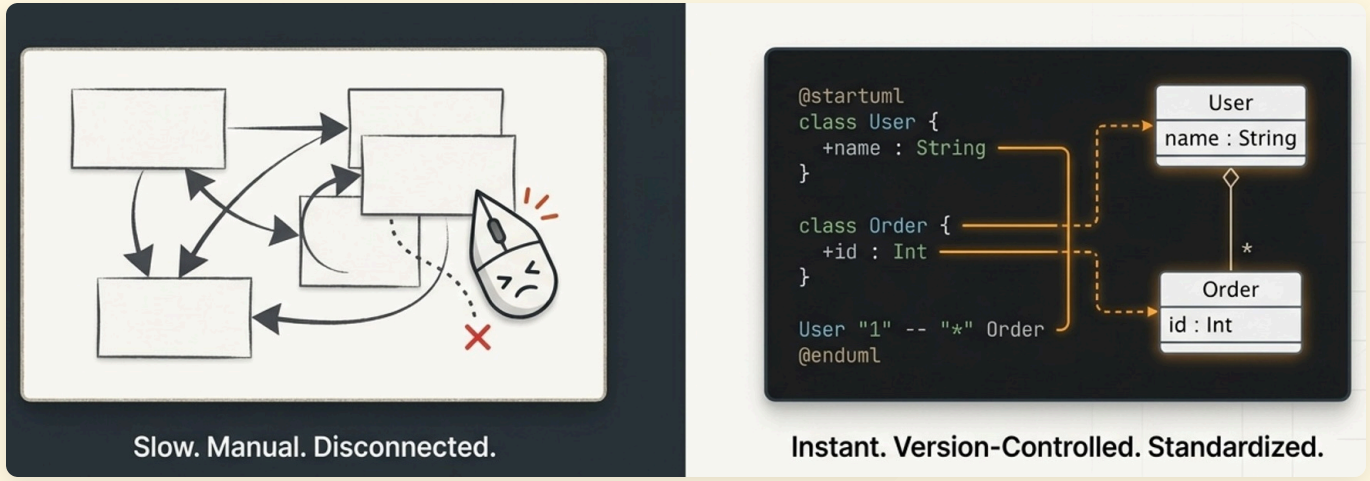
圖表即代碼（以純文字建構架構）：

- **宣告式架構 (Declarative Architecture)：**
  - 撰寫簡潔易讀的純文字腳本，定義系統元件與資料流向。
- **PlantUML 語法範例：**

```
component "Order Service" as OS
database "Order DB" as ODB
OS --> ODB : Stores Order Data
```

- **自動排版引擎 (Automated Layout)：**
  - 內建 Graphviz 自動計算座標、連線與對齊，消除手動拖拉負擔。
- **核心工程價值：** 取代脆弱的繪圖工具，使架構能納入 Git 進行 PR 審查。

終結圖框拖曳地獄，直接撰寫架構 (Stop Dragging Boxes)



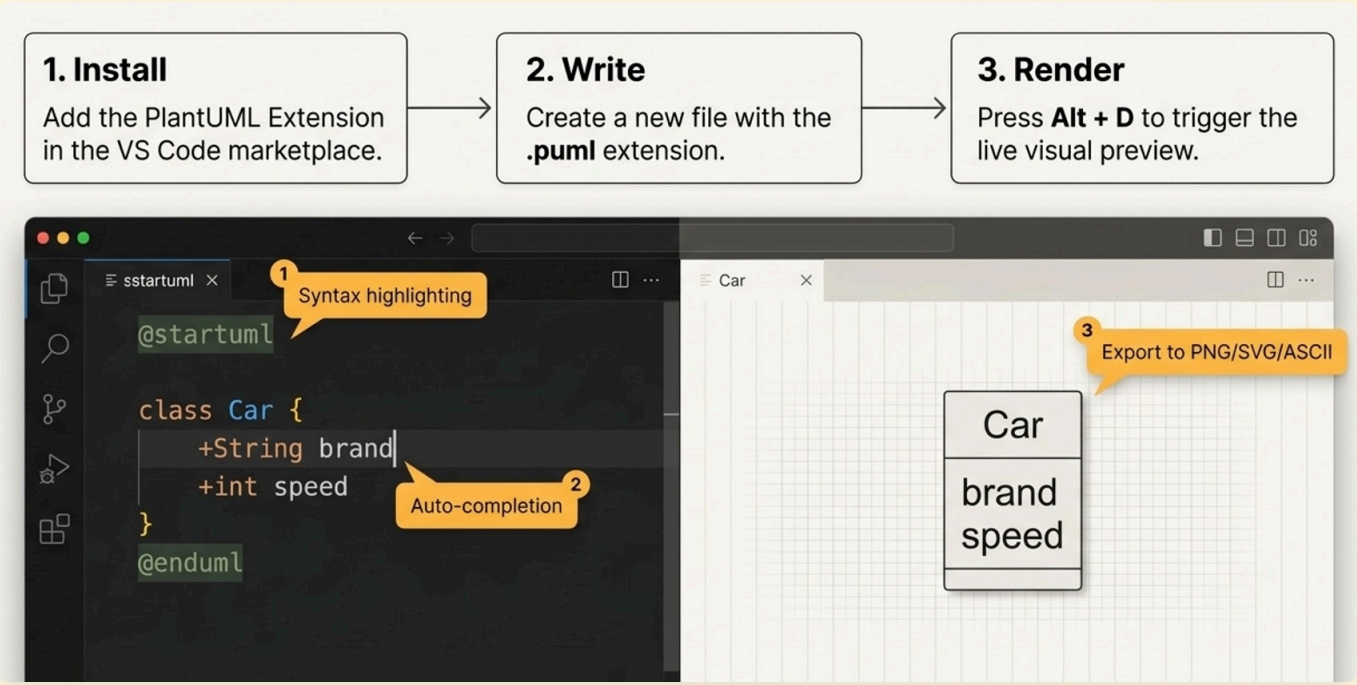
消除排版視覺摩擦與維護負債：

- **傳統 GUI 繪圖方式（左側 — 視覺疲勞）：**
  - 白板與拖拉繪圖工具繁瑣緩慢，與原始碼完全脫節。
  - 每增修一個方塊，都需耗時重新手動微調數十條連接線。
- **PlantUML 文字塑模（右側 — 即時標準化）：**

```
class User { +name : String }
class Order { +id : Int }
User "1" -- "*" Order
```

- **工程優勢：**
  - 100% 專注於軟體抽象概念與結構關聯，告別調像素的無效勞動。

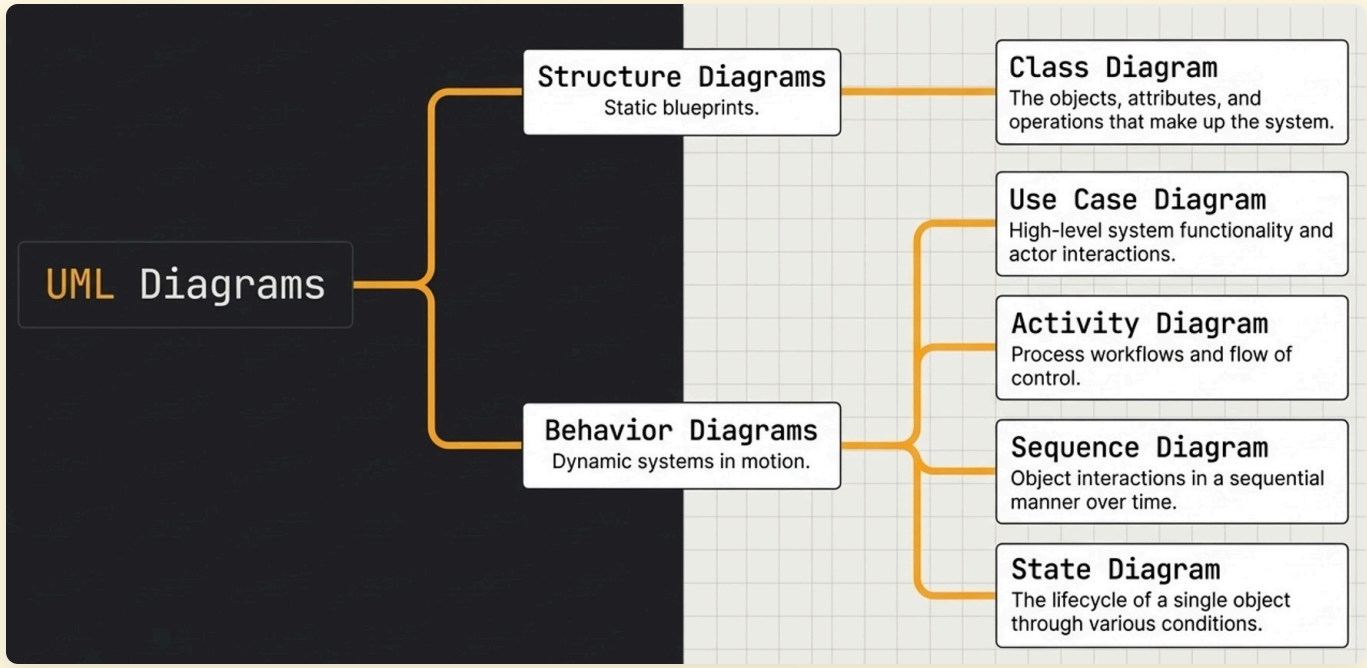
極簡環境配置：VS Code 即時渲染預覽 (Frictionless Setup)



三步驟啟動本地端高效架構繪圖：

- **1. 安裝擴充套件 (Install)：**
  - 於 VS Code 市集安裝官方 **PlantUML** 擴充功能。
- **2. 撰寫架構代碼 (Write)：**
  - 建立任意 `.puml` 檔案，享有語法高亮與智慧自動補全。
- **3. 即時渲染預覽 (Render)：**
  - 按下 `Alt + D`（Mac: `Option + D`）啟動雙欄即時預覽。
- **多格式直接匯出：**
  - 原生支援直接匯出為向量 **SVG**、高解析度 **PNG** 或終端友善的 **ASCII 文字圖**。

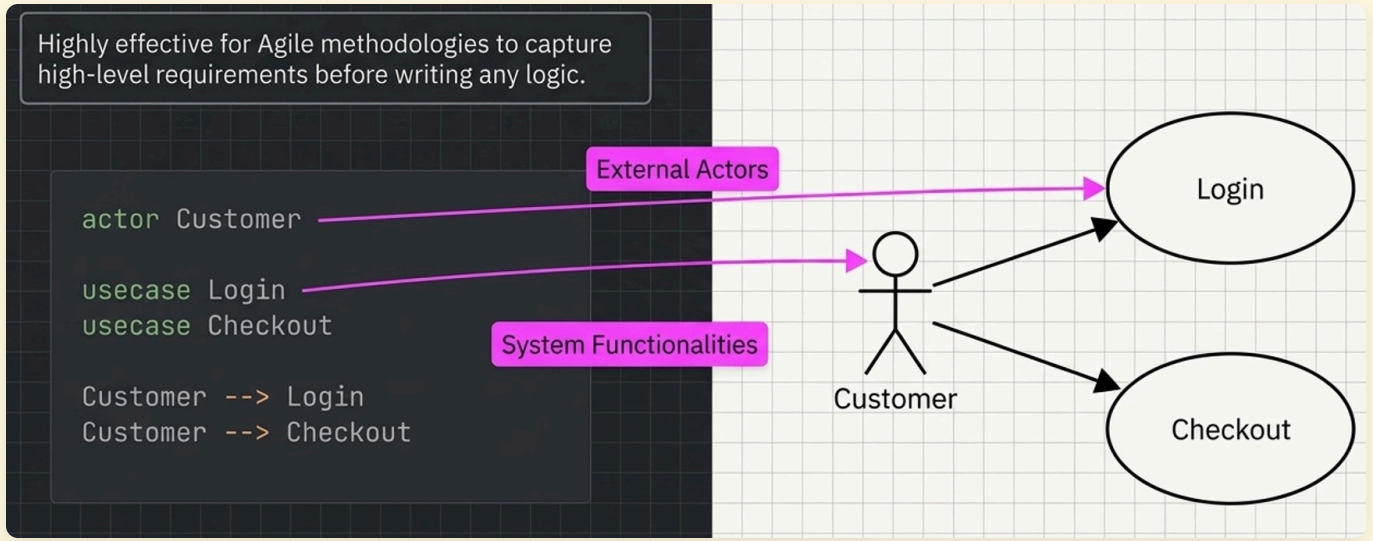
架構師的五大核心視角全景 (5 Essential Lenses)



以統一語法覆蓋系統各維度架構視圖：

- **結構圖 (靜態骨幹 - Static Backbone)：**
  - **類別圖 (Class Diagram)：** 定義領域實體、屬性、操作與靜態關聯。
- **行為圖 (動態行為 - Dynamic Motion)：**
  - **使用案例圖 (Use Case)：** 外部參與者互動與高階系統邊界。
  - **活動圖 (Activity)：** 業務工作流程、並行分支與泳道。
  - **循序圖 (Sequence)：** 跨生命線之時間時序訊息傳遞。
  - **狀態圖 (State)：** 單一實體生命週期狀態與事件躍遷。
- **核心價值：** 一套語法完整貫通五大關鍵 UML 視圖。

純文字宣告使用案例與功能邊界 (Use Case Syntax)



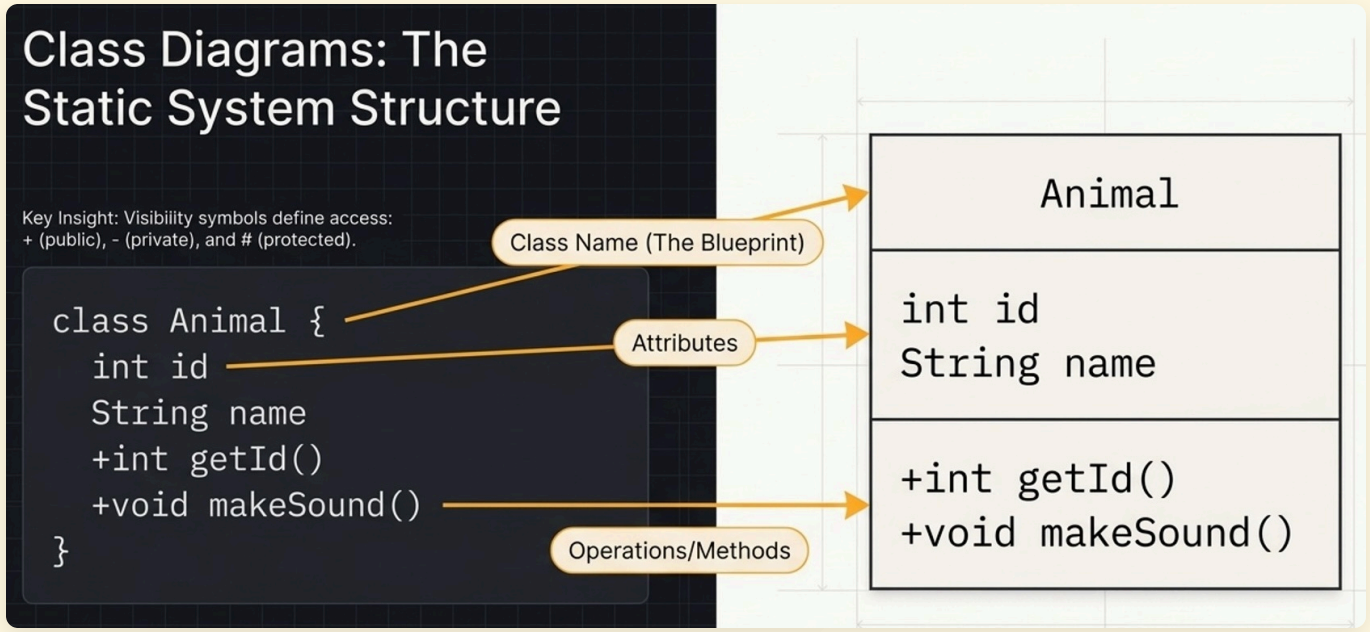
以純文字定義功能需求與系統範圍：

- **直覺語法關鍵字：**
  - `actor Customer`：宣告外部參與者（自動渲染為火柴人）。
  - `usecase Login`：宣告系統功能案例（渲染為邊界橢圓）。
  - `Customer --> Login`：建立參與者與功能之通訊關聯。
- **PlantUML 腳本範例：**

```
actor Customer
usecase Login
usecase Checkout
Customer --> Login
Customer --> Checkout
```

- **敏捷優勢：** 於編寫程式前迅速完成使用者故事對齊與範疇鎖定。

類別圖語法：直接對映物件導向結構 (Class Syntax)



精確鏡像物件導向代碼結構：

- **標準三層類別方框 (3 Compartments)：**
  - 頂部：類別名稱 (`class Animal`)。
  - 中部：屬性欄位 (`int id`, `String name`)。
  - 底部：操作方法 (`+int getId()`, `+void makeSound()`)。
- **可見度修飾詞符號 (Visibility)：**
  - `+` Public | `-` Private | `#` Protected | `~` Package
- **PlantUML 腳本範例：**

```
class Animal {
  int id
  String name
  +int getId()
  +void makeSound()
}
```

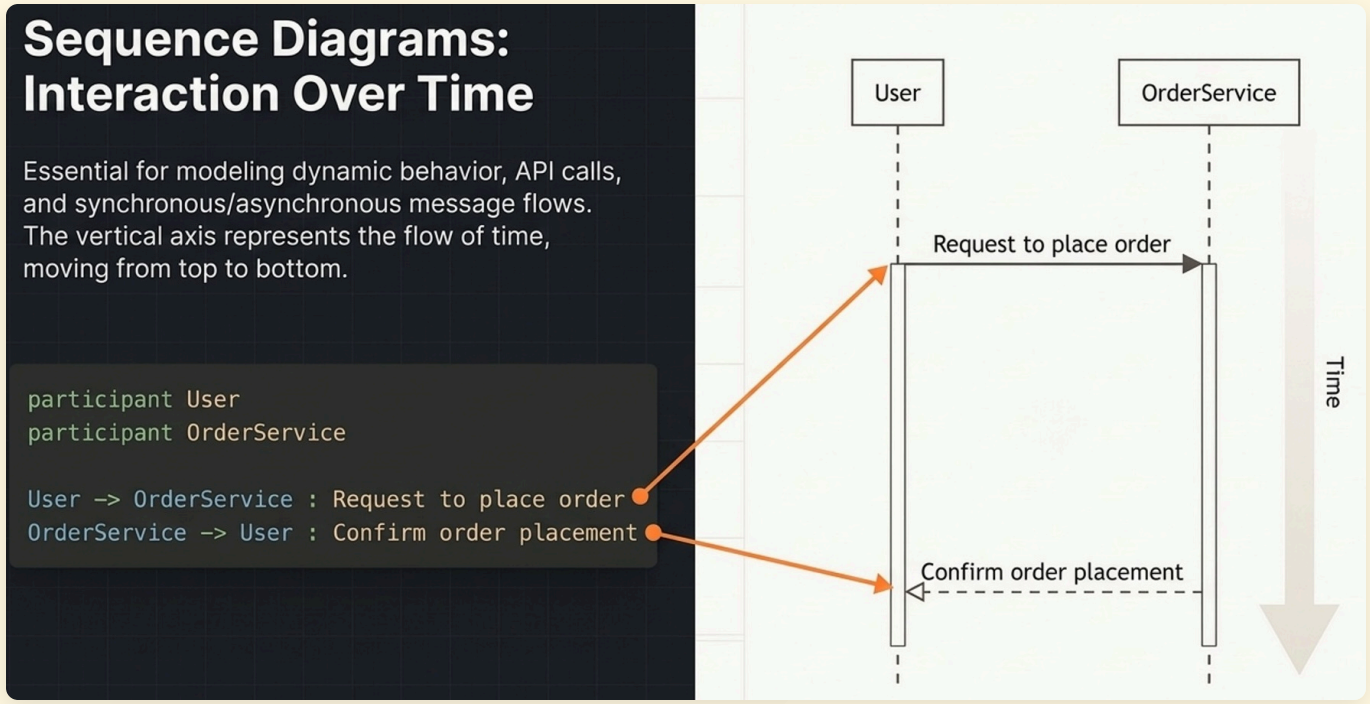
類別關聯語法對照表：直觀象形符號 (Relationship Matrix)

Relationship Type	Definition	PlantUML Syntax	Visual Output
Inheritance	Subclass inherits attributes/operations of a superclass.	`<|--`	
Association	One class uses the services of another.	`--`	
Aggregation	A whole/part relationship where the part can exist independently.	`o--`	
Composition	Strong ownership; the part cannot exist without the whole.	`*--`	

易記且符合視覺形狀的關聯連線符號：

- **繼承 (<|--)：**
  - 子類別繼承屬性與方法（空心尖三角形 `<|` 指向父類別）。
- **關聯 (--)：**
  - 結構化參照關聯，以單純實線連接兩類別。
- **聚合 (o--)：**
  - 共享「擁有」關係；部分可獨立存在（空心菱形 `o`）。
- **組合 (*--)：**
  - 強烈整體與部分歸屬；生命週期同生共死（實心菱形 `*`）。
- **象形助記設計：** 符號外觀完全對映標準 UML 圖例形狀。

循序圖語法：純文字描繪時間順序 (Sequence Syntax)



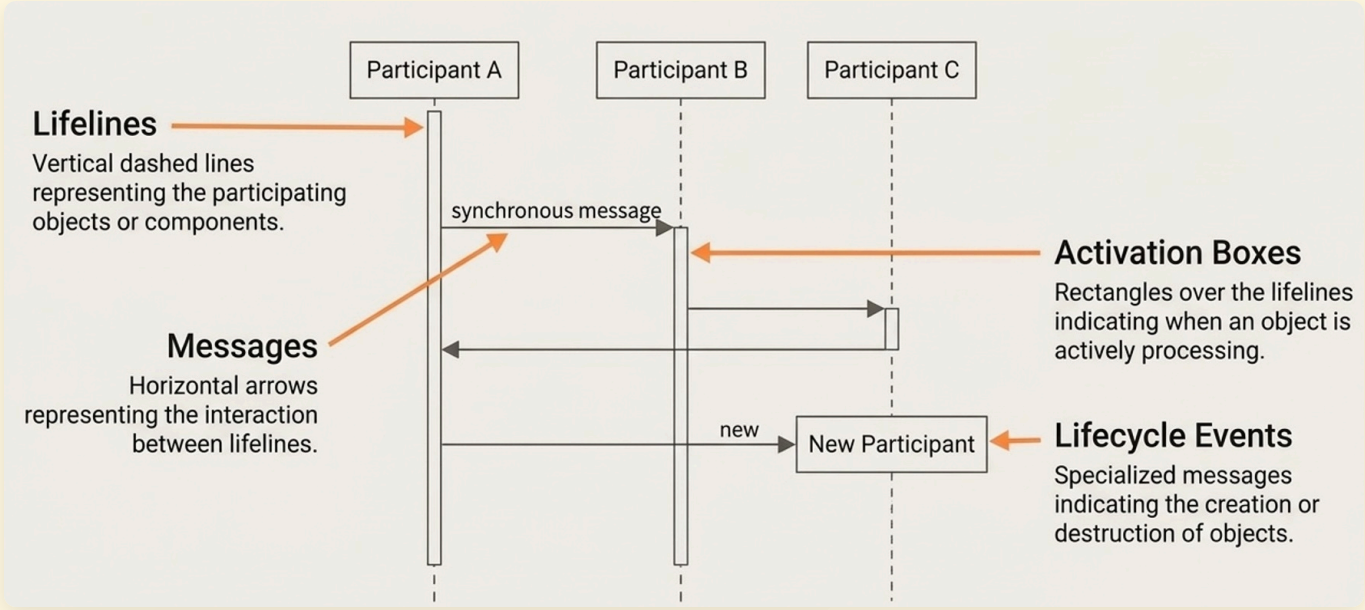
以純文字捕捉動態訊息時序流：

- **時序互動核心規則：**
  - 時間軸由上至下垂直流動。
  - 參與者使用 `participant Name` 宣告。
  - 實線箭頭 `->` 代表同步請求呼叫。
  - 虛線箭頭 `-->` 代表回傳回應結果。
- **PlantUML 腳本範例：**

```
participant User
participant OrderService
User -> OrderService : Request to place order
OrderService --> User : Confirm order placement
```

- **自動對齊：** 生命線、啟動條與箭頭水平對齊全由引擎運算。

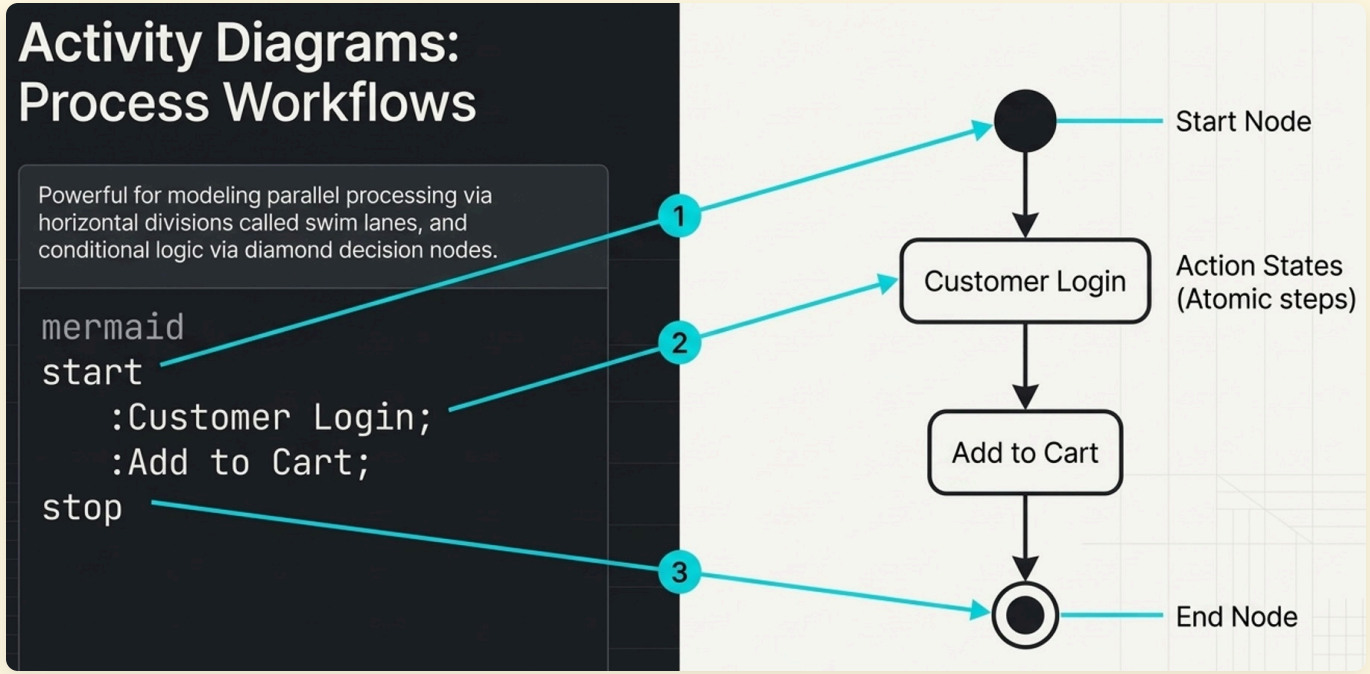
循序圖四大核心語法要素剖析 (Sequence Anatomy)



PlantUML 循序圖之核心視覺骨架：

- **1. 生命線 (Lifelines)：**
  - 垂直虛線，代表協同互動之物件或微服務參與者。
- **2. 訊息箭頭 (Messages)：**
  - 水平通訊箭頭（`->` 同步請求、`->>` 非同步呼叫）。
- **3. 啟動長條 (Activation Boxes)：**
  - 生命線上的細長矩形（`activate` / `deactivate`），標示運算活躍期。
- **4. 生命週期事件 (Lifecycle Events)：**
  - 專屬語法（`create` / `destroy`），標示實體動態建立或銷毀。
- **核心價值：** 簡短文字代碼產出像素級精確的互動圖。

活動圖與泳道語法：業務流程建模 (Activity Syntax)



業務流程與工作流建模語法：

- **流程控制節點標記：**
  - `start`：自動渲染黑色實心初始起點節點。
  - `:動作名稱;`：以冒號與分號包覆宣告原子動作狀態框。
  - `stop`：自動渲染牛眼雙圓之結束終點節點。
- **PlantUML 腳本範例：**

```
start
:Customer Login;
:Add to Cart;
stop
```

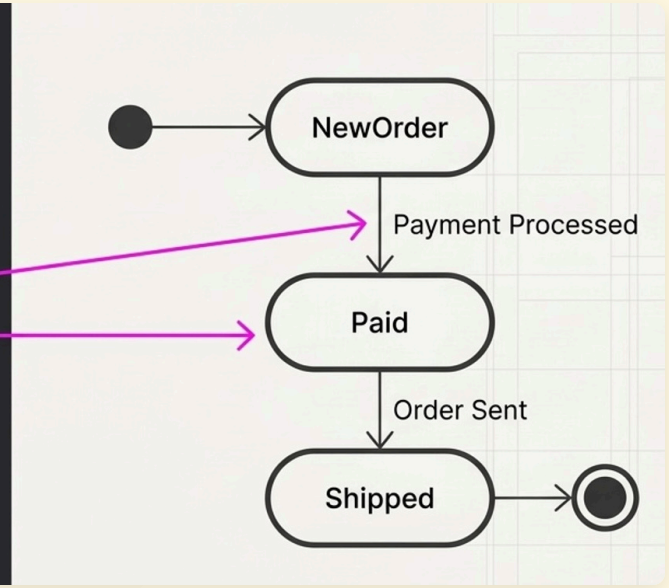
- **進階流程控制支援：**
  - 以 `if (...) then (...) else (...) endif` 宣告決策菱形。
  - 以 `|泳道名稱|` 分割多角色職責泳道。

狀態圖語法：單一物件反應型生命週期 (State Syntax)

While Activity diagrams show multi-actor processes, State diagrams focus strictly on the lifecycle of a single object in response to external events.

```
[*] --> NewOrder
NewOrder --> Paid : Payment Processed
Paid --> Shipped : Order Sent
Shipped --> [*]
```

Format: `StateA --> StateB : Event String`



單一物件反應型生命週期建模：

- **標準狀態轉移語法格式：**
  - 格式：`StateA --> StateB : 事件觸發說明`
- **PlantUML 腳本範例：**

```
[*] --> NewOrder
NewOrder --> Paid : Payment Processed
Paid --> Shipped : Order Sent
Shipped --> [*]
```

- **生命週期標記：**
  - `[*]`：同時代表起始偽狀態與結束終止狀態。
- **核心價值：** 數秒內建立有限狀態機，具備清晰躍遷事件標註。

觀念檢核測驗 8 (CCQ 8)

在軟體架構工程實踐中，相較於傳統以滑鼠手動拖拉圖框的二進位繪圖軟體（如 Visio 或特定商業 CASE 工具），採用 **PlantUML 宣告式文字塑模 (Code-as-Architecture)** 的最核心工程優勢為何？

- **A.** 能保證自動生成 100% 無 Bug 的 Java 實作程式碼，無需單元測試。
- **B.** 可以在純文字中編寫，輕鬆納入 Git 進行精確的行級版本控制、Pull Request 審查與 CI/CD 自動編譯。
- **C.** 完全廢除物件導向原則，讓系統架構不再需要考慮類別或介面。
- **D.** 限制只能產出循序圖，防止工程師建立過多無意義的圖表。

4.9 概念複習與統整

> *"模型是軟體工程的世界語；精通模型，方能洞見架構的真實靈魂。"*

核心觀念統整：互動填空小測驗

測試你對本章核心架構概念的掌握度：

1. 於 Rational Software 攜手催生 UML 標準化的「UML 三巨頭」為 Grady Booch、Jim Rumbaugh 與 ＿＿。
2. 系統塑模的四大基礎核心視角為外部視角、＿＿ 視角、結構視角與行為視角。
3. 在使用案例模型中，多個使用案例強制共用的必要子程序應使用 ＿＿ 關聯。
4. 徹底分離使用者介面、業務邏輯與資料庫持久實體的架構模式稱為 ＿＿ 模式。
5. 在 UML 類別圖中，實心黑色菱形 (◆) 代表 ＿＿ 關聯，部分與整體生命週期緊密綁定連帶銷毀。
6. 在 UML 類別圖中，空心菱形 (◇) 代表 ＿＿ 關聯，部分具備獨立生存的生命週期。
7. UML 狀態機圖中，轉換標籤的國際標準語法為：事件觸發 [＿＿] / 執行動作。

經典文獻與延伸閱讀 (References)

- **權威經典教科書與國際標準：**
  - Sommerville, I. (2016). *Software Engineering* (10th ed.). Chapter 5: System Modeling. Pearson.
  - Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
  - Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
  - Cockburn, A. (2000). *Writing Effective Use Cases*. Addison-Wesley.
  - Object Management Group (OMG). (2017). *OMG Unified Modeling Language (OMG UML) Specification*, Version 2.5.1.
- **現代宣告式文字塑模資源：**
  - PlantUML 官方標準網站：plantuml.com
  - Mermaid.js 宣告式圖表手冊：mermaid.js.org